PYTHON MACHINE LEARNING · ADVANCED MACHINE LEARNING · COMPLETE ANSWER KEY

Advanced Machine Learning

Every self-check item answered and explained, plus reference notes for each lab, for all 13 sessions. Use for grading and reteaching. Confidential instructor material.

13 sessions | 156 self-check items + lab solutions | Confidential · do not distribute

CONTENTS

S01 **Interpreting Models and SHAP**

- 7.1 Interpreting Models
- 7.2 SHAP Values

S02 **Local Explanations, Fairness and K-Means**

- 7.3 LIME and Fairness Checks
- 8.1 K-Means Clustering

S03 **Choosing k and Hierarchical Clustering**

- 8.2 Choosing k and Customer Segments
- 8.3 Hierarchical Clustering

S04 **Density-Based Clustering and Gaussian Mixtures**

- 8.4 Density-Based Clustering
- 8.5 Gaussian Mixture Models

S05 **Anomaly Detection and PCA**

- 8.6 Anomaly Detection
- 9.1 Principal Component Analysis

S06 **LDA, t-SNE and UMAP**

- 9.2 Linear Discriminant Analysis
- 9.3 t-SNE and UMAP

S07 **Neural Networks and PyTorch**

- 10.1 Neural Networks from Scratch
- 10.2 PyTorch Fundamentals

S08 **CNNs and Sequence Models**

10.3 Convolutional Neural Networks

10.4 Sequence Models

S09 **Transfer Learning and Text Classification**

10.5 Transfer Learning

11.1 Text Classification

S10 **Recommendations and Reproducibility**

11.2 Recommendation Systems

12.1 Reproducibility and Experiment Tracking

S11 **Serving, Monitoring and the Churn Case Study**

12.2 Serving and Monitoring Models

13.1 Case Study: Predicting Churn

S12 **Capstone Kickoff and Studio I**

13.2 Capstone Project

S13 **Capstone Studio II and Showcase**

13.2 Capstone Project

INSTRUCTOR KIT · ANSWER KEY

Session 1: Interpreting Models and SHAP

Answers and reasoning for every Session 1 item: the Foundations refresher, both self-checks, the predict-then-run items, the lab reference outputs, the lesson pages' own quizzes and the exit ticket.

Session 1 | 12 self-check items | Confidential

START HERE · FOUNDATIONS REFRESHER

Not graded. The reasoning for each answer also says where the convention comes back in this kit.

D1

What does this print?

```
import pandas as pd
from sklearn.model_selection import train_test_split
customers = pd.read_csv("data/customers_churn.csv")
train, test = train_test_split(customers, test_size=0.25, stratify=customers["churn
print(len(train), len(test), test["customer_id"].iloc[0])
```

A) `4500 1500 C100001`

B) `4500 1500 C102112`

C) `1500 4500 C102112`

D) `4800 1200 C102112`

> **Why:** A quarter of 6,000 is 1,500 test customers; the shuffle with `random_state=42` puts C102112 first. Every Kit B lesson on churn starts from this split, so the same first test ID means the same rows as the lesson pages. **Skill:** the canonical churn split. **Review if missed:** Foundations Lessons 1.1 and 1.2.

D2

What does this print?

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
pipe = Pipeline([("prep", StandardScaler()), ("model", LogisticRegression())])
print(list(pipe.named_steps), type(pipe["model"]).__name__)
```

A) `['standardscaler', 'logisticregression'] LogisticRegression`

B) `['prep', 'model'] Pipeline`

C) `KeyError: 'model'`

D) `['prep', 'model'] LogisticRegression`

> **Why:** `Pipeline` keeps the names you give it, and `pipe["model"]` returns that step. Lesson 7.1 reads coefficients from the last step (`logreg[-1].coef_`) and scales from `logreg[1].scale_`: the same indexing. Automatic lowercase names come from `make_pipeline`, not from `Pipeline`. **Skill:** named pipelines. **Review if missed:** Foundations Lesson 2.2.

D3

What does this print?

```
import pandas as pd
from sklearn.model_selection import StratifiedKFold, train_test_split
customers = pd.read_csv("data/customers_churn.csv")
train, test = train_test_split(customers, test_size=0.25, stratify=customers["churn
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
y_train = train["churned"]
print([len(val) for _, val in cv.split(y_train, y_train)])
```

A) `[900, 900, 900, 900, 900]`

B) `[3600, 3600, 3600, 3600, 3600]`

C) `[1125, 1125, 1125, 1125]`

D) `[300, 300, 300, 300, 300]`

> **Why:** Five folds of the 4,500 training customers: each validation fold holds 900. The test set's 1,500 never enter cross-validation. Lesson 1.4's tuned threshold is chosen on these folds. **Skill:** cross-validation on training rows. **Review if missed:** Foundations Lesson 1.3.

D4

What does this print?

```
from sklearn.metrics import recall_score, precision_score
y_true = [1, 1, 1, 0, 0, 0, 0, 0]
y_pred = [1, 0, 0, 1, 0, 0, 0, 0]
print(round(recall_score(y_true, y_pred), 3), precision_score(y_true, y_pred))
```

A) `0.5 0.333`

B) `0.75 0.75`

C) `0.333 0.5`

D) `0.125 0.5`

> **Why:** Three real positives, one of them caught: recall 1/3. Two flags, one of them right: precision 1/2. 0.75 is accuracy (6 of 8), which a rare class makes easy. Lesson 7.3's fairness audit compares recall and precision by age group. **Skill:** recall and precision. **Review if missed:** Foundations Lesson 1.4.

D5

What does this print?

```
from sklearn.metrics import roc_auc_score
print(roc_auc_score([0, 0, 1, 1], [0.1, 0.4, 0.35, 0.8]))
```

A) `0.5`

B) `0.75`

C) `1.0`

D) `0.25`

> **Why:** AUC is the share of (positive, negative) pairs ranked the right way round: of the four pairs, only (0.35 vs 0.4) is wrong, so 3/4. Permutation importance in Lesson 7.1 measures drops in exactly this number. **Skill:** ROC AUC as ranking. **Review if missed:** Foundations Lesson 1.4.

D6

What does this print?

```
import numpy as np
y = np.array([1, 0, 1, 0, 0, 1])
p = np.array([0.9, 0.3, 0.2, 0.6, 0.1, 0.4])
flags = p >= 0.25                              # $15 per offer, $60 per churner rea
tp = (flags & (y == 1)).sum()
print(flags.sum(), tp, 60 * tp - 15 * flags.sum())
```

A) `4 2 120`

B) `3 3 135`

C) `6 3 90`

D) `4 2 60`

> **Why:** Four probabilities reach the 0.25 break-even (15 / 60); two of those four are churners, so 2 x $60 - 4 x $15 = $60. Lesson 7.3 flags the top 20% instead; Session 3's segment economics reuses the $15 / $60 numbers. **Skill:** campaign economics and a threshold. **Review if missed:** Foundations Lesson 1.4.

7.1 · INTERPRETING MODELS: SELF-CHECK

Q1

In a scaled logistic regression, `days_since_last_order` has a coefficient of 0.89. Which sentence is right?

A) One more day without an order raises the churn probability by 0.89

B) The model is 89% sure that recency causes churn

C) One more standard deviation of days without an order multiplies the odds of churn by about 2.4, other features held fixed

D) One more standard deviation of days without an order raises the churn probability by 2.4 points

Why: Coefficients of a scaled logistic regression are changes in log-odds per standard deviation; e^0.89 is about 2.43, an odds ratio. Lesson 7.1 then divides by the scaler's standard deviation (87.2 days) to speak per 30 days: x1.36.

A is wrong: Coefficients act on the log-odds, not the probability, and per standard deviation, not per day.

B is wrong: A coefficient measures association inside the model; it is not a probability and not a causal claim.

D is wrong: Odds ratios multiply odds; they do not add percentage points to a probability.

Q2

What does this print?

```
import numpy as np
import pandas as pd
coefs = pd.Series([0.40, -0.69], index=["support_tickets_12m", "app_user"])
print(np.exp(coefs).round(2).tolist())
```

`[1.49, 0.5]`

Why: `np.exp` turns each coefficient into an odds ratio: e^0.40 is about 1.49 (each standard deviation more tickets multiplies the odds by 1.5), e^-0.69 is about 0.50 (the odds halve). A coefficient of 0 would give 1.0, no change.

Q3

You add a column of pure random noise to the churn data. The forest's `feature_importances_` ranks it sixth, above `return_rate` and `app_user`; permutation importance on the test set gives it 0.005. What follows?

A) The noise column must hold a little real churn signal

B) Impurity importance cannot be trusted for the ranking: it is measured on training data and rewards columns the trees can memorize with

C) `app_user` is useless and can be dropped

D) Permutation importance is broken for random forests

Why: Impurity importance (MDI) is computed during training and favors many-valued columns that offer split points to overfit with. Permutation importance on held-out data puts the noise near zero, where it belongs. A noise column is a cheap sanity check for any importance you plan to report.
A is wrong: The noise was drawn independently of everything; it cannot carry churn signal.
C is wrong: `app_user` matters on test data (0.009 for the forest, 0.019 for boosting); impurity ranks it low because a 0/1 column offers only one split.
D is wrong: It is the method that got it right here.

Q4

In Lab 7.1 A, shuffling `hour` costs the fraud model 0.074 of average precision but nothing (-0.005) of ROC AUC. What is the best reading?

A) The hour column is broken and should be removed

B) Average precision is a buggy metric for permutation importance

C) The model does not use the hour at all

D) Importance depends on the metric: the hour lifts a few night frauds to the very top of the list, which average precision rewards and the whole-ranking AUC hardly notices

Why: Permutation importance always answers "how much does THIS score depend on the column?". The review team works from the top of the list, so the metric that matches their decision (average precision) is the one to report.
A is wrong: The column carries a real signal: partial dependence shows night payments at 0.017 against 0.007 by day.
B is wrong: Average precision is a standard metric for rare positives (Lesson 1.4).
C is wrong: If the model ignored it, no metric would drop.

Q5

What does this print?

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.inspection import partial_dependence
X = pd.DataFrame({"a": [0.0, 1.0, 2.0, 3.0], "b": [0.0, 0.0, 4.0, 4.0]})
model = LinearRegression().fit(X, X["a"] + X["b"])       # learns y = a + b exactly
res = partial_dependence(model, X, ["a"], method="brute", custom_values={"a": [0.0,
print(res["average"][0].round(2))
```

`[ 2. 12.]`

> **Why:** Partial dependence sets column `a` to each grid value for EVERY row, predicts and averages. The model is y = a + b, and the average of `b` over the four rows is 2, so the averages are 0 + 2 and 10 + 2. Nothing is retrained and no real customer needs to have a = 10.

Q6

Which sentence belongs in the churn-drivers briefing?

A) Customers with long gaps since their last order are predicted to churn more; the risk jumps after about four months

B) Long gaps since the last order cause churn, so emailing customers at three months will cut churn

C) Recency is the cause of 14% of churn

D) The app reduces churn by half, so the app campaign will work

> **Why:** It says what was measured: what the model relies on and how its predictions move. The other three turn associations into causes, which only an experiment can show.
>
> **B** is wrong: A causal claim and a prescription from a model of observational data.
>
> **C** is wrong: 0.14 is the drop in AUC when the column is shuffled, not a share of churn, and not causal.
>
> **D** is wrong: The 0.55 odds ratio describes app users in the data; engaged customers may install the app AND stay for the same reason.

7.1 · INTERPRETING MODELS: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
import numpy as np
print(round(np.exp(0.69), 2), round(np.exp(-0.69), 2), np.exp(0.0))
```

```
1.99 0.5 1.0
```

Why: `np.exp(0.69)` is about 2 (the odds double), `np.exp(-0.69)` about 0.5 (they halve), `np.exp(0)` exactly 1 (no change). This is the odds-ratio rule of thumb from Lesson 7.1.

P2

Predict the output, then run it.

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier
from sklearn.inspection import permutation_importance
rng = np.random.default_rng(0)
X = rng.normal(size=(200, 2))
y = (X[:, 0] > 0).astype(int)                # only column 0 matters
tree = DecisionTreeClassifier(max_depth=1, random_state=0).fit(X, y)
r = permutation_importance(tree, X, y, n_repeats=5, random_state=0)
print(r.importances_mean.round(2))
```

```
[0.49 0. ]
```

Why: The tree splits on column 0 only, so shuffling column 0 destroys about half its accuracy (it falls to a coin toss on a balanced target), and shuffling column 1 changes nothing: exactly 0. Permutation importance measures what the model uses.

7.2 · SHAP VALUES: SELF-CHECK

Q7

Lab 7.2 B's fraud rule never looks at `gift_card`. Explained against a typical payment, what Shapley value does `gift_card` get, and why?

A) A quarter of the total, because the four features share the gap equally

B) 0, because adding it never changes the score in any arrival order

C) A negative value, to balance the interaction bonus

D) It cannot be computed: Shapley values need every feature to matter

> **Why:** A feature's Shapley value is its average contribution over all arrival orders. If its arrival never changes the score, every contribution is 0, so its share is 0 (the "null player" rule). The lab prints `gift_card: 0.0`.
> **A** is wrong: Equal splits only happen for features that play identical roles; the interaction bonus is split 0.15 / 0.15 between night and new device for that reason.
> **C** is wrong: Shares are not invented to balance anything; they add up to the gap exactly as computed.
> **D** is wrong: Unused features are allowed and simply get 0.

Q8

What does this print?

```
import numpy as np
from scipy.special import expit
base = -2.0
shares = np.array([1.5, 0.7, -0.2])
print(round(base + shares.sum(), 2), round(expit(base + shares.sum()), 3))
```

`0.0 0.5`

> **Why:** SHAP values for a boosting classifier are log-odds: base -2.0 plus 1.5 + 0.7 - 0.2 gives 0.0 log-odds, and the logistic function `expit` turns 0 log-odds into a probability of exactly 0.5.

Q9

For the fraud model, `device_new` tops the mean |SHAP| ranking (0.247) while `velocity_24h` tops permutation importance. How can both be right?

A) One of the two calculations has a bug

B) SHAP ranks only the riskiest payments

C) Mean |SHAP| measures how much a feature moves predictions; permutation importance measures how much the score depends on it, and the two need not agree

D) Permutation importance is always more correct

Why: A feature can move many predictions a lot (a new device shifts almost every payment's log-odds) and still add less ranking skill than another (velocity separates the frauds at the top). Different questions, both legitimate; report the one that matches the decision.

A is wrong: Both are exact for what they measure; Lesson 7.2 sees the same kind of disagreement on churn (tenure and discounts swap places).

B is wrong: Mean |SHAP| here averages over all 15,000 test payments.

D is wrong: Neither is "more correct": they answer different questions.

Q10

A random forest's explanation has `values.shape` equal to `(200, 11, 2)`. Which object do you pass to `shap.plots.waterfall` for the first customer's churn class?

A) `forest_sv[0]`

B) `forest_sv[:, :, 0]`

C) `forest_sv.values.sum()`

D) `forest_sv[0, :, 1]`

Why: A random forest's SHAP values are in probability units and come once per class, so the last axis is the class. Row 0, every feature, class 1 (churned) is one explanation the waterfall can draw.

A is wrong: That is still an (11, 2) matrix: the waterfall raises "can currently only plot a single explanation".

B is wrong: Every customer, class 0 (stayed): the wrong class and too many rows.

C is wrong: A single number, not an explanation.

Q11

You explain the logistic-regression pipeline with a PermutationExplainer and swap the background from 100 random customers to 100 Gold-tier customers. What changes?

A) The base value and the shares change; the customer's predicted probability does not

B) Nothing: Shapley values do not depend on the background

C) The predicted probability changes, because the model is refitted on Gold customers

D) The shares stay the same but no longer add up

> **Why:** The background defines "a typical value" when a feature is left out, so it sets the base value and therefore the gap that the shares split. The model is never refitted, so the prediction is the same; additivity still holds. Lab 7.2 B shows the same with a night-time reference: night's share drops to 0 and new device's rises to 0.3.
> **B** is wrong: The background is half of the definition: "compared with whom?"
> **C** is wrong: Explainers never refit the model.
> **D** is wrong: Additivity holds for any background; that is what makes them Shapley values.

Q12

What does this print?

```
phrases = {"discount_share": "{:.0%} of spend on discounted items",
           "days_since_last_order": "no order in {:.0f} days"}
print(phrases["discount_share"].format(0.432), "|", phrases["days_since_last_order"
```

`43% of spend on discounted items | no order in 364 days`

> **Why:** `{:.0%}` multiplies by 100 and adds a percent sign (0.432 becomes 43%), and `{:.0f}` prints a whole number. Reason codes are templates filled with the customer's real value: neutral, factual, about the account.

7.2 · SHAP VALUES: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
from scipy.special import expit
print(expit(0.0), round(expit(-2.951), 3), round(expit(3.55), 3))
```

```
0.5 0.05 0.972
```

Why: `expit(0)` is 0.5; the churn model's base value -2.951 squashes to 0.050; C104051's end point 3.55 squashes to 0.972. The same numbers as demo Steps 5 and 7.

P4

Predict the output, then run it.

```
import itertools
def score(a, b):
    return 0.1 + (0.4 if a and b else 0.0)    # a bonus only when BOTH are present
shares = {"a": 0.0, "b": 0.0}
for order in itertools.permutations(["a", "b"]):
    present = {"a": 0, "b": 0}
    for name in order:
        before = score(**present)
        present[name] = 1
        shares[name] += (score(**present) - before) / 2
print(shares)
```

```
{'a': 0.2, 'b': 0.2}
```

Why: The bonus appears only when the second feature arrives, and each feature is second in one of the two orders, so each gets half of 0.4. The 0.1 starting score is the base value and is not shared out.

LAB REFERENCE SOLUTIONS

◆ ENVIRONMENT CHECK: LABS/MODULE-01/L7_0-ORIENTATION/CHECK_ENVIRONMENT.PY

A tool, not an exercise. On a ready machine it prints the versions, the data and image folders, and ends with READY (paths shortened here):

```
Advanced Machine Learning: environment check
================================================================
1. Python
  [ok]   Python 3.12.14         Linux x86_64
       running from: /path/to/pinecrest-ml/.venv/bin/python
2. Virtual environment
  [ok]   environment active     .venv
3. Course packages
  [ok]   numpy                  2.5.4
  [ok]   pandas                 3.0.6
  [ok]   matplotlib             3.11.2
  [ok]   scikit-learn           1.9.1
  [ok]   scipy                  1.18.1
  [ok]   joblib                 1.6.0
  [ok]   skops                  0.16.0
  [ok]   xgboost                3.4.1
  [ok]   lightgbm               4.7.0
  [ok]   shap                   0.53.0
  [ok]   lime                   0.2.0.1
  [ok]   umap-learn             0.5.12
  [ok]   torch                  2.14.1+cpu
  [ok]   torchvision            0.29.1+cpu
  [ok]   mlflow                 3.17.1
  [ok]   fastapi                0.143.2
  [ok]   uvicorn                0.54.0
  [ok]   httpx                  0.28.1
  [ok]   imbalanced-learn       0.14.2
  [ok]   optuna                 5.0.0
  [ok]   seaborn                0.13.2
  [ok]   jupyterlab             4.6.4
  [ok]   PyTorch device         CPU only (expected)
4. Course data
  [ok]   data folder            /path/to/pinecrest-ml/data
  [ok]   7 data files           all present, row counts as expected
5. Image data and pretrained weights (Lesson 9.3 and Module 10)
  [ok]   Fashion-MNIST          /path/to/pinecrest-ml/data/FashionMNIST
  [ok]   EuroSAT                /path/to/pinecrest-ml/data/eurosat
  [ok]   ResNet18               /path/to/torch/hub/checkpoints
  [ok]   MobileNet V3 Small     /path/to/torch/hub/checkpoints
================================================================
READY for Advanced Machine Learning
```

Without the image data it lists four `[todo]` lines and ends with `READY for Sessions 1 to 4` and the `--download` instruction; with a missing package, no environment or no data folder it prints `[FIX]` lines and a numbered NOT READY list. `--download` fetches Fashion-MNIST and EuroSAT into the course `data/` folder (where the lessons' `root="data"` looks) and the two pretrained networks into PyTorch's cache. With `LABS_OFFLINE=1` it never downloads.

- FOUNDATIONS REFRESHER PRACTICE: LABS/MODULE-01/L7_0-ORIENTATION/DIAGNOSTIC_STARTER.PY → DIAGNOSTIC_SOLUTION.PY

Six functions, one per refresher question; the checker compares each with a tiny example, then the last cell runs them on the real churn data. The solution prints:

```
PASS     D1 canonical split     got (15, 5, 1)
PASS     D2 named pipeline      got ['prep', 'model']
PASS     D3 cross-validation    got (1.0, 0.0)
PASS     D4 recall, precision   got (0.667, 0.667)
PASS     D5 campaign profit     got 75
PASS     D6 best threshold      got 0.25
6 of 6 checks pass.

split: 4500 / 1500, first test customer C102112
logistic regression: CV ROC AUC 0.833 +/- 0.026
threshold chosen on out-of-fold training probabilities: 0.21 (break-even 15/60 = 0.25)
test set, once: 385 offers, recall 0.641, precision 0.413, profit $3,765
```

Reading the last cell: the threshold is chosen on out-of-fold training probabilities (0.21, close to the 0.25 break-even) and the test set is used once. Common sticking points: `Pipeline` given the steps without names (a TypeError), `cross_val_score` on the whole table instead of the training rows, a threshold chosen on the test set (it "wins" more on the test set and means nothing).

- LAB 7.1 A: LABS/MODULE-01/L7_1-INTERPRETING-MODELS/FRAUD_DRIVERS_STARTER.PY → FRAUD_DRIVERS_SOLUTION.PY

The solution prints:

```
test ROC AUC 0.863   average precision 0.513   (a random ranking: AP 0.009)
                          ap_drop  ap_std
velocity_24h                0.242   0.012
account_age_days            0.120   0.015
device_new                  0.116   0.011
amount                      0.102   0.017
hour                        0.074   0.008
card_attempts               0.048   0.003
billing_shipping_mismatch   0.024   0.004
country_mismatch            0.019   0.003
items                       0.010   0.002
                 ap_drop  auc_drop
velocity_24h       0.242     0.046
device_new         0.116     0.038
hour               0.074    -0.005
country_mismatch   0.019    -0.000
average predicted fraud by hour: [0.018, 0.017, 0.017, 0.017, 0.017, 0.017, 0.009, 0.0(
night (0-5) 0.0174   day (6-23) 0.0072
known device (13,552 payments)  hour 2, 10, 15, 21 -> [0.006, 0.004, 0.004, 0.005]
new device   (1,448 payments)  hour 2, 10, 15, 21 -> [0.122, 0.028, 0.028, 0.027]
           payments  actual  predicted
band
night           531  0.0640     0.0589
morning        3621  0.0058     0.0070
afternoon      5636  0.0071     0.0068
evening        5212  0.0069     0.0050
```

What each TODO teaches: (1) by average precision, card velocity leads (0.242), then account age, a new device and the amount; (2) by ROC AUC every drop is smaller and `hour` vanishes (-0.005): importance belongs to a metric, and the review team's metric is the top-of-list one; (3) the hour dial: night payments 0.0174 against 0.0072 by day, a step at 6:00, not a slope; (4) the interaction: for known devices the hour hardly matters (0.006 at 2 a.m.), for new devices it is everything (0.122 at 2 a.m., 0.028 by day); (5) the data agree: 6.40% real fraud at night (5.89% predicted) against 0.58 to 0.71% in the daytime bands.

- LAB 7.1 B: LABS/MODULE-01/L7_1-INTERPRETING-MODELS/PIPELINE_IMPORTANCE_STARTER.PY → PIPELINE_IMPORTANCE_SOLUTION.PY

The solution prints:

```
logistic regression pipeline: test ROC AUC 0.828
40 model columns from 16 business columns
lowest odds ratios : {'scale__spend_12m': 0.65, 'onehot__home_store_Asheville': 0.66,
highest odds ratios: {'scale__support_tickets_12m': 1.5, 'scale__discount_share': 1.99,
home_store: {'onehot__home_store_Asheville': 0.66, 'onehot__home_store_Bend': 0.98, 'or
region Southeast: 1.32
loyalty_tier (per step Basic -> Silver -> Gold, scaled): 1.48
real churn rate by tier: {'Basic': 0.217, 'Gold': 0.065, 'Silver': 0.129}
boosting pipeline: test ROC AUC 0.863
top 4: {'days_since_last_order': 0.143, 'discount_share': 0.087, 'tenure_months': 0.062
the five descriptive columns: {'home_store': -0.002, 'region': -0.0, 'age_group': -0.00
descriptive columns shuffled together: AUC drop -0.002 +/- 0.002
```

Reading it: (1) `get_feature_names_out()` names all 40 model columns with their transformer prefix; (2) two coefficients that mislead: Asheville is the only Southeast store, so `home_store_Asheville` (0.66) and `region_Southeast` (1.32) share one fact and split its credit in opposite directions; `loyalty_tier` gets 1.48 although Gold churns least (6.5%), because tier is computed from spend and tenure, which are already in the model: "a Gold customer with a Basic customer's spend" does not exist; (3) permutation importance on the RAW `X_test` shuffles each business column once, however many one-hot columns it became: all five descriptive columns are about 0; (4) shuffled together they still cost nothing (-0.002 +/- 0.002). TODO 5's answer: no. The model not USING age says nothing about how its errors fall on age groups: Lesson 7.3 measures that.

- HANDOUT: LABS/MODULE-01/L7_1-INTERPRETING-MODELS/DEBUG_INTERPRET.PY (DELIBERATELY BROKEN)

1. `AttributeError: 'Pipeline' object has no attribute 'coef_'` → `logreg[-1].coef_`.
2. No error: the script prints the log-odds (`0.31`) as an "odds ratio". Fix: wrap it in `np.exp(...)` → 1.36.
3. `ValueError: The column 0 contains integer data ...` from `partial_dependence` → build X with `.astype(float)` (and refit on it).
4. No error: `permutation_importance` on `X_train, y_train` prints `days_since_last_order (0.151)` as if it were held-out. Fix: `X_test, y_test` → `days_since_last_order (0.140)`.

Fixed, it prints the three lines in its docstring: 1.36; [0.077, 0.214, 0.443]; days_since_last_order (0.140).

- LAB 7.2 A: LABS/MODULE-01/L7_2-SHAP-VALUES/FRAUD_REASONS_STARTER.PY → FRAUD_REASONS_SOLUTION.PY

The solution prints:

```
35000 15000 fraud rate 0.0088
(15000, 9) base value -5.840 log-odds = probability 0.0029
largest additivity gap: 2.0e-14
mean |SHAP|: {'device_new': 0.247, 'velocity_24h': 0.235, 'account_age_days': 0.211, 'a
T031633 0.99 ['5 payments on this card in 24 hours', 'amount $836', 'account only 10 da
T025327 0.988 ['5 payments on this card in 24 hours', 'amount $1,730', 'account only 50
T009194 0.985 ['4 payments on this card in 24 hours', 'amount $500', 'account only 0 da
top reason across the 25: {'velocity_24h': 19, 'device_new': 5, 'card_attempts': 1}
real frauds in the queue: 25 of 25
new_device     0     1
when
day 6-23   -0.01 -0.05
night 0-5   0.33  1.24
```

Reading it: (1) the base value is -5.84 log-odds, 0.0029 as a probability, below the 0.0088 fraud rate for the same reason as churn's 5% against 16.6%: the squash of the average log-odds is not the average probability; additivity holds to 2e-14; (2) mean |SHAP| puts `device_new` first, although velocity leads permutation importance: a new device moves many predictions somewhat, velocity separates the frauds at the top (self-check Q9); (3, 4) every one of the 25 riskiest payments is a real fraud, 19 led by card velocity; the reasons are neutral facts about the payment; (5) the hour's share is about 0 by day and +0.33 log-odds at night on a known device, +1.24 on a new device: the same interaction Lab 7.1 A found with partial dependence.

- LAB 7.2 B: LABS/MODULE-01/L7_2-SHAP-VALUES/SHAPLEY_BY_HAND_STARTER.PY → SHAPLEY_BY_HAND_SOLUTION.PY

The solution prints:

```
reference score: 0.01   payment score: 0.51
orders tried: 24
Shapley values: {'velocity': 0.2, 'new_device': 0.15, 'night': 0.15, 'gift_card': 0.0}
shap Exact: [[0.2, 0.15, 0.15, 0.0]]
compared with a typical night payment: {'velocity': 0.2, 'new_device': 0.3, 'night': 0.
```

Reading it: four features give 4! = 24 arrival orders. Velocity adds 0.20 whenever it arrives. The 0.30 bonus goes to whichever of night and new device arrives second: each is second in half the orders, so 0.15 each. `gift_card` never changes the score, so it gets 0 (the null-player rule). Against a typical NIGHT payment the reference already has night = 1, so night's arrival never adds anything (0.0) and the new device, whose arrival now always completes the pair, gets the whole 0.30. The total gap is the same 0.50: what changed is the question "compared with whom?", the background.

- HANDOUT: LABS/MODULE-01/L7_2-SHAP-VALUES/DEBUG_SHAP.PY (DELIBERATELY BROKEN)

1. `InvalidModelError: Model type not yet supported by TreeExplainer: <class 'sklearn.pipeline.Pipeline'>` → the explainer was given `logreg` (a pipeline); the script means the boosting model: `shap.TreeExplainer(hgb)`.

2. No error: `proba.mean() + sv.values[i].sum()` prints a "probability" of `6.652`. Fix: `expit(sv.base_values[i] + sv.values[i].sum())` → 0.972.

3. `ValueError` from pandas (the values are 2-dimensional: one column per class) → `forest_sv.values[0, :, 1]`, the churn class.

Fixed, it prints the two lines in its docstring: 0.972 (model says 0.972) and the forest's top reason, days_since_last_order.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 7.1 · INTERPRETING MODELS

1. **B** e^0.65 is about 1.9: one more standard deviation of discount share multiplies the odds of churn by about 1.9, other features held fixed.

2. **C** Impurity importance is computed on training data and favors many-valued features the trees can overfit with.

3. **A** The information is redundant: the model rebuilds spend from orders and order value, so measure the group together.

4. **B** For each of 3, 40 and 90, the average predicted churn when every test customer's tenure is set to that value.

5. **C** An interaction: the effect of tickets depends on another feature (app use), and the average blurs it.

LESSON 7.2 · SHAP VALUES

1. **A** Equally, 0.05 each: each feature completes the pair in half of the arrival orders.

2. **C** The raw score in log-odds (`decision_function`) for that customer.

3. **B** For customers with long gaps, recency pushes the predicted risk up a lot (association, not cause).

4. **A** `shap.PermutationExplainer` with a function that calls the pipeline and a background sample.

5. **C** Spend overlaps with orders and average order value, so the credit is shared across the three.

OPENING: READING THE ROOM

There is no retrieval warm-up in Session 1. The opening's two checks have their answers above: the environment check output (what READY looks like) and the refresher D1 to D6. Learners who end the opening NOT READY leave with the numbered fix list in their workbook's Start Here box; learners at READY for Sessions 1 to 4 have the `--download` line to run at home.

EXIT TICKET

1. **Three tools:** permutation importance = how much the held-out score depends on a feature; partial dependence = how the average prediction moves as one feature changes; neither = what would happen in the world if Pinecrest changed the feature (that needs an experiment).
2. **Waterfall -2.95 → +0.50:** the model says `expit(0.50)` = 0.62. The bars are in log-odds and start at the base value -2.95 (5% as a probability), so they cannot be added to 16.5%, which is a probability (and not the base value either).
3. **Reason code:** "no order in 240 days". The agent must not say "you are leaving because ..." or promise an outcome ("if you order now you will stay"): the reason says why the model flagged the account, not why the customer behaves as they do.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 A or D ("raises the probability by 0.89 / 2.4 points").** Has not separated probability, odds and log-odds. Write the chain on the board: coef → `np.exp` → odds ratio; probability only at the end.
- **Q3 A ("the noise holds signal").** Trusts the tool over the setup. Ask how the noise was made.
- **Q4 C ("the model does not use the hour").** Reads one metric's importance as the truth. Show TODO 3's partial dependence: night is 2.4 times day.
- **Q5 written as `[0. 10.]`.** Forgot that the other features keep their real values; the average of b (2) is added to every grid point.
- **Q8 written as `0.0 0.0` or `2.2 ...`.** Added shares to a probability, or forgot the base value. Base + shares = 0.0 log-odds; `expit(0)` = 0.5.
- **Q11 B ("nothing changes").** Thinks Shapley values are a property of the model alone. Lab 7.2 B's night-owl reference is the counterexample.
- **Exit ticket 2: "0.165 + ...".** The most common answer in the room, and the reason the units get a slide of their own in Session 2's warm-up.

INSTRUCTOR KIT · ANSWER KEY

Session 2: Local Explanations, Fairness and K-Means

Answers and reasoning for every Session 2 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs, the lesson pages' own quizzes and the exit ticket.

Session 2 | 12 self-check items | Confidential

WARM-UP

W1

A scaled logistic regression gives `discount_share` a coefficient of 0.68. Say what that means in one sentence.

One more standard deviation of discount share multiplies the odds of churn by e^0.68, about 2.0, other features held fixed.

W2

Shuffling `days_since_last_order` in the TEST set lowers the boosting model's ROC AUC by 0.140. What does that measure, and why the test set?

How much the model's held-out score depends on that feature (permutation importance); on training rows memorized noise would look important too.

W3

What does this print?

```
import numpy as np
from scipy.special import expit
base = -2.95                    # C104051's waterfall starts here (log-odds)
total_shares = 6.50             # and its 11 SHAP values add up to this
print(round(base + total_shares, 2), round(expit(base + total_shares), 3))
```

`3.55 0.972`: base + shares = 3.55 log-odds; `expit(3.55)` = 0.972, C104051's churn probability.

W4

Why should a briefing never rank features with a forest's `feature_importances_`?

It is measured on training data and favors many-valued columns: in Lesson 7.1 it ranked a column of pure noise sixth.

W5

Which phrase can an agent read aloud: "you are leaving because you stopped buying" or "no order in 240 days"? Why?

"No order in 240 days": a neutral fact about the account. The other turns a model's association into a claim about the person's motives.

7.3 · LIME AND FAIRNESS CHECKS: SELF-CHECK

Q1

In Lab 7.3 A, LIME explains a fraud payment the model scores at 0.990. The explanation has fidelity 0.12 and a surrogate prediction of 0.077. What should you do?

A) Treat it as unreliable: the flat map does not imitate the model near this payment; report SHAP's reasons, or fewer reasons

B) Report it: fidelity only matters for regression models

C) Trust the surrogate: the payment's real risk is 0.077

D) Raise `num_features` until fidelity reaches 0.9, then report every rule

Why: Fidelity (the surrogate's weighted R^2) and the surrogate's own prediction are LIME's built-in honesty checks. Here both fail: almost every random variant of a payment looks legitimate when fraud is 0.88% of the data, so the straight line is fitted to a flat neighborhood. SHAP's TreeExplainer is exact for this model and agrees on velocity as the lead reason.

B is wrong: Fidelity is about how well the surrogate imitates the model, for any kind of model.

C is wrong: The surrogate is an approximation of the model; the model's 0.990 is the prediction being explained.

D is wrong: More features raise the fit a little (Lesson 7.3 Exercise 1) and make the explanation useless for a person.

Q2

What does this print?

```
import numpy as np
width = 0.75 * np.sqrt(4)
print(np.exp(-np.array([0.0, 1.5, 3.0]) ** 2 / width ** 2).round(3))
```

`[1. 0.368 0.018]`

Why: LIME's kernel weight is exp(-distance² / width²). With 4 features the default width is 0.75 × 2 = 1.5: a variant at distance 0 weighs 1, at one width (1.5) e^-1 = 0.368, at two widths e^-4 = 0.018. Variants far from the customer barely count.

Q3

A what-if check sets C104051's days since the last order to 60 and the model still predicts 0.645. What is the best lesson from this?

A) The model is broken: recency is its top driver

B) Getting C104051 to order will not keep them

C) A single top reason hides the rest: discounts and tickets still hold the risk up, and the what-if describes the model, not the customer

D) LIME was wrong to rank recency first

Why: What-if checks ask the model a counterfactual question. The answer shows that "make them order" would not, according to the model, make this customer safe. It says nothing about what the customer would really do: that needs an experiment.

A is wrong: Recency still matters: 364 days gives 0.972, 120 days 0.619.

B is wrong: That is a claim about the world; the what-if only describes the model's answer.

D is wrong: LIME's binned rule (more than 110 days) and the what-if answer different questions.

Q4

Pinecrest's offer is a benefit: a churner who is flagged gets a discount, one who is missed does not. Which comparison across age groups matches that harm?

A) Flag rate (demographic parity)

B) Recall among churners (equal opportunity)

C) Precision among the flagged

D) Accuracy

Why: The harm is a churner who misses out on an offer. Recall asks, for each group's real churners, what share were flagged. Lesson 7.3 compares exactly that (54% to 77%, and a permutation test says the gap is consistent with chance).

A is wrong: Equal flag rates can coexist with very different chances for real churners to be reached.

C is wrong: Precision measures wasted offers within a group: the cost side, not the missed-benefit side.

D is wrong: Accuracy mixes both errors and is dominated by the many non-churners.

Q5

What does this print?

```
import pandas as pd
audit = pd.DataFrame({"group": ["A", "A", "A", "B", "B", "B", "B"],
                      "churned": [1, 1, 0, 1, 1, 1, 0],
                      "flag":    [1, 0, 1, 1, 1, 0, 0]})
churners = audit[audit["churned"] == 1]
print(churners.groupby("group")["flag"].mean().round(2).tolist())
```

`[0.5, 0.67]`

Why: Recall by group is the flag rate among each group's churners only. Group A has two churners, one flagged: 0.5. Group B has three churners, two flagged: 0.67. The non-churner in A who was flagged does not enter the recall at all.

Q6

Lab 7.3 B finds the model over-predicts churn for 45 to 54 year-olds by 2.5 points and under-predicts for 35 to 44 by 4.2 points; 64% of shuffles produce a gap at least as large. What do you report?

A) The model is miscalibrated against older customers and must be retrained per age group

B) The model is proven equally calibrated for every age group

C) Age should be added as a feature to fix the calibration

D) No evidence of a real calibration difference at this sample size; the groups are small, so keep monitoring

> **Why:** Gaps this size are what chance produces with groups of 128 to 309 customers. The honest report has both halves: no evidence of a problem, and not enough data to rule out a moderate one.
> **A** is wrong: A gap that shuffled labels match 64% of the time is not evidence of anything.
> **B** is wrong: Failing to find a difference is not proof that none exists.
> **C** is wrong: Age is used only to audit; feeding it to the model is a shortcut the lesson warns against.

7.3 · LIME AND FAIRNESS CHECKS: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
import numpy as np
width = 0.75 * np.sqrt(11)                    # LIME's default kernel width for 11
distance = np.array([0.0, 2.0, 5.0])          # in standard deviations from the cu
print(np.exp(-distance ** 2 / width ** 2).round(3))
```

`[1. 0.524 0.018]`

> **Why:** Weight = $\exp(-d^2 / \text{width}^2)$ with width $0.75 \times \sqrt{11} = 2.49$: the customer itself weighs 1, a variant two standard deviations away about half, one five away almost nothing.

P2

Predict the output, then run it.

```
import pandas as pd
audit = pd.DataFrame({"age": ["young"] * 4 + ["older"] * 4,
                      "churned": [1, 1, 0, 0, 1, 1, 1, 0],
                      "flag":    [1, 0, 0, 1, 1, 1, 0, 0]})
print(audit[audit["churned"] == 1].groupby("age")["flag"].mean().round(2).to_dict()
```

```
{'older': 0.67, 'young': 0.5}
```

> **Why:** Recall per group counts churners only: the young group's two churners include one flagged (0.5); the older group's three include two flagged (0.67). The young non-churner who was flagged does not count.

8.1 · K-MEANS CLUSTERING: SELF-CHECK

Q7

What does this print?

```
from sklearn.cluster import KMeans
km = KMeans(n_clusters=2, n_init=10, random_state=42).fit([[1], [3], [10], [14]])
print(km.inertia_)
```

```
10.0
```

> **Why:** Two clusters, {1, 3} with center 2 and {10, 14} with center 12. Inertia adds the squared distances to the own center: 1 + 1 + 4 + 4 = 10.

Q8

Lab 8.1 A clusters 50,000 payments on amount, account age, items and card velocity without scaling. The four groups differ only in account age. Why?

A) Account age is the most important fraud signal

B) K-Means sorts by the first column with many values

C) The other three columns have too many ties

D) Distances are dominated by the column with the largest spread: account age varies by about 700 days, items by about 1

> **Why:** K-Means measures straight-line distance, so one day of account age counts as much as one whole item. Logging the skewed columns and standardizing gives every column a say; then the groups become brand-new accounts, busy cards, big baskets and small everyday orders.
> **A** is wrong: K-Means never sees the fraud column; the dominance is about units, not importance.
> **B** is wrong: Column order does not matter to K-Means.
> **C** is wrong: Ties are not the issue; spread is.

Q9

What does `KMeans(n_clusters=5, n_init=10, random_state=42)` do with `n_init=10`?

A) Runs the whole algorithm from ten starts and keeps the run with the lowest inertia

B) Runs ten rounds of assign and move, then stops

C) Tries k from 1 to 10 and keeps the best k

D) Averages the centers of ten runs

> **Why:** One run can end in a local minimum (the lesson's random starts reached inertias from 565 to 8,423). Ten starts with k-means++ seeding make a poor final answer unlikely; the course writes `n_init=10` whenever the result matters.
> **B** is wrong: That is `max_iter` (the number of rounds allowed in one run).
> **C** is wrong: K-Means never chooses k; Lesson 8.2 does that with evidence.
> **D** is wrong: Centers from different runs are not averaged: cluster numbers do not even match between runs.

Q10

The segmentation was fitted on scaled data. Three customers sign up next month. How do you assign them?

A) Fit a new scaler on the three customers, then `predict`

B) Pass the fitted scaler's `transform` of their rows to the fitted model's `predict`

C) Refit K-Means on all customers plus the three

D) `predict` on their raw, unscaled rows

Why: New rows must be measured against the customer base the model learned from: the same fitted scaler, then the same centers. Lesson 8.2 puts both in one pipeline so that `predict` does it in one call.
A is wrong: Three rows scaled against each other land in the wrong segments (Lesson 8.2's Common Mistake 4).
C is wrong: Refitting can move borders and renumber clusters every month.
D is wrong: The centers live in scaled units; raw dollars would all look enormous.

Q11

What does this print?

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler().fit([[2.0], [4.0], [6.0]])
print(scaler.inverse_transform([[1.5]]).round(3))
```

`[[6.449]]`

Why: The scaler learned mean 4 and (population) standard deviation 1.633. `inverse_transform` undoes the scaling: 4 + 1.5 × 1.633 = 6.449. That is how a scaled cluster center becomes a center in real units.

Q12

In Lab 8.1 A, the "Busy cards" group has a fraud rate of 2.13% and holds 243 of the 438 frauds; the clustering never saw `is_fraud`. Which statement is right?

A) Busy cards is a fraud detector: flag the whole group

B) The groups differ in fraud, which shows they capture real behavior, but 97.9% of busy-card payments are legitimate: a cluster describes, it does not detect

C) The clustering leaked the fraud labels

D) The groups are meaningless because none of them is mostly fraud

Why: An outcome the clustering never saw differing between groups is evidence the groups are real. A useful description is not a classifier: flagging 11,401 payments to catch 243 frauds would swamp the review team. Lesson 8.6 detects anomalies properly.
A is wrong: That would flag 11,158 legitimate payments.
C is wrong: `is_fraud` was never in X.
D is wrong: Groups need not be mostly fraud to be meaningful; they describe how payments look.

8.1 · K-MEANS CLUSTERING: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
from sklearn.cluster import KMeans
km = KMeans(n_clusters=2, n_init=10, random_state=42).fit([[0], [2], [10], [12]])
print(sorted(km.cluster_centers_.ravel().tolist()), km.inertia_)
```

`[1.0, 11.0] 4.0`

Why: Two clusters {0, 2} and {10, 12}, centers 1 and 11; each point is 1 from its center, so inertia $4 \times 1^2 = 4.0$.

P4

Predict the output, then run it.

```
from sklearn.cluster import KMeans
km = KMeans(n_clusters=2, n_init=10, random_state=42).fit([[0], [2], [10], [12]])
print(km.predict([[5], [7]]) == km.predict([[1], [11]]))
```

```
[ True True]
```

Why: 5 is nearer 1 than 11 and 7 is nearer 11 than 1, so 5 lands with 1 and 7 with 11: both comparisons are True.

LAB REFERENCE SOLUTIONS

- LAB 7.3 A: LABS/MODULE-02/L7_3-LIME-FAIRNESS/LIME_FRAUD_STARTER.PY → LIME_FRAUD_SOLUTION.PY

The solution prints:

```
transaction_id  p_fraud  is_fraud
       T031633    0.990         1
       T033385    0.567         1
       T028586    0.057         0
  velocity_24h > 1.00                +0.032
  country_mismatch > 0.00            +0.031
  device_new <= 0.00                 -0.016
  card_attempts > 1.00               +0.013
fidelity 0.12   surrogate 0.077   model 0.990
num_samples= 500 T031633: top rule agrees in 3 of 3 seeds, fidelity [0.35, 0.34, 0.28],
num_samples= 500 T033385: top rule agrees in 2 of 3 seeds, fidelity [0.33, 0.26, 0.25],
num_samples= 500 T028586: top rule agrees in 1 of 3 seeds, fidelity [0.03, 0.05, 0.1],
num_samples=5000 T031633: top rule agrees in 2 of 3 seeds, fidelity [0.14, 0.1, 0.1], t
num_samples=5000 T033385: top rule agrees in 3 of 3 seeds, fidelity [0.15, 0.1, 0.1], t
num_samples=5000 T028586: top rule agrees in 3 of 3 seeds, fidelity [0.06, 0.07, 0.05],
SHAP's top feature: {'T031633': 'velocity_24h', 'T033385': 'velocity_24h', 'T028586':
```

Reading it: (1) the flat map fails on the riskiest payment: fidelity 0.12 and a surrogate of 0.077 for a model probability of 0.990. With 0.88% fraud, nearly every random variant of a payment looks legitimate, so the line is fitted to a flat neighborhood near zero; (2) with 500 variants the top rule agrees in 3, 2 and 1 of 3 seeds; with 5,000 in 2, 3 and 3, and every fidelity stays low (0.05 to 0.35); (3) SHAP names velocity, velocity, new device, which agree with LIME's most common first rules. TODO 4: hand the review team SHAP's reasons; LIME fails both of its own checks here, and it struggles less on churn because 16.5% positives give its neighborhoods real variation.

- LAB 7.3 B: LABS/MODULE-02/L7_3-LIME-FAIRNESS/FAIRNESS_CALIBRATION_STARTER.PY → FAIRNESS_CALIBRATION_SOLUTION.PY

The solution prints:

```
offers 300 of 1500; churners among them 52.3%
         customers  actual  predicted    gap
age
18-24          162   0.173      0.137 -0.036
25-34          309   0.178      0.143 -0.035
35-44          295   0.173      0.131 -0.042
45-54          255   0.122      0.147  0.025
55-64          204   0.167      0.174  0.007
65+            147   0.163      0.174  0.011
unknown        128   0.195      0.157 -0.039
         precision  profit_per_customer
age
18-24         0.54                 2.96
25-34         0.54                 3.25
35-44         0.57                 3.31
45-54         0.41                 2.00
55-64         0.50                 3.82
65+           0.55                 3.78
unknown       0.63                 4.80
largest calibration gap 0.042; typical by chance 0.046; share of shuffles at least as
```

Reading it: the $15 / $60 break-even (0.25) flags exactly the same 300 customers as the lesson's top 20% (the 80th percentile of the probabilities is 0.2506). (1) The model under-predicts churn for the three youngest groups and the unknown group by about 4 points and over-predicts for 45 to 54 by 2.5; (2) 45 to 54 has the lowest precision (0.41) and earns least per customer ($2.00) because its churn rate is lowest (12.2%) while it is flagged slightly more often; (3) the largest calibration gap (0.042) is smaller than the median gap of shuffled labels (0.046): no evidence of a real difference. TODO 4, a model answer: "We checked calibration and campaign profit by age group on 1,500 test customers: gaps of up to 4 points are well within what chance produces with groups of 128 to 309. We did not check other groups, other thresholds, or the effect of the offer itself."

HANDOUT: LABS/MODULE-02/L7_3-LIME-FAIRNESS/DEBUG_LIME.PY (DELIBERATELY BROKEN)

1. `KeyError: 0` from `explain_instance` → the row is a pandas Series; pass `.to_numpy()`.
2. `ValueError: Domain error in arguments ... truncnorm` → the explainer's training data has gaps (email opens); use `X_train.fillna(fill).to_numpy()`.
3. `NotImplementedError: LIME does not currently support classifier models without probability scores` → `predict_fn` returns `model.predict`; return `model.predict_proba`.
4. No error: `num_samples=100`. It prints a different explanation from the reported one (discounts first, then recency, then orders: discount_share > 0.29, days_since_last_order > 110.00, orders_12m <= 2.00). Fix: `num_samples=5000`.

Fixed, it prints the docstring's three rules for C104051 (recency +0.295, discounts +0.240, tenure -0.071).

LAB 8.1 A: LABS/MODULE-02/L8_1-KMEANS/PAYMENT_CLUSTERS_STARTER.PY → PAYMENT_CLUSTERS_SOLUTION.PY

The solution prints:

```
      amount  account_age_days  items  velocity_24h
mean   193.8             662.1    1.9           0.4
std    159.9             697.9    0.9           0.6
min      9.8               0.0    1.0           0.0
max   3161.2            7030.0    7.0           7.0
         payments   amount  account_age_days  items  velocity_24h  is_fraud
cluster
0           14848  193.046           797.045  1.889         0.351     0.003
3            1772  194.489          2982.378  1.900         0.332     0.004
2            6543  190.686          1633.330  1.894         0.357     0.004
1           26837  194.995           197.373  1.903         0.366     0.013
         payments   amount  account_age_days  items  velocity_24h  is_fraud
cluster
3           20287  118.677           728.196  1.372         0.000     0.003
0           14316  332.160           718.765  2.869         0.175     0.008
1            3996  189.302             4.977  1.882         0.311     0.008
2           11401  155.468           703.485  1.620         1.247     0.021
seed 0: inertia 97,067
seed 1: inertia 97,067
seed 2: inertia 97,067
seed 42: inertia 97,067
                      payments  fraud_rate  frauds
group
Small everyday orders     20287      0.0027      55
Big baskets               14316      0.0076     109
Brand-new accounts         3996      0.0078      31
Busy cards                11401      0.0213     243
```

Reading it: (1) raw, account age (std about 700 days) dwarfs items (std about 1): the four clusters are bands of account age with identical amounts, items and velocity; (2) after `np.log` of the amount and `np.log1p` of the age (log1p because brand-new accounts have age 0) and standardizing, every column has a say: small everyday orders, big baskets, brand-new accounts (5 days on average) and busy cards (1.2 payments in 24 hours); (3) the same inertia for four seeds; (4, 5) fraud differs fourfold between groups the clustering built without it, which says the groups capture real behavior; but 97.9% of busy-card payments are legitimate, so a cluster is a description, not a detector (self-check Q12).

LAB 8.1 B: LABS/MODULE-02/L8_1-KMEANS/SCALING_CHOICES_STARTER.PY → SCALING_CHOICES_SOLUTION.PY

The solution prints:

```
standard deviations: {'days_since_last_order': 87.3, 'spend_12m': 417.6, 'support_ticke
         customers    days    spend  tickets  churn
cluster
3              293   51.72  1691.66     1.26   0.04
1             1165   60.04  1061.40     1.06   0.08
2             1946   72.34   640.15     0.96   0.12
0             2596  106.10   235.59     0.85   0.25
         customers    days    spend  tickets  churn
cluster
1             1298   47.25  1182.99     0.80   0.05
2             2721   59.21   402.30     0.43   0.10
0             1201   56.25   566.13     2.54   0.22
3              780  270.91   358.22     0.57   0.51
centers in real units:
   days_since_last_order  spend_12m  support_tickets_12m
0                   56.3      566.1                  2.5
1                   47.3     1183.9                  0.8
2                   59.2      402.7                  0.4
3                  270.9      358.2                  0.6
         customers    days   spend  tickets  churn
cluster
0             2966   36.70  651.65     0.47   0.07
1             1232   40.93  713.83     2.51   0.17
3             1263  139.60  453.23     0.67   0.24
2              539  307.44  380.05     0.65   0.53
standardized    0    1     2    3
raw
0             504    0  1550  542
1             216  890     3   56
2             466  132  1168  180
3              15  276     0    2
```

Reading it: (1) raw, spend (std $418) decides: four spend bands with churn falling as spend rises; recency and tickets barely vary; (2) standardized, two groups appear that the raw version could not see: 780 silent customers (271 days, churn 51%) and 1,201 who complain (2.5 tickets, churn 22%); the centers in real units say so directly; (3) counting recency twice is a legitimate business choice: it splits recency more finely (539 customers at 307 days, churn 53%); (4) the crosstab spreads raw groups across several standardized ones: different groupings, not the same groups renamed.

HANDOUT: LABS/MODULE-02/L8_1-KMEANS/DEBUG_KMEANS.PY (DELIBERATELY BROKEN)

1. `ValueError: could not convert string to float: 'Bend'` → `home_store` is text; remove it from `behavior`.
2. `ValueError: Input X contains NaN.` → `email_opens_90d` has 537 gaps; remove it (the lesson's four behavior columns have none).

3. No error: the centers are printed in standard deviations (big-ticket row `{'orders_12m': -0.78, 'avg_order_value': 1.89, 'discount_share': -0.54, 'return_rate': -0.42}`). Fix: `scaler.inverse_transform(kmeans.cluster_centers_)`.

4. No error: "cluster 1" is no longer the big-ticket segment; it prints `1522 customers, average order $97.98, churn 0.191` (the occasional buyers). Fix: pick the cluster from the profile, e.g. `centers["avg_order_value"].idxmax()`.

Fixed, it prints the five centers in real units and `big-ticket segment: 1073 customers, average order $336.92, churn 0.159`.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 7.3 · LIME AND FAIRNESS CHECKS

1. **C** A simple weighted linear model fitted to the original model's scores on nearby variants.

2. **A** Treat it as unreliable: the surrogate does not imitate the model near this customer.

3. **B** SHAP measures the share of the gap from an average customer; the slope measures sensitivity nearby, and the curve is flat at 364 days.

4. **C** Outcomes can still differ by age through proxy features, so measure them by group.

5. **B** No evidence of a real difference, but the groups are too small to prove there is none; keep monitoring.

LESSON 8.1 · K-MEANS CLUSTERING

1. **B** 1.0: clusters {1, 2} and {10, 11}, each point 0.5 from its center: 4 × 0.25.

2. **A** Spend decides almost alone, because its numbers are much larger.

3. **C** Some starts led to local minima; use k-means++ starts and several runs.

4. **C** Probably nothing: cluster numbers are arbitrary and can come back shuffled.

5. **A** Two interlocking half-moons.

EXIT TICKET

1. **LIME's two checks:** fidelity (`exp.score`; and `exp.local_pred` close to the model, at least on the same side of the threshold) and stability (rerun with other `random_state` values; the top reasons should hold).

2. **Audit by age:** other features can be proxies for age, so outcomes can still differ; for a missed benefit, compare recall by group, with group sizes and a permutation test.

3. **Scaling and n_init:** distances depend on units, so the column with the largest spread would decide the

clusters; `n_init=10` runs from ten starts and keeps the lowest inertia, guarding against local minima.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 C ("the real risk is 0.077").** Confuses the surrogate with the model. The model's prediction is what is being explained.
- **Q4 A (flag rate).** Equal treatment in the sense of equal numbers, not equal chances for the people the offer is for. Use worked example 2.
- **Q6 A or B.** Reads a non-significant gap as either a verdict or a certificate. Rehearse the two-part sentence.
- **Q7 written as 2.5 or 4.0.** Forgot to square, or averaged instead of summing.
- **Q9 B ("ten rounds").** Confuses `n_init` with `max_iter`.
- **Q12 A.** Treats a descriptive group as a classifier: ask how many legitimate payments would be flagged.

INSTRUCTOR KIT · ANSWER KEY

Session 3: Choosing k and Hierarchical Clustering

Answers and reasoning for every Session 3 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs, the lesson pages' own quizzes and the exit ticket.

Session 3 | 12 self-check items | Confidential

WARM-UP

W1

Name LIME's two checks before you trust an explanation.

Fidelity (`exp.score` , and the surrogate's prediction close to the model) and stability (the top reasons survive other seeds).

W2

Recall by age group ranges from 54% to 77% and a permutation test finds a gap this large in 47% of shuffles. Say the honest conclusion in one sentence.

No evidence that recall differs by age group, and the groups are too small to rule out a moderate difference.

W3

What does this print?

```
from sklearn.cluster import KMeans
km = KMeans(n_clusters=2, n_init=10, random_state=42).fit([[0], [4], [10], [14]])
print(km.inertia_)
```

`16.0` : clusters {0, 4} and {10, 14}, centers 2 and 12; each point is 2 away: $4 \times 2^2 = 16$.

W4

Why scale before K-Means?

Distances depend on units: the column with the largest spread would decide the clusters (dollars over orders, account age over items).

W5

Yesterday cluster 3 was the discount segment; after a refit it is cluster 1. What happened?

Probably nothing: cluster numbers are arbitrary names. Name clusters from their profiles.

8.2 · CHOOSING K AND CUSTOMER SEGMENTS: SELF-CHECK

Q1

Your loop prints inertia for k = 2 to 9. Why is "pick the k with the lowest inertia" useless?

A) Inertia is only defined for k = 5

B) Inertia can only fall as k grows, so its minimum is always the largest k you tried

C) Inertia rises after the true k, so its minimum is too small

D) Inertia is random, so its minimum changes every run

> **Why:** With one more center the old solution is still available, so the best inertia never rises; with one center per customer it would be 0. Read the shape (the elbow, 2,599 → 470 at 5 → 6) and use scores that balance cohesion and separation.
> **A** is wrong: Inertia exists for any k.
> **C** is wrong: It never rises; that is exactly why its minimum is meaningless.
> **D** is wrong: With `n_init=10` and a seed it is reproducible; and randomness is not the problem.

Q2

What does this print?

```
from sklearn.metrics import silhouette_samples
print(silhouette_samples([[0], [2], [10]], [0, 0, 1]).round(2))
```

```
[0.8 0.75 0. ]
```

Why: Point 0: a = 2 (to point 2), b = 10, s = 8/10 = 0.8. Point 2: a = 2, b = 8, s = 6/8 = 0.75. Point 10 is alone in its cluster: scikit-learn gives a single-member cluster a silhouette of 0.

Q3

Bootstrap refits at k = 3 give a worst ARI of 0.60, while k = 4 and 5 stay above 0.98. What does that say about three clusters?

A) Three is the best k because its ARI varies most

B) The ARI is broken for k = 3

C) Three clusters are too few to have a silhouette

D) Three is not a natural number of groups here: K-Means must make an arbitrary choice about what to merge, and resamples disagree

Why: Real groups reappear when the data is resampled. An unstable k means the algorithm is splitting or merging along lines the data does not insist on. Stability ruled out three and favored four and five.

A is wrong: Variation is the problem, not a merit.

B is wrong: The ARI works for any k; it is reporting disagreement.

C is wrong: Any k from 2 has a silhouette (k = 3 had 0.367).

Q4

In Lab 8.2 B, a blanket retention offer loses money in four of five segments, and on the test customers the churn model's 300 offers earn $4,920 against -$7,620 for offering everyone. What is the lesson for the segment brief?

A) Segments describe who is together; choosing whom to call is the model's job

B) Segments are useless and should be dropped

C) Offer everyone in Deal Hunters and nobody else: that beats the model

D) Refit the segments with churn as a fifth column

> **Why:** A segment's churn rate is an average: only Deal Hunters (0.31) clear the 0.25 break-even, and even there the model earns more ($2,310 vs $1,425 on test). Segments earn their keep as a way to talk about customers and plan different actions, not as a call list.
> **B** is wrong: They are a communication and planning tool, and their churn rates differ eight-fold.
> **C** is wrong: Deal Hunters blanket: $1,425 on test; the model inside Deal Hunters alone: $2,310.
> **D** is wrong: Clustering on the outcome turns a description into a leaky model.

Q5

What does this print?

```
from sklearn.metrics import adjusted_rand_score
a = [0, 0, 0, 1, 1, 1]
b = [3, 3, 3, 8, 8, 8]
print(adjusted_rand_score(a, b), round(adjusted_rand_score(a, [0, 0, 1, 1, 1, 1]),
```

`1.0 0.32`

> **Why:** The first pair are the same grouping with other numbers: ARI 1.0. In the second, one point moved from the first group to the second: most pairs still agree, but the adjustment for chance brings the score down to 0.32.

Q6

For the activity-share clusters (Lab 8.2 A), the silhouette is 0.453 at k = 5 and 0.429 at k = 6, both are stable, and k = 6 adds a "general gear" group of 484 customers. What is the most sensible next step?

A) Choose 9, the k with the lowest inertia

B) Stop: the scores disagree, so the data has no clusters

C) Compare the profiles for 5 and 6 and choose the one whose groups lead to distinct actions

D) Average the two clusterings

> **Why:** When the statistics narrow the range to two candidates, the business picks within it and writes the trade-off down. A merchandising team that would treat "general gear" buyers differently has a reason to prefer six.
> **A** is wrong: Lowest inertia always picks the largest k.
> **B** is wrong: Both are well-supported, stable answers; that is not "no clusters".
> **D** is wrong: Labelings cannot be averaged; the numbers do not even match between them.

8.2 · CHOOSING K AND CUSTOMER SEGMENTS: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
from sklearn.metrics import silhouette_samples
print(silhouette_samples([[0], [1], [4], [5]], [0, 0, 1, 1]).round(2))
```

```
[0.78 0.71 0.71 0.78]
```

> **Why:** Point 0: a = 1 (its mate at 1), b = average of 4 and 5 = 4.5, s = 3.5 / 4.5 = 0.78. Point 1: a = 1, b = 3.5, s = 0.71. The other pair mirrors them.

P2

Predict the output, then run it.

```
from sklearn.metrics import adjusted_rand_score
print(adjusted_rand_score([0, 0, 1, 1], [1, 1, 0, 0]), adjusted_rand_score([0, 0, 1
```

```
1.0 -0.5
```

> **Why:** The first pair is the same grouping with swapped numbers: 1.0. The second splits every original pair and puts unrelated points together: worse than chance, -0.5.

8.3 · HIERARCHICAL CLUSTERING: SELF-CHECK

Q7

What does this print?

```
import numpy as np
from scipy.cluster.hierarchy import linkage
Z = linkage(np.array([[0.0], [1.0], [3.0], [10.0], [11.0], [30.0]]), method="single
print(len(Z), Z[-1, 3])
```

`5 6.0`

Why: n points always need n - 1 merges: 5 rows for 6 points. The last merge joins everything, so its size (column 3) is 6.

Q8

Lab 8.3 A clusters 24 products by their monthly sales. With Euclidean distance the tree puts the two best sellers on their own; with correlation distance it finds a winter family and a summer family. Why?

A) Euclidean distance is wrong for numbers between 0 and 100

B) Correlation distance ignores the months

C) Euclidean distance mostly measures volume (how many units), correlation distance compares the shape of the year

D) Average linkage only works with correlation

Why: Two products with the same seasonal shape but ten times the volume are far apart in Euclidean distance and at distance 0 in correlation distance. The question was "which sell in the same season?", a question about shape.

A is wrong: Euclidean distance works for any numbers; it answers a different question.

B is wrong: It compares the 12 months' pattern; it ignores their level.

D is wrong: Average linkage works with any distance (Ward is the one restricted to Euclidean).

Q9

Which call cuts a tree at height 50 and lets the number of clusters follow?

A) `AgglomerativeClustering(n_clusters=None, distance_threshold=50, linkage="ward")`

B) `AgglomerativeClustering(n_clusters=5, distance_threshold=50, linkage="ward")`

C) `AgglomerativeClustering(affinity="euclidean", distance_threshold=50)`

D) `AgglomerativeClustering(n_clusters=50, linkage="ward")`

> **Why:** Exactly one of `n_clusters` and `distance_threshold` may be set, and the default `n_clusters=2` counts as set, so pass `n_clusters=None`.
> **B** is wrong: Both set: `ValueError: Exactly one of n_clusters and distance_threshold has to be set`.
> **C** is wrong: `affinity` was removed: TypeError (and n_clusters still defaults to 2).
> **D** is wrong: That asks for 50 clusters, not a cut at height 50.

Q10

A dendrogram's highest merges are at 1.54, then 0.29, 0.28, 0.22. How many families does the tree suggest, and why?

A) Four, one per merge listed

B) Twenty-four, one per product

C) It depends only on the business

D) Two: the long empty stretch between 0.29 and 1.54 means any cut there gives the same two groups

> **Why:** A large vertical gap in the tree is the hierarchical version of the elbow: cuts anywhere inside it give the same answer, so the answer is robust. Below 0.29 merges are cheap and the extra splits buy little.
> **A** is wrong: Merges are not clusters; undoing the top merge gives two clusters.
> **B** is wrong: That is the bottom of the tree, before any merge.
> **C** is wrong: The business picks within what the data supports; here the gap is very clear.

Q11

What does this print?

```
import numpy as np
from scipy.spatial.distance import pdist
summer = [10, 20, 40, 20]          # units per season
winter = [40, 20, 10, 20]
big_summer = [100, 200, 400, 200]  # same shape as summer, ten times the volume
print(pdist(np.array([summer, winter, big_summer]), metric="correlation").round(2))
```

`[1.89 0. 1.89]`

Why: The pairs are (summer, winter), (summer, big_summer), (winter, big_summer). Summer and big_summer have the same shape, so correlation distance is 0 although their volumes differ tenfold; winter is strongly negatively correlated with both (1.89, close to the maximum of 2).

Q12

Why does Lab 8.3 B build the Ward tree on a sample of 500 customers and then assign everyone to the nearest cluster mean?

A) Ward cannot handle more than 500 rows

B) Agglomerative clustering needs every pairwise distance (18 million for 6,000, billions for 100,000) and has no predict, so: tree on a sample, nearest mean for the rest

C) A sample always gives better clusters than the full data

D) The nearest-mean step makes the clusters hierarchical

Why: The cost grows with the square of n, and the fitted tree has no rule for new rows. The sample recipe kept the structure (ARI 0.971 with K-Means), and one K-Means run from the sample's means reached the full answer (ARI 1.000).

A is wrong: It ran on all 6,000 in under a second here; the limit is memory and time as n grows.

C is wrong: The sample matched the full answer closely, not better.

D is wrong: Nearest-mean assignment is a flat rule; the hierarchy lives only in the sample's tree.

8.3 · HIERARCHICAL CLUSTERING: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
from scipy.cluster.hierarchy import linkage
Z = linkage([[0.0], [1.0], [2.0], [10.0], [30.0]], method="single")
print(len(Z), Z[:, 2].tolist())
```

```
4 [1.0, 1.0, 8.0, 20.0]
```

> **Why:** Five points need four merges: 0-1 at 1, then 2 joins at 1 (nearest member 1), 10 joins at 8 (nearest member 2), 30 at 20.

P4

Predict the output, then run it.

```
from scipy.spatial.distance import pdist
print(pdist([[1, 0, 1, 0], [0, 1, 0, 1], [2, 0, 2, 0]], metric="correlation").round
```

```
[2. 0. 2.]
```

> **Why:** The first and third rows have the same pattern (the third is twice the first): distance 0. The second is the exact opposite of both: distance 2, the maximum.

LAB REFERENCE SOLUTIONS

- LAB 8.2 A: LABS/MODULE-03/L8_2-CHOOSING-K/CHOOSE_K_TASTES_STARTER.PY → CHOOSE_K_TASTES_SOLUTION.PY

The solution prints:

```
(3000, 6) average shares: {'camping': 0.21, 'climbing': 0.12, 'general': 0.28, 'hiking
   inertia  silhouette   drop
k
2    652.0       0.310    NaN
3    490.4       0.310  161.6
4    361.7       0.379  128.6
5    246.4       0.453  115.3
6    195.8       0.429   50.6
7    179.2       0.384   16.6
8    164.6       0.359   14.6
9    153.2       0.329   11.4
k=4: mean ARI 0.817   worst 0.641
k=5: mean ARI 0.966   worst 0.925
k=6: mean ARI 0.969   worst 0.950
k=7: mean ARI 0.901   worst 0.658
primary_activity  camping  climbing  general  hiking  paddling  running  customers mai
3                    0.12      0.03     0.38    0.41      0.04     0.03       1009
4                    0.58      0.02     0.27    0.07      0.04     0.02        738
2                    0.08      0.66     0.14    0.04      0.05     0.03        424   cl
0                    0.08      0.02     0.29    0.04      0.55     0.02        423   pa
1                    0.07      0.03     0.21    0.06      0.05     0.58        406
primary_activity  camping  climbing  general  hiking  paddling  running  customers
2                    0.12      0.03     0.28    0.50      0.05     0.03        685
4                    0.62      0.02     0.24    0.07      0.04     0.02        649
1                    0.15      0.03     0.56    0.16      0.06     0.04        484
3                    0.08      0.67     0.14    0.04      0.05     0.03        416
0                    0.07      0.03     0.19    0.06      0.05     0.59        387
5                    0.08      0.02     0.26    0.04      0.58     0.02        379
```

Reading it: no scaling is needed because every row is a set of shares adding up to 1. Elbow and silhouette both point at five (0.453); stability rules out four and seven (worst ARI 0.641 and 0.658) and accepts five and six; the profile shows five tastes, one per activity, with the "general" products (headlamps, bottles) spread over all of them. At six, a general-gear group of 484 splits off, mostly from the hiking group. Either is defensible; the model answer to TODO 4: "five if merchandising plans by activity; six if general gear gets its own campaign". Note the parallel with Lesson 8.2: how customers shop and what they buy both give five groups, which need not be the same people.

- LAB 8.2 B: LABS/MODULE-03/L8_2-CHOOSING-K/SEGMENT_ECONOMICS_STARTER.PY → SEGMENT_ECONOMICS_SOLUTION.PY

The solution prints:

```
{'Trail Regulars': 1449, 'Casual Campers': 1374, 'Deal Hunters': 1253, 'Big-Trip Planne
                   customers  churn  profit_per_customer
segment
Deal Hunters            1253   0.31                 3.72
Casual Campers          1374   0.20                -2.82
Big-Trip Planners       1113   0.16                -5.24
Gear Testers             811   0.10                -8.93
Trail Regulars          1449   0.04               -12.56
break-even churn rate for a blanket offer: 0.25
                   customers  blanket_profit  model_offers  model_profit
segment
Big-Trip Planners        289           -1335            54           930
Casual Campers           329           -1695            88          1200
Deal Hunters             305            1425           126          2310
Gear Testers             232           -1740            21           345
Trail Regulars           345           -4275            11           135
totals: {'customers': 1500, 'blanket_profit': -7620, 'model_offers': 300, 'model_profi
```

Reading it: a blanket offer to a segment pays only when its churn rate beats the 0.25 break-even (15 / 60): Deal Hunters (0.31) earn $3.72 per customer, Trail Regulars lose $12.56. On the 1,500 test customers, offering everyone loses $7,620 while the model's 300 offers earn $4,920, and the model wins inside every single segment. TODO 4: segments describe and communicate (different messages, products, channels per segment); the model chooses whom to call. The same 300 offers as the lesson 7.3 audit.

HANDOUT: LABS/MODULE-03/L8_2-CHOOSING-K/DEBUG_CHOOSE_K.PY (DELIBERATELY BROKEN)

1. `ValueError: Number of labels is 1. Valid values are 2 to n_samples - 1 (inclusive)` → start the loop at k = 2.
2. No error: k is chosen with `table["inertia"].idxmin()`, which always picks the largest k (8 here, silhouette 0.362). Fix: `table["silhouette"].idxmax()` → 5 (0.479).
3. No error: the newcomers are scaled with a NEW scaler fitted on the three of them, which spreads three similar discount-heavy shoppers over three segments (`[2, 2, 4]` with k = 5). Fix: `prep.transform(newcomers)` → `[0, 0, 0]`, all Deal Hunters.

LAB 8.3 A: LABS/MODULE-03/L8_3-HIERARCHICAL/SEASONAL_FAMILIES_STARTER.PY → SEASONAL_FAMILIES_SOLUTION.PY

The solution prints:

```
(24, 12) units per week, product averages from 11.0 to 79.7
merges: 23   five highest merge heights: [1.54, 0.292, 0.281, 0.215, 0.179]
height 0.15: 8 families, sizes [2, 4, 1, 1, 2, 1, 10, 3]
height 0.25: 4 families, sizes [6, 2, 3, 13]
height 0.50: 2 families, sizes [6, 18]
height 1.00: 2 families, sizes [6, 18]
col_0  Accessories  Apparel  Camp Kitchen  Climbing  Footwear  Packs  Sleeping  Snow  T
row_0
1                0        4             0         0         0      0         0     2
2                0        0             0         0         2      0         0     0
3                3        0             3         2         1      2         2     0
month       1     2     3     4     5     6     7     8     9     10    11    12
family 1  28.4  25.1  17.6  13.8  13.0  11.0  10.5  11.3  17.8  21.8  37.3  41.6
family 2  18.1  19.5  24.8  28.5  37.1  30.0  28.8  26.1  28.3  20.0  27.2  26.2
family 3  15.0  16.2  20.3  25.9  39.1  40.2  40.1  32.3  28.5  21.1  21.1  22.9
family 2: ['Foothill Trail Runners', 'Talus Hiking Boots']
Euclidean sizes: [22, 1, 1]
Euclidean small families: ['Cairn Hiking Socks', 'Wayfinder Insulated Mug']
cophenetic correlation (average linkage, correlation distance): 0.979
```

Reading it: 24 products, 23 merges. The top merge (1.54) joins two groups whose years are opposite (correlation distance above 1 means negatively correlated): a winter family (4 Apparel + 2 Snow, peaking in December) and everything else (peaking in May to July). Any cut between 0.29 and 1.54 gives the same two families: the big gap. At three families the two footwear products separate with a spring peak. Euclidean distance answers a different question (how many units): it isolates the two best sellers (socks 79.7 and mug 54.6 units a week). The cophenetic correlation of 0.979 says the tree keeps the original distances well.

- LAB 8.3 B: LABS/MODULE-03/L8_3-HIERARCHICAL/SAMPLE_THEN_ASSIGN_STARTER.PY → SAMPLE_THEN_ASSIGN_SOLUTION.PY

The solution prints:

```
pairwise distances for all 6,000 customers: 17,997,000
full Ward vs K-Means (ARI): 0.907
sample   500 (  124,750 distances): sizes [851, 1105, 1238, 1392, 1414], ARI vs K-Means
sample 1,500 (1,124,250 distances): sizes [856, 1088, 1259, 1379, 1418], ARI vs K-Means
K-Means from the 500-sample means: inertia 5,525 (ten random starts: 5,525), ARI 1.000
       orders_12m  avg_order_value  discount_share  return_rate  churned
group
3            8.84            88.58            0.10         0.04     0.04
0            6.35           167.26            0.18         0.28     0.10
4            2.50           331.90            0.11         0.05     0.16
2            1.85            96.05            0.14         0.06     0.20
1            5.42            64.16            0.51         0.08     0.31
```

Reading it: a 500-customer sample needs 0.7% of the distances and still recovers the five groups (ARI 0.971 with K-Means, 0.910 with the full Ward tree, about as close as full Ward and K-Means are to each other, 0.907). One K-

Means run started from the sample's means reaches exactly the ten-start answer (ARI 1.000): a classic combination. The profile is Lesson 8.2's five segments. TODO 4: at 100,000 customers the sample tree and the nearest-mean step stay fast; full agglomerative clustering (5 billion distances) would not run on a laptop.

HANDOUT: LABS/MODULE-03/L8_3-HIERARCHICAL/DEBUG_HIERARCHICAL.PY (DELIBERATELY BROKEN)

1. `TypeError: AgglomerativeClustering.__init__() got an unexpected keyword argument 'affinity'` → `metric="precomputed"`.
2. `ValueError` about both `n_clusters` and `distance_threshold` → `n_clusters=None` (a cut at 0.97 lets the number follow).
3. `ValueError` from Ward with a precomputed (correlation) distance → `linkage="average"`.
4. No error: the SciPy cross-check uses `method="single"`, so it compares against a chained tree and prints an agreement of `0.000`. Fix: `method="average"` → 1.000.

Fixed: `family sizes: [31, 36, 36, 38, 59]` (the lesson's five families) and agreement 1.000. The order in which learners meet the errors depends on which they fix first.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 8.2 · CHOOSING K AND CUSTOMER SEGMENTS

1. **C** 0.75: (8 - 2) / max(2, 8).
2. **A** 1.0: the same groups with swapped numbers.
3. **B** Compare the profiles for k = 4, 5 and 6 and choose the one whose segments lead to distinct actions.
4. **C** They are right-skewed; a log keeps a few extreme customers from dominating the distances.
5. **A** MiniBatchKMeans, scoring k with a sampled silhouette.

LESSON 8.3 · HIERARCHICAL CLUSTERING

1. **B** `4 5.0`: four merges for five points; the last merge holds all five.
2. **A** Single linkage: one trail of in-between customers is enough to chain the groups.
3. **C** `metric="cosine", linkage="average"` (Ward needs Euclidean; `affinity` is gone).
4. **B** 4: undoing the three top merges leaves four clusters, and below them every merge is cheap.
5. **C** Hundreds of thousands of rows and new rows to assign later.

EXIT TICKET

1. **Four kinds of evidence:** elbow (where extra clusters stop paying off), silhouette (cohesion against separation, per cluster too), stability (groups survive resampling: ARI), usefulness (sizes, outcomes the clustering never saw, distinct actions).
2. **Correlation distance:** Euclidean distance between purchase or sales profiles is dominated by volume (popular products, best sellers); correlation compares the pattern (who buys it, when it sells) whatever the level.
3. **K-Means vs agglomerative:** K-Means for many rows, round groups on Euclidean features and a `predict` for new rows; agglomerative for the full hierarchy, any distance, and no random start, on thousands rather than millions of rows.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 C ("inertia rises after the true k").** Has the elbow picture backwards. Show the table: it falls every time.
- **Q2 written as [0.8 0.75 1.0].** Expects a lone point to be perfectly placed; scikit-learn defines its silhouette as 0.
- **Q4 C ("blanket offer to Deal Hunters beats the model").** Compares the segment's blanket profit with the model's total. Compare within the segment: $1,425 vs $2,310.
- **Q8 A.** Thinks distances need scaled data to be valid. It is a question of what the distance measures.
- **Q10 A ("four, one per merge").** Reads merges as clusters. Undo the top merge: two clusters.
- **Q11 written with a non-zero middle value.** Confuses correlation with closeness of values.

INSTRUCTOR KIT · ANSWER KEY

Session 4: Density-Based Clustering and Gaussian Mixtures

Answers and reasoning for every Session 4 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs, the lesson pages' own quizzes and the exit ticket.

Session 4 | 12 self-check items | Confidential

WARM-UP ANSWERS

Retrieval on Lessons 8.2 and 8.3. Not graded.

W1

What does this print?

```
from sklearn.metrics import adjusted_rand_score
a = [0, 0, 1, 1, 2, 2]
b = [2, 2, 0, 0, 1, 1]
print(adjusted_rand_score(a, b))
```

A) `0.0`

B) `0.333`

C) `1.0`

D) An error: the labels do not match

> **Why:** The adjusted Rand index compares who is grouped with whom and ignores cluster numbers: both labelings put the same pairs together. Lesson 8.2 used it for stability; today it scores clusterings against known groups.

W2

Lesson 8.2 chose k for Pinecrest's customers. Which piece of evidence is NOT a reason to pick k?

A) Where inertia stops dropping steeply (the elbow)

B) A high mean silhouette, checked per cluster

C) Clusters that reappear when you refit on bootstrap samples

D) The k where inertia reaches its minimum

> **Why:** Inertia always falls as k grows (one center per point gives zero), so its minimum means nothing. The elbow, the silhouette, stability and usefulness are the four checks of Lesson 8.2.

W3

What does this print?

```
from scipy.cluster.hierarchy import linkage
Z = linkage([[0.0], [1.0], [10.0], [11.0]], method="single")
print(Z[:, 2])
```

A) `[ 1. 1. 10.]`

B) `[1. 1. 9.]`

C) `[0. 1. 9.]`

D) `[ 1. 10. 11.]`

> **Why:** Single linkage merges 0 with 1 at distance 1, 10 with 11 at distance 1, then the two pairs at their CLOSEST points, 1 and 10: distance 9. A big jump in merge height (1 to 9) is where a dendrogram suggests cutting.

W4

Two groups of customers are joined by a thin trail of in-between customers. Which linkage merges them too early?

A) Single

B) Complete

C) Average

D) Ward

> **Why:** Single linkage measures the closest pair, so a chain of in-between points makes two groups look close: chaining. Density-based methods today treat that trail very differently, as possible noise.

W5

Lesson 8.2 segmented customers on four columns with a log and a scaler. To segment next month's NEW customers, what do you reuse?

A) Only the K-Means centers

B) The whole fitted pipeline (log, scaler, K-Means): call predict on the new rows

C) A new scaler fitted on the new customers, then the old centers

D) Nothing: refit K-Means on the new customers

> **Why:** The centers live in the scaled, logged space of the training customers, so new rows must go through the same fitted steps. Keep one pipeline. Today's density methods cannot do this at all, and Gaussian mixtures can.

8.4 · DENSITY-BASED CLUSTERING: SELF-CHECK

Q1

What does this print?

```
import numpy as np
from sklearn.cluster import DBSCAN
pts = np.array([[0.0], [1.0], [2.0], [10.0], [11.0], [30.0]])
print(DBSCAN(eps=1.5, min_samples=3).fit_predict(pts))
```

A) `[0 0 0 1 1 2]`

B) `[ 0 0 0 1 1 -1]`

C) `[ 0 0 0 -1 -1 -1]`

D) `[-1 -1 -1 -1 -1 -1]`

Why: With `min_samples=3` (the point itself counts) only the point at 1.0 has a crowded neighborhood (0, 1 and 2 are all within 1.5), so it is the one core point; 0 and 2 are within reach of it and join cluster 0 as border points. 10 and 11 have only each other: two points are not a crowd, and no core point is near, so both are noise, like the lone 30.

A is wrong: DBSCAN never gives a lone point its own cluster; isolated points are noise (-1).

B is wrong: 10 and 11 are close to each other, but a pair is not three points: neither is core.

D is wrong: The point at 1.0 has three points within 1.5 (itself included), so there is one cluster.

Q2

You compute each point's distance to its 10th nearest neighbor (itself included), sort the distances and plot them. What is the plot for?

A) Choosing the number of clusters

B) Choosing a starting eps for DBSCAN with min_samples=10: just above the knee

C) Choosing `min_samples`: the height of the knee

D) Checking whether the silhouette is high enough

Why: Each distance is the smallest eps that would make that point a core point with min_samples=10. Crowded points have small distances, isolated ones large: the bend in the sorted curve is a sensible first eps, then a short sweep around it.

A is wrong: DBSCAN has no cluster-count setting; the count comes out of eps and min_samples.

C is wrong: min_samples is chosen first (about twice the features); the plot depends on it.

D is wrong: The k-distance plot uses no cluster labels at all.

Q3

What does this print?

```
import numpy as np
labels = np.array([0, 0, 1, -1, 1, -1, 2])
print(len(set(labels)), len(set(labels) - {-1}))
```

`4 3`

Why: `set(labels)` holds 0, 1, 2 and -1: four labels. -1 is the noise label, not a cluster, so the real count is three. Report the noise count separately; it is a result.

Q4

You switch from DBSCAN to HDBSCAN on a new data set. Which setting do you choose first, and from what?

A) `eps`, from the knee of the k-distance plot

B) `n_clusters`, from the elbow of the inertia curve

C) `min_cluster_size`, from the smallest group you would act on, then check that nearby values agree

D) `xi`, from the steepest wall in the reachability plot

Why: HDBSCAN needs no eps: it keeps the clusters that stay stable across all density levels. The setting you can reason about is the smallest group worth naming (`HDBSCAN(min_cluster_size=..., copy=True)`); Lesson 8.4's Exercise 3 and today's lab both check that neighboring values give the same answer.

A is wrong: That is DBSCAN's setting; HDBSCAN replaces it.

B is wrong: Density methods have no k.

D is wrong: xi belongs to OPTICS's cluster extraction.

Q5

HDBSCAN on card payments returns clusters that each hold payments with exactly 1, 2, 3 or 4 items. What is the best reading?

A) Four genuinely different kinds of shopper, sorted by basket size

B) The items column needs more weight than the others

C) The features were not scaled, so items dominated

D) items only takes whole values, so payments stack on exact values and look dense: profile further and treat the split as an artifact unless the business cares about it

Why: A whole-number column turns each value into a shelf of identical points, the densest thing a density method can see (Lesson 8.4, Common Mistake 5). Today's payment lab shows it, and the real finding there is the noise, not the shelves.
A is wrong: Nothing else about those clusters differs much; the split follows one integer column.
B is wrong: Weighting does not change the fact that values are stacked.
C is wrong: The lab scaled all five features; the problem is that the values are whole numbers.

Q6

What does this print?

```
from sklearn.cluster import HDBSCAN
print(hasattr(HDBSCAN(), "fit_predict"), hasattr(HDBSCAN(), "predict"))
```

A) `True True`

B) `True False`

C) `False True`

D) `False False`

Why: DBSCAN, OPTICS and HDBSCAN describe the data they were fitted on: they have fit_predict but no predict. For new customers, refit, or assign each to the segment of its nearest clustered neighbor and call it noise when it is too far from everyone (Lesson 8.4, Exercise 4).
A is wrong: There is no predict: a new point could even change the clusters.
C is wrong: fit_predict exists on every clustering estimator.
D is wrong: fit_predict exists.

8.4 · DENSITY-BASED CLUSTERING: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
import numpy as np
from sklearn.cluster import DBSCAN
pts = np.array([[1.0], [1.2], [1.4], [3.0], [3.1], [3.2], [9.0]])
print(DBSCAN(eps=0.25, min_samples=3).fit_predict(pts))
```

```
[ 0 0 0 1 1 1 -1]
```

> **Why:** 1.0, 1.2 and 1.4: the middle point has three points within 0.25 (itself included), so it is core and the other two join as border points. 3.0, 3.1 and 3.2 likewise. 9.0 has nobody near: noise.

P2

Predict the output, then run it.

```
import numpy as np
labels = np.array([0, 0, -1, 1, 1, 2, -1, 2])
print(len(set(labels)), (labels == -1).mean())
```

```
4 0.25
```

> **Why:** `set(labels)` holds 0, 1, 2 and -1: four labels, three of them clusters. Two of eight labels are -1: a noise share of 0.25.

8.5 · GAUSSIAN MIXTURE MODELS: SELF-CHECK

Q7

What does this print?

```
import numpy as np
from sklearn.mixture import GaussianMixture
x = np.array([[0.0], [0.1], [5.0], [5.1], [5.2]])
gm = GaussianMixture(n_components=2, random_state=42).fit(x)
print(gm.predict_proba([[100.0]]).max().round(2))
```

A) `0.0`

B) `0.5`

C) `nan`

D) `1.0`

Why: Each row of `predict_proba` adds up to 1, however far the point is from everything: 100 is far closer to the component near 5 than to the one near 0, so it gets that component with certainty. How typical a point is comes from `score_samples`, not from the probabilities.

A is wrong: One of the two probabilities must be large: they add up to 1.

B is wrong: 0.5 would mean equally close to both components; 100 is not.

C is wrong: A far-away point still gets well-defined probabilities.

Q8

BIC for 1 to 6 components is 9,800, 8,100, 7,950, 7,930, 7,920 and 7,915. What is the most sensible choice?

A) About 2 or 3: the big drops stop there; then check whether the third component means something

B) 6, because BIC is lowest there

C) 1, because BIC only rewards simple models

D) None: BIC needs a minimum to be usable

Why: Lower is better, but read the size of each improvement: 1,700, then 150, then 20, 10, 5. After two or three components the extra ones polish shapes of not-quite-Gaussian data. This is the same reading as Lesson 8.5's project.

B is wrong: The last drops are tiny; more components cost parameters and explain little.

C is wrong: BIC penalizes parameters but also rewards fit; the first drops are large.

D is wrong: A curve with no minimum is normal on real data; read where the big gains stop.

Q9

Three groups were made by stretching three identical round blobs with the same matrix. Which covariance_type should BIC prefer?

A) `"spherical"`

B) `"diag"`

C) `"tied"`

D) `"full"`

> **Why:** tied gives every component the same ellipse shape, which is exactly how the groups were made, with the fewest parameters that fit. In Lesson 8.5 BIC picked tied (3,032.8) over full (3,061.9).
> **A** is wrong: Round components cannot follow stretched groups (ARI 0.568 in the lesson).
> **B** is wrong: Axis-aligned ellipses cannot tilt.
> **D** is wrong: full also fits, but pays for shapes it does not need, so its BIC is a little worse.

Q10

What does this print, and what does it tell you?

```
from sklearn.metrics import adjusted_rand_score
run_a = [0, 0, 1, 1, 2, 2]
run_b = [1, 1, 2, 2, 0, 0]
print(adjusted_rand_score(run_a, run_b))
```

`1.0` : the two runs found the same groups under different numbers

> **Why:** ARI ignores the names of the clusters and compares only who is grouped with whom. Component numbers are arbitrary: name segments by their profile, and match runs with ARI or a crosstab.

Q11

A customer sits far from every component, yet their largest predict_proba value is 0.99. Which output flags them as unusual?

A) `score_samples`: a very low log-density

B) `predict`: it returns -1 for outliers

C) `weights_`: the component's share is small

D) `means_`: the customer equals no mean

> **Why:** predict_proba compares components with each other, so a far-away point can still be 99% 'nearest component'. score_samples measures how likely the point is under the whole mixture: the lowest values are the atypical customers (the bridge to Lesson 8.6).
>
> **B** is wrong: A mixture's predict returns a component number, never -1; that convention belongs to outlier detectors (8.6).
>
> **C** is wrong: weights_ describe components, not individual customers.
>
> **D** is wrong: No real customer sits exactly on a mean; that says nothing about how typical they are.

Q12

What happens when this runs?

```
from sklearn.mixture import GaussianMixture
GaussianMixture(n_components=4, random_state=42).fit([[0.0], [1.0], [2.0]])
```

`ValueError: Expected n_samples >= n_components but got n_components = 4, n_samples = 3`

> **Why:** Every component needs data to estimate a mean and a covariance, so there can never be more components than rows, and in practice you need many more rows than that. Fewer components, a simpler covariance_type, or more data.

8.5 · GAUSSIAN MIXTURE MODELS: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
import numpy as np
from sklearn.mixture import GaussianMixture
x = np.array([[0.0], [0.2], [0.4], [8.0], [8.2]])
gm = GaussianMixture(n_components=2, random_state=42).fit(x)
print(np.sort(gm.weights_).round(1))
```

```
[0.4 0.6]
```

> **Why:** Two well-separated groups: three of five values near 0 (weight 0.6) and two near 8 (0.4). `np.sort` puts the smaller first.

P4

Predict the output, then run it.

```
import numpy as np
from sklearn.mixture import GaussianMixture
x = np.array([[0.0], [0.2], [0.4], [8.0], [8.2]])
gm = GaussianMixture(n_components=2, random_state=42).fit(x)
print(gm.predict_proba([[4.1], [100.0]]).sum(axis=1).round(2))
```

```
[1. 1.]
```

> **Why:** Each row of `predict_proba` adds up to 1, for a point in the middle (4.1) and for a point far from everything (100) alike. Typicality is `score_samples`' job.

LAB REFERENCE SOLUTIONS

- LAB 8.4-A: LABS/MODULE-04/L8_4-DENSITY-CLUSTERING/PAYMENT_DENSITY_STARTER.PY → PAYMENT_DENSITY_SOLUTION.PY

The solution prints:

```
(5000, 5) frauds in the sample: 38 (0.76%)
k-distance percentiles (50, 80, 95): [0.45 0.77 1.24]
eps=0.3: 15 clusters, largest   758, noise 69.7%
eps=0.5: 10 clusters, largest  1252, noise 29.4%
eps=0.7: 15 clusters, largest  1330, noise 14.4%
eps=1.0: 13 clusters, largest  1434, noise 5.6%

HDBSCAN: 8 clusters, noise share 8.8%
         payments  amount  account_age  hour  velocity  items  fraud
cluster
-1            442  220.13         95.0  11.8      0.89   2.97  0.054
 0            187  153.79        458.0  15.0      2.00   1.71  0.027
 1            138  268.10        604.0  15.5      0.00   4.00  0.007
 2            536  236.67        548.0  14.8      0.00   3.00  0.004
 3           1417  101.53        451.0  15.0      0.00   1.00  0.001
 4           1346  167.94        450.0  14.9      0.00   2.00  0.001
 5            411  103.55        530.0  15.0      1.00   1.00  0.005
 6            136  235.19        632.0  14.6      1.00   3.00  0.000
 7            387  162.81        512.0  14.8      1.00   2.00  0.003
Most clusters sit on whole-number shelves: one exact items count (1 to 4) and velocity

fraud rate: noise 5.4%, clustered 0.31%
noise holds 8.8% of the payments and 24 of the 38 frauds
min_cluster_size=25   clusters=12  noise=10.7%  fraud in noise=4.5%
min_cluster_size=50   clusters= 8  noise=8.8%  fraud in noise=5.4%
min_cluster_size=100  clusters= 8  noise=8.8%  fraud in noise=5.4%
min_cluster_size=200  clusters= 5  noise=15.3%  fraud in noise=3.9%
```

What each TODO teaches: (1-2) on real data the k-distances climb with no knee and no eps gives a usable answer, the varying-density problem; (3) HDBSCAN with a minimum group size of 50 finds 8 clusters and 8.8% noise; (4) the clusters are shelves: one exact items count and one velocity value each, the whole-number artifact of Lesson 8.4's Common Mistake 5; (5) the real finding is the noise: 8.8% of the payments hold 24 of the 38 frauds, fraud 5.4% against 0.31% in the clusters, without the label ever being used; (6) 50 and 100 agree exactly, and 25 or 200 still put 3.9% to 4.5% fraud in the noise. Lesson 8.6 builds detectors made for this job.

LAB 8.4-B: LABS/MODULE-04/L8_4-DENSITY-CLUSTERING/RINGS_STARTER.PY → RINGS_SOLUTION.PY

The solution prints:

```
(600, 2) points per ring: [300 300]
K-Means k=2  ARI -0.002
k-distance percentiles (50, 90, 99): [0.118 0.186 0.251]
eps=0.1  clusters=18  noise=307  ARI=0.232
eps=0.2  clusters=3   noise=3    ARI=0.946
eps=0.3  clusters=2   noise=0    ARI=1.000
eps=0.5  clusters=2   noise=0    ARI=1.000
eps=0.8  clusters=1   noise=0    ARI=0.000

silhouette, K-Means (wrong answer)  : 0.328
silhouette, DBSCAN (right answer)   : 0.158
The silhouette rewards round, separated blobs; two rings around one center are neither
```

K-Means cannot separate a ring from the ring inside it (ARI about 0). The k-distances suggest eps around 0.2 to 0.25, and 0.3 to 0.5 recover both rings exactly. The silhouette then votes for the wrong answer, because it rewards compact, separated blobs around different centers; two rings share one center. Never pick a density clustering by silhouette alone.

HANDOUT: LABS/MODULE-04/L8_4-DENSITY-CLUSTERING/DEBUG_DENSITY.PY (DELIBERATELY BROKEN)

1. `KeyError: "['returns_rate'] not in index"` → the column is `return_rate`.
2. Silent: no scaling. Spend is in dollars, return rate a fraction, so eps=0.5 means "within 50 cents" → `X = StandardScaler().fit_transform(customers[cols])`.
3. Silent: `len(set(labels))` counts -1 → `len(set(labels) - {-1})`.
4. Silent: the silhouette includes the noise as one scattered cluster → score `X[labels != -1]` only.
5. `AttributeError: 'DBSCAN' object has no attribute 'predict'` → delete it (refit, or assign with a nearest-neighbor rule). Fixed, the report reads: clusters: 4, noise share: 7.4%, churn among noise customers: 0.083, silhouette of the clustered customers: 0.268 (0.226 with the noise included).

LAB 8.5-A: LABS/MODULE-04/L8_5-GAUSSIAN-MIXTURES/DISCOUNT_MIXTURE_STARTER.PY → DISCOUNT_MIXTURE_SOLUTION.PY

The solution prints:

```
(6000, 1) mean 0.208, median 0.141, max 0.833
      bic    drop
1 -3660.0     NaN
2 -7256.0  3596.0
3 -7586.0   330.0
4 -7651.0    65.0
5 -7704.0    54.0
means  : [0.11 0.44]
stds   : [0.063 0.146]
weights: [0.705 0.295]  deal-driven component: 1
0.10    0.012
0.20    0.114
0.25    0.471
0.30    0.911
0.40    1.000
Name: P(deal-driven), dtype: float64
               customers  churn
p_deal
(-0.001, 0.1]       3812  0.119
(0.1, 0.5]           506  0.128
(0.5, 0.9]           232  0.134
(0.9, 1.0]          1450  0.305
borderline (0.2 to 0.8): 430 customers, discount share 0.219 to 0.281
hard rule 'discount_share > 0.3' flags 1431; the mixture's majority vote flags 1682
```

Reading it: BIC's first drop (3,596) dwarfs the rest, so two populations: about 70% of customers use few discounts (mean 0.11) and about 30% are discount-driven (mean 0.44, a wider spread). The probabilities switch between 0.25 and 0.30; customers between 0.219 and 0.281 are genuinely between the two. Churn was never used, yet confident deal-driven customers churn at 0.305 against 0.119: the Deal Hunters of Lesson 8.2 found from one column. A hard 0.3 rule and the mixture's 0.5 vote disagree on about 250 customers, exactly the borderline band.

- LAB 8.5-B: LABS/MODULE-04/L8_5-GAUSSIAN-MIXTURES/LIFECYCLE_MIXTURE_STARTER.PY → LIFECYCLE_MIXTURE_SOLUTION.PY

The solution prints:

```
       bic    drop
k
1  50618.0     NaN
2  47210.0  3408.0
3  46717.0   493.0
4  46332.0   385.0
5  46089.0   243.0
6  46049.0    40.0
           customers  tenure  days_since   spend  churn  weight
component
0               2169  67.700        38.0  742.08  0.109   0.360
1               1272  21.751        38.0  683.35  0.071   0.214
2               1293  55.872       101.0  193.60  0.205   0.228
3               1266   3.142        60.0  405.64  0.317   0.198

ARI between seeds 42 and 7: 0.981
seed 7      0     1     2     3
seed 42
0          16    11  2142     0
1          11  1261     0     0
2        1289     1     3     0
3           0     0     0  1266

confidence below 0.8: 1340 customers (22.3%)
{0: 0.964, 1: 0.893, 2: 0.916, 3: 0.951}

new customer: [[0.    0.014 0.009 0.977]]
```

Reading it: BIC has no minimum and the drops shrink gradually, so four components is a judgment call; defend it with the profiles. Newcomers (3 months) churn at 0.317 and fading customers (long gaps, low spend) at 0.205, against 0.071 for established ones: lifecycle matters for retention. A different seed finds the same groups (ARI 0.981) under other numbers: read the crosstab, where each row has one big cell. 22% of customers sit below 0.8 confidence, many more than in Lesson 8.5's project, because lifecycle is a continuum. The new customer goes through the SAME fitted scaler.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 8.4 · DENSITY-BASED CLUSTERING

1. **B** `[ 0 0 0 -1]`: 0.0, 0.1 and 0.2 are core points chained into cluster 0; 5.0 has no neighbor within 0.15, so it is noise.
2. **C** Decrease eps, using a k-distance plot as a guide: a radius that is too large merges everything.
3. **A** HDBSCAN: it finds tight and loose clusters in one run.
4. **C** A larger, less dense cluster: the height of a valley floor is its typical reachability distance.

5. **A** The whole-number feature created dense ridges; the split is probably an artifact.

LESSON 8.5 · GAUSSIAN MIXTURE MODELS

1. **C** Treat the customer as borderline between components 0 and 1 (0.48 against 0.47 is a coin flip).
2. **A** Each component can be an ellipse with its own spread and orientation.
3. **B** About 5 components: the big improvements stop there.
4. **B** 1.0: each row of predict_proba adds up to 1, however far the point is.
5. **C** score_samples, flagging the lowest log-densities.

EXIT TICKET

1. **Three clusters** (-1 is noise). The 30% are customers in no dense crowd: report them as their own group, with their profile, not as a failure. In Lesson 8.4 they churned above average.
2. **Mixture, not HDBSCAN:** predicts for new rows; probabilities; a density (`score_samples`) and synthetic samples. **HDBSCAN, not a mixture:** clusters of any shape and varying density; an explicit noise label; no number of clusters to choose.
3. **No.** The probabilities only say which component is nearest; they add up to 1 however far the customer is. Check `score_samples`: a very low log-density marks an atypical customer.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 B (`[ 0 0 0 1 1 -1]`).** Thinks two close points make a cluster. Recite: min_samples counts the point itself; a pair is not a crowd of three.
- **Q3 written as `4 4` or `3 3`.** Either counts -1 as a cluster or forgets that it is a label. Ask what `set` returns.
- **Q5 A (four kinds of shopper).** Believes the algorithm. Point to the profile table of the payment lab: the clusters differ in one integer column only.
- **Q7 A (0.0) or B (0.5).** Expects probabilities to fall far from the data. The exit ticket's question 3 targets the same misunderstanding.
- **Q8 B (6).** Reads the minimum. Same habit as inertia in the warm-up: read the size of the drops.
- **Q11 B (predict returns -1).** Mixes the mixture's `predict` with outlier detectors' convention. Lesson 8.6 introduces that convention; flag it now.

INSTRUCTOR KIT · ANSWER KEY

Session 5: Anomaly Detection and PCA

Answers and reasoning for every Session 5 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs, the lesson pages' own quizzes and the exit ticket.

Session 5 | 12 self-check items | Confidential

WARM-UP ANSWERS

Retrieval on Lessons 8.4 and 8.5. Not graded.

W1

What does this print?

```
import numpy as np
from sklearn.cluster import DBSCAN
pts = np.array([[0.0], [0.1], [0.2], [4.0], [4.1], [4.2], [9.0]])
labels = DBSCAN(eps=0.15, min_samples=2).fit_predict(pts)
print(labels, len(set(labels) - {-1}))
```

A) `[0 0 0 1 1 1 2] 3`

B) `[ 0 0 0 1 1 1 -1] 2`

C) `[ 0 0 0 1 1 1 -1] 3`

D) `[-1 -1 -1 -1 -1 -1 -1] 0`

> **Why:** Each group of three has crowded neighborhoods with min_samples=2; 9.0 is alone: noise. Two clusters: -1 is not one (Lesson 8.4).

W2

Why can you not score next month's customers with a fitted HDBSCAN?

A) It forgets its clusters after fit

B) It has fit_predict but no predict: it describes the data it saw

C) It needs the churn label

D) It only works on two features

> **Why:** Density methods describe their own data; a new point could even merge two clusters. Refit, or assign with a nearest-neighbor rule (Lesson 8.4, Exercise 4).

W3

What does this print?

```
import numpy as np
from sklearn.mixture import GaussianMixture
x = np.array([[0.0], [0.3], [0.5], [6.0], [6.4]])
gm = GaussianMixture(n_components=2, random_state=42).fit(x)
print(gm.predict_proba([[50.0]]).round(2).sum())
```

A) `0.0`

B) `0.5`

C) `1.0`

D) A very small number

> **Why:** Every row of predict_proba adds up to 1, even for a point far from everything (Lesson 8.5). How typical it is comes from score_samples: today's anomaly score.

W4

BIC for 1 to 6 components: 50,600; 47,200; 46,700; 46,300; 46,100; 46,050. Which reading is right?

A) 6, the minimum

B) The big drop is from 1 to 2; after that gains shrink: choose 2 to 4 and defend it with profiles

C) BIC cannot be used without a minimum

D) 1: simpler is always better

Why: These are the lifecycle-lab numbers. Read where the large drops stop; then let the profiles decide.

W5

A customer has probabilities 0.97 for one component but the lowest log-density in the data. Borderline, atypical, or both?

A) Borderline

B) Atypical

C) Both

D) Neither

Why: Confident about the nearest component (not borderline), but unlike every component (atypical). Today, atypical becomes an anomaly score.

8.6 · ANOMALY DETECTION: SELF-CHECK

Q1

What does this print?

```
import numpy as np
from sklearn.ensemble import IsolationForest
rng = np.random.default_rng(42)
X = np.vstack([rng.normal(0, 1, size=(49, 2)), [[8.0, 8.0]]])
pred = IsolationForest(contamination=0.02, random_state=42).fit(X).predict(X)
print(np.unique(pred, return_counts=True))
```

A) `(array([0, 1]), array([ 1, 49]))`

B) `(array([-1, 1]), array([ 1, 49]))`

C) `(array([-1, 1]), array([49, 1]))`

D) `(array([1]), array([50]))`

Why: Outlier detectors label outliers -1 and inliers 1, never 0. contamination=0.02 sets the threshold so that 2% of the 50 training rows, one row, falls below it: the planted point at (8, 8).

A is wrong: 0 is the classifiers' label; detectors use -1.

C is wrong: The counts are the wrong way round: one outlier, 49 inliers.

D is wrong: contamination=0.02 guarantees that about 2% of the training rows are flagged.

Q2

Your detector's ROC AUC on June is 0.12, computed from score_samples. What is the most likely explanation?

A) The detector ranks frauds worse than random: drop it

B) contamination is too low

C) Too few trees in the forest

D) score_samples is a normality score (higher = more normal); negate it and the AUC is 0.88

Why: score_samples is higher for normal points, so the frauds sit at the bottom. An AUC far BELOW 0.5 is a perfect ranking turned upside down: 1 - 0.12 = 0.88. Negate once and call it anomaly_score.

A is wrong: Worse than random by that much means the ranking is good, just reversed.

B is wrong: contamination does not change the scores at all, so it cannot change the AUC.

C is wrong: More trees make scores smoother, not reversed.

Q3

What does this print?

```
import numpy as np
from sklearn.ensemble import IsolationForest
X = np.random.default_rng(3).normal(size=(200, 2))
for c in [0.01, 0.05]:
    model = IsolationForest(contamination=c, random_state=42).fit(X)
    print(c, (model.predict(X) == -1).sum())
```

`0.01 2 / 0.05 10`

> **Why:** contamination sets only the threshold (offset_): 1% and 5% of 200 rows fall below it. The scores, and therefore the ranking, are identical for both models.

Q4

An Isolation Forest with contamination=0.02 fitted on January to March flags 2.16% of March but 1.72% of June. Why, and what does the lesson recommend?

A) The forest was refitted on June; fix the random_state

B) June's payments are distributed differently from the fit months, so a fixed threshold drifts: rank each month and review the top N the team can handle

C) contamination is a random setting; average several runs

D) Rounding: 2% of a smaller month is fewer payments

> **Why:** The threshold is a percentile of the training scores. A new month with a slightly different mix falls on either side of it differently. A top-N queue always fills the budget; watch the score distribution separately to notice drift.
> **A** is wrong: Nothing was refitted; the same model scored every month.
> **C** is wrong: contamination is a fixed number, not random.
> **D** is wrong: A percentage is a share; the size of the month does not change it.

Q5

In today's lab, the Isolation Forest's 60 most unusual customers churn at 0.067, well below the 0.165 average. What is the right conclusion?

A) The detector is broken: anomalies should churn more

B) Unusual customers are loyal; target them with fewer offers

C) An anomaly score finds the unusual, not the bad: these are heavy buyers; whether unusual means a problem is a separate question

D) Use the LOF's top 60 instead, because they churn at 0.333

> **Why:** The forest isolated heavy buyers (about 10 orders, $1,810 spend, many returns). Unusual, and mostly good customers. Lesson 8.6, Common Mistake 5: a score is a reason to look, never a verdict.
> **A** is wrong: Nothing says anomalies must churn; the detector did its job.
> **B** is wrong: Sixty customers chosen for being unusual say nothing about how to treat the rest.
> **D** is wrong: Choosing a detector by the outcome you hoped for is not a method; the LOF list answers a different question.

Q6

You fit Local Outlier Factor on May's payments and want to score June's. Which setup works?

A) `LocalOutlierFactor()`, then `predict(june)`

B) `LocalOutlierFactor()`, then `fit_predict(june)`

C) `LocalOutlierFactor(novelty=True)` fitted on May, then `-score_samples(june)`

D) `LocalOutlierFactor(novelty=True)` fitted on May, then `score_samples(may)`

> **Why:** novelty=True learns from one set and scores new rows. Negate for an anomaly score. Scaled features, because LOF uses distances.
> **A** is wrong: The default (novelty=False) has no predict: AttributeError.
> **B** is wrong: That fits a new LOF on June alone; May taught it nothing.
> **D** is wrong: Scoring the training rows in novelty mode is optimistic: each point is its own neighbor.

8.6 · ANOMALY DETECTION: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
from sklearn.metrics import roc_auc_score
print(roc_auc_score([0, 0, 0, 1], [0.9, 0.7, 0.8, 0.1]))
```

`0.0`

> **Why:** The one fraud has the LOWEST score, so the ranking is perfectly reversed: AUC 0.0. Exactly what `score_samples` without a minus sign does.

P2

Predict the output, then run it.

```
import numpy as np
pred = np.array([1, 1, -1, 1, 1, 1, -1, 1])
print((pred == 1).sum(), (pred == -1).sum())
```

`6 2`

> **Why:** Six inliers (1) and two outliers (-1). Counting `pred == 1` as outliers would report six.

9.1 · PRINCIPAL COMPONENT ANALYSIS: SELF-CHECK

Q7

What does this print?

```
import numpy as np
from sklearn.decomposition import PCA
rng = np.random.default_rng(42)
a, b = rng.normal(size=200), rng.normal(size=200)
X = np.column_stack([a, b, a + b])
print(PCA().fit(X).explained_variance_ratio_.round(3))
```

A) `[0.333 0.333 0.333]`

B) `[1. 0. 0.]`

C) `[0.5 0.5 0. ]`

D) `[0.732 0.268 0. ]`

> **Why:** The third column is exactly the sum of the first two, so the data lie in a plane: the last component explains nothing. The first two are not equal because a + b adds spread along one diagonal.
> **A** is wrong: Equal shares would need three unrelated columns.
> **B** is wrong: a and b are independent: two directions carry spread.
> **C** is wrong: The plane is not round: the copy a + b stretches it along one diagonal.

Q8

Unscaled PCA on the nine payment features gives a PC1 that explains 95% of the variance and is almost exactly account_age_days. Why?

A) Account age is the most important fraud signal

B) Account age is measured in days, up to about 7,000, so its variance dwarfs the 0/1 flags and small counts

C) PCA always puts the first column first

D) The other columns are categorical

> **Why:** Variance depends on units. Without scaling, PCA mostly tells you which column has the biggest numbers. Scaled, account age barely appears in PC1 or PC2.
> **A** is wrong: PCA never sees is_fraud.
> **C** is wrong: Column order plays no role.
> **D** is wrong: They are numeric 0/1 flags and counts; their variance is just small.

Q9

What does this print?

```
import numpy as np
from sklearn.decomposition import PCA
X = np.random.default_rng(1).normal(size=(50, 6))
p = PCA(n_components=3).fit(X)
print(p.transform(X).shape, p.components_.shape)
```

`(50, 3) (3, 6)`

> **Why:** transform gives one row per sample and one column per kept component; components_ has one row per component and one column per original feature (each row is a direction in the original space).

Q10

In which situation is PCA before a model most likely to help?

A) Ten business columns that each mean something, with the strongest signal in a low-variance column

B) Four categorical columns, one-hot encoded

C) A target that depends on one column PCA spreads over several components

D) 784 highly correlated pixel columns feeding a slow k-nearest-neighbors model

> **Why:** Many correlated columns of one unit: the first components carry most of the signal, the rest is noise, and a distance-based model gets faster. With ten meaningful business columns PCA usually costs accuracy (Lesson 9.1: two components AUC 0.687 against 0.829).
> **A** is wrong: Exactly the churn case where PCA lost the signal.
> **B** is wrong: The variance of 0/1 indicators is a poor measure of similarity.
> **C** is wrong: That is the weakness PCA has, not a reason to use it.

Q11

On June's payments, PCA reconstruction error reached AUC 0.819 but only 19 frauds in the 165-payment queue; Isolation Forest caught 32. What explains the gap?

A) Much of this fraud is extreme on a few columns rather than breaking the correlations most payments share

B) PCA leaked the labels

C) Four components were too many

D) Reconstruction error cannot be used as a score

> **Why:** Reconstruction error flags rows that break shared patterns. Many frauds here are simply extreme (high velocity, new accounts, many card attempts), which Isolation Forest isolates directly. PCA-based detection shines where normal data has strong correlated structure, like sensors or sales.
> **B** is wrong: PCA never saw is_fraud.
> **C** is wrong: The number of components changes the score a little; it does not explain the pattern.
> **D** is wrong: It worked (AUC 0.819), just with a less pure top of the ranking.

Q12

On the ten scaled churn features, how many components does `PCA(n_components=0.90)` keep, and is that a useful reduction?

`7` : hardly a reduction

> **Why:** The cumulative shares reach 0.905 at seven of ten components. The scree is flat: these business columns are only partly redundant, so PCA cannot compress them much.

9.1 · PRINCIPAL COMPONENT ANALYSIS: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
import numpy as np
from sklearn.decomposition import PCA
rng = np.random.default_rng(0)
X = rng.normal(size=(100, 5))
print(PCA(n_components=2).fit(X).components_.shape)
```

```
(2, 5)
```

Why: `components_` has one row per kept component and one column per original feature.

P4

Predict the output, then run it.

```
import numpy as np
from sklearn.decomposition import PCA
x = np.array([1.0, 2.0, 3.0, 4.0, 5.0])
X = np.column_stack([x, 3 * x, -x])
print(PCA().fit(X).explained_variance_ratio_.round(2))
```

```
[1. 0. 0.]
```

Why: Every column is a multiple of x, so all points lie on one line: one component carries all the variance.

LAB REFERENCE SOLUTIONS

- LAB 8.6-A: LABS/MODULE-05/L8_6-ANOMALY-DETECTION/MONTHLY_QUEUE_STARTER.PY → MONTHLY_QUEUE_SOLUTION.PY

The solution prints:

```
fit on January to March: 24,881 payments (labels not used)
April AUC of score_samples : 0.153  (higher = MORE normal)
April AUC of -score_samples: 0.847  (the anomaly score)
       payments  frauds  budget  caught    auc  random_queue
month
4          8301      72     166      33  0.847           1.4
5          8556      65     171      35  0.872           1.3
6          8262      64     165      33  0.848           1.3
month 1: fixed threshold flags 1.88%
month 2: fixed threshold flags 1.97%
month 3: fixed threshold flags 2.16%
month 4: fixed threshold flags 1.92%
month 5: fixed threshold flags 1.88%
month 6: fixed threshold flags 1.72%
```

Reading it: the raw score gives AUC 0.153, the same ranking upside down (1 - 0.153 = 0.847). Negated, a forest fitted once on the first quarter keeps catching about half of each month's frauds in a 2% queue (33, 35, 33), roughly 25 times a random queue. The fixed threshold, a percentile of the January to March scores, flags a different share each month (1.72% to 2.16%): ranking and reviewing the top N fills the budget exactly every month.

- LAB 8.6-B: LABS/MODULE-05/L8_6-ANOMALY-DETECTION/UNUSUAL_CUSTOMERS_STARTER.PY → UNUSUAL_CUSTOMERS_SOLUTION.PY

The solution prints:

```
     iso_score  lof_score
min      0.394      0.954
50%      0.466      1.048
max      0.685      1.757

rank correlation of the two scores: 0.515
customers in both top-60 lists: 9
           orders_12m  spend_12m  avg_order_value  returns_12m  support_tickets_12m  d
iso_top60
False            4.96     586.01           140.38         0.55                 0.93
True             9.67    1810.44           194.32         4.78                 2.32
           orders_12m  spend_12m  avg_order_value  returns_12m  support_tickets_12m  d
lof_top60
False            5.00     595.67           140.55         0.59                 0.94
True             6.23     853.81           177.53         1.27                 2.05

churn: overall 0.165, IF top 60 0.067, LOF top 60 0.333
```

Reading it: the two scores agree only moderately (0.515) and share 9 of 60 customers. Isolation Forest isolates globally extreme customers: heavy buyers with many returns and tickets, who churn LESS (0.067). LOF finds customers who are odd for their neighborhood, closer to average overall, and they churn at 0.333. Neither list is "the bad customers": an anomaly score says where to look; the profile says what you found.

HANDOUT: LABS/MODULE-05/L8_6-ANOMALY-DETECTION/DEBUG_ANOMALY.PY (DELIBERATELY BROKEN)

1. Silent: `train = tx.sample(frac=0.8, ...)` mixes June into training → `train = tx[tx["timestamp"] < "2025-06-01"]`.
2. Silent: `(pred == 1).mean()` counts inliers → `(pred == -1).mean()`: 1.65% flagged in June.
3. Silent: `score = iso.score_samples(test)` is a normality score → negate it: AUC 0.848, 32 frauds in the top 165 (contamination does not change these).
4. `AttributeError: This 'Pipeline' has no attribute 'score_samples'` (the default LOF) → `LocalOutlierFactor(n_neighbors=50, novelty=True)`: AUC 0.757.

LAB 9.1-A: LABS/MODULE-05/L9_1-PCA/PAYMENT_PCA_STARTER.PY → PAYMENT_PCA_SOLUTION.PY

The solution prints:

```
(41738, 9)
unscaled, share of variance (first 3): [0.95 0.05 0.  ]
{'account_age_days': 1.0, 'amount': -0.001, 'hour': 0.0, 'items': -0.0, 'device_new': 

scaled, share of variance: [0.159 0.123 0.112 0.111 0.11  0.108 0.108 0.105 0.064]
cumulative:                [0.159 0.281 0.394 0.505 0.615 0.723 0.831 0.936 1.   ]
components kept for 90%: 8
                            PC1   PC2
account_age_days          -0.02 -0.17
amount                     0.70 -0.07
hour                      -0.03 -0.23
items                      0.70 -0.13
device_new                 0.04  0.35
billing_shipping_mismatch  0.03  0.28
country_mismatch           0.05  0.39
card_attempts              0.09  0.52
velocity_24h               0.09  0.52
           PC1   PC2
is_fraud
0        -0.01 -0.03
1         1.56  3.10
fraud rate among the 834 highest PC1 scores: 0.084 (base rate 0.009)
fraud rate among the 834 highest PC2 scores: 0.206 (base rate 0.009)
```

Reading it: unscaled, PC1 is account age (days, up to about 7,000) and nothing else. Scaled, the variance is spread over eight of nine components, so PCA cannot compress these features much. PC1 is basket size (amount and items move together); PC2 collects the risk flags (card attempts, velocity, country mismatch, new device). Frauds sit far out on PC2 (3.10), and the top 2% of PC2 is 21% fraud. A direction of spread happened to carry the fraud signal: you know only because you checked, since PCA never saw the label.

LAB 9.1-B: LABS/MODULE-05/L9_1-PCA/DAILY_SALES_PCA_STARTER.PY → DAILY_SALES_PCA_SOLUTION.PY

The solution prints:

```
1096 days, 6 closed days left out: (1090, 4)
share of variance: [0.841 0.067 0.051 0.042]
            PC1   PC2
store
Asheville  0.49  0.70
Bend       0.47  0.21
Boise      0.53 -0.20
Flagstaff  0.51 -0.66
                  size  mean
event
none              1036 -0.05
Holiday rush        18  0.43
Memorial Day sale   12  1.16
Labor Day sale      12  1.19
Black Friday        12  1.23
date
2023    0.117
2024    0.013
2025   -0.130
Name: PC2, dtype: float64
store  Asheville    Bend   Boise  Flagstaff
date
2023      4622.0  5187.0  6868.0     4997.0
2024      4675.0  5264.0  7864.0     5776.0
2025      4405.0  5558.0  8388.0     6597.0

mean absolute gap from PC1 alone: 0.105 log units (about 11.1% of a day's sales)
```

Reading it: one unit and strong correlation, so no scaling and a dominant PC1 (84%) with equal weights: how big the day was for the whole chain (sale events score highest). PC2 contrasts Asheville with Flagstaff and drifts down year by year: Flagstaff grows while Asheville stays flat. PC1 alone rebuilds a day to within about 11%. Closed days were left out because log(0) is undefined and six all-zero days would dominate a component.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 8.6 · ANOMALY DETECTION

1. **A** Random cuts isolate them in fewer steps, so their average path length is short.
2. **C** `[-1 1]`: outliers -1, inliers 1; a few fringe points of the first 50 are flagged.
3. **B** Only the threshold changes: more points labeled -1, in the same order.
4. **A** Local Outlier Factor: it compares the point with its own dense neighborhood.
5. **C** How many frauds land in the top 165, compared with a baseline.

1. **B** One column with large units dominates the variance.
2. **C** `[1. 0.]`: b = 2a, so all points lie on one line.
3. **A** The directions with the most spread are not the directions that predict churn.
4. **B** Nothing important: a component's sign is arbitrary.
5. **C** All columns share one unit, and their different spreads are meaningful.

EXIT TICKET

1. `anomaly_score = -model.score_samples(X)`; `queue = np.argsort(-anomaly_score)[:165]`. Forget `predict`'s labels; count frauds in the queue.
2. contamination moves only the threshold; scores, ranking and AUC do not change. Decide how many to review from the team's capacity and the costs ($10 per review, $375 per missed fraud in the lesson).
3. The column with the largest units (spend in dollars) took over PC1. Skip scaling only when all columns share one meaningful unit (monthly sales, pixel brightness).

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 A (`[0, 1]`).** Classifier habit. Write the convention table again.
- **Q2 A (drop the detector).** Has not met a flipped sign. Show 1 - 0.12.
- **Q4 A or C.** Thinks the model changed. Same model, different month.
- **Q5 A or D.** Expects anomaly to mean bad. Reread Common Mistake 5.
- **Q8 A.** Thinks PCA knows about fraud. It never sees labels.
- **Q12 written as 2 or 3.** Reads 90% as "a few components"; the scree is flat here.

INSTRUCTOR KIT · ANSWER KEY

Session 6: LDA, t-SNE and UMAP

Answers and reasoning for every Session 6 item: the warm-up, both self-checks, the predict-then-run items, Checkpoint 1's rubric and model answer, the lab reference outputs, the lesson pages' own quizzes and the exit ticket.

Session 6 | 12 self-check items | Checkpoint 1 | Confidential

WARM-UP ANSWERS

Retrieval on Lessons 8.6 and 9.1. Not graded.

W1

What does this print?

```
import numpy as np
pred = np.array([1, -1, 1, 1, -1, 1])
print((pred == -1).sum(), (pred == 1).mean().round(2))
```

A) `4 0.33`

B) `2 0.67`

C) `2 0.33`

D) `4 0.67`

> **Why:** -1 is the outlier label: two of them; 1 marks inliers, four of six (Lesson 8.6).

W2

You raise an Isolation Forest's contamination from 0.01 to 0.05. What changes?

A) The scores

B) The ranking

C) Only the threshold: more rows get -1, in the same order

D) The trees

> **Why:** contamination only sets offset_. Choose the threshold from the review budget.

W3

What does this print?

```
import numpy as np
from sklearn.decomposition import PCA
x = np.arange(6.0)
X = np.column_stack([x, 2 * x + 1])
print(PCA().fit(X).explained_variance_ratio_.round(2))
```

A) `[0.5 0.5]`

B) `[0.67 0.33]`

C) `[1. 0.]`

D) `[0.8 0.2]`

> **Why:** The second column is an exact line through the first, so all points lie on one line: one component (Lesson 9.1).

W4

Logistic regression on the first two principal components of the churn columns scored AUC 0.687, against 0.829 on all ten. Why?

A) PCA leaked

B) PCA picks directions of spread, not of churn; the recency signal is spread over several components

C) Two features are too few for any model

D) The features were not scaled

Why: Today's Lesson 9.2 fixes exactly this: a direction chosen WITH the labels.

W5

When should you NOT standardize before PCA?

A) Never

B) When columns have different units

C) When all columns share one meaningful unit, like monthly sales or pixels

D) When there are many rows

Why: Standardizing would erase real differences in spread. Today's pixels are a one-unit table too.

9.2 · LINEAR DISCRIMINANT ANALYSIS: SELF-CHECK

Q1

What does this print?

```
from sklearn.datasets import make_blobs
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
X, y = make_blobs(n_samples=200, centers=4, n_features=2, random_state=1)
print(LinearDiscriminantAnalysis().fit(X, y).transform(X).shape)
```

A) `(200, 3)`

B) `(200, 2)`

C) `(200, 4)`

D) `(200, 1)`

Why: LDA gives at most min(features, classes - 1) axes: four classes would allow three, but there are only two features, so two.

A is wrong: Three would need at least three features.

C is wrong: Never one axis per class: at most classes minus one.

D is wrong: One axis is the two-class case.

Q2

What does LDA choose a direction for, compared with PCA?

A) Class means far apart relative to the spread inside each class; PCA chooses the largest total spread and never sees the labels

B) The largest total spread, like PCA, but faster

C) Curved directions where PCA uses straight ones

D) The direction of the most important single feature

Why: Fisher's criterion: between-class spread over within-class spread. The team photo: each club bunched up, the clubs far apart.

B is wrong: That is PCA's goal.

C is wrong: Both are linear; curves are Lesson 9.3.

D is wrong: An LDA axis is a weighted mix of all features.

Q3

What does this print?

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
X = [[1.0], [2.0], [3.0], [9.0], [10.0]]
y = ["a", "a", "a", "b", "b"]
print(LinearDiscriminantAnalysis(priors=[0.5, 0.5]).fit(X, y).priors_)
```

`[0.5 0.5]`

> **Why:** Priors you pass replace the class shares learned from the data (here they would be 0.6 and 0.4). Priors move the decision threshold; they never change the axis or the ranking.

Q4

LDA on ten payment categories gives nine axes, but the first carries 0.995 of the separation and leans on amount (+1.42) and items (-0.59). What is the best reading?

A) Nine useful directions: plot them all

B) The categories are perfectly separable

C) The payment details separate categories along essentially one direction, price per item, and only weakly (accuracy 0.290 against 0.219)

D) Amount and items were not scaled

> **Why:** explained_variance_ratio_ for LDA is the share of class separation. Nearly all of it is on LD1, which is high amount with few items: an expensive single product. Six categories are never predicted. A weak, honest result.
> **A** is wrong: Eight of the axes carry almost nothing.
> **B** is wrong: 29% accuracy is far from separable.
> **D** is wrong: The pipeline scaled them; the weights are comparable.

Q5

On 400 customers with 290 added noise columns and SHUFFLED churn labels, LDA fitted on all rows and then cross-validated scores AUC 0.998. Inside a pipeline it scores 0.481. Why?

A) The real columns still carry churn signal

B) kNN with 15 neighbors overfits

C) Shuffling did not work

D) LDA used every row's label, including the test folds', to build its axis; with 300 columns it can separate any labels

> **Why:** A supervised reducer outside cross-validation turns the test labels into a feature. Inside the pipeline each fold builds its axis from its training rows only, and the score falls to chance, the truth for shuffled labels.
> **A** is wrong: The labels were shuffled: there is no signal left to find.
> **B** is wrong: kNN cannot invent 0.998 on random labels.
> **C** is wrong: The honest score of 0.481 shows the shuffle worked.

Q6

You have 80 rows, 1,000 features, and want an LDA plot AND classification. Which setting?

A) `LinearDiscriminantAnalysis()`

B) `LinearDiscriminantAnalysis(solver="eigen", shrinkage="auto")`

C) `LinearDiscriminantAnalysis(solver="lsqr", shrinkage="auto")`

D) `LinearDiscriminantAnalysis(shrinkage="auto")`

> **Why:** Far more features than rows needs shrinkage; only the eigen solver supports both shrinkage and transform.
> **A** is wrong: No shrinkage: a noisy covariance estimate.
> **C** is wrong: lsqr classifies but has no transform.
> **D** is wrong: The default svd solver raises NotImplementedError with shrinkage.

9.2 · LINEAR DISCRIMINANT ANALYSIS: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
from sklearn.datasets import make_blobs
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
X, y = make_blobs(n_samples=90, centers=3, n_features=5, random_state=0)
print(LinearDiscriminantAnalysis().fit(X, y).transform(X).shape)
```

```
(90, 2)
```

> **Why:** min(5 features, 3 - 1) = 2 axes, one row per sample.

P2

Predict the output, then run it.

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
X = [[0.0], [0.5], [1.0], [5.0]]
y = [0, 0, 0, 1]
print(LinearDiscriminantAnalysis().fit(X, y).priors_)
```

```
[0.75 0.25]
```

> **Why:** Learned priors are the class shares in the training labels: 3 of 4 and 1 of 4.

9.3 · T-SNE AND UMAP: SELF-CHECK

Q7

On a t-SNE map, segment A is a big island and segment B a small island far away. What can you conclude?

A) A is more varied than B

B) B is very different from every other segment

C) Little about size or distance: check spread and separation in the original data

D) B is noise

> **Why:** t-SNE keeps neighborhoods, not sizes or gaps (the three-blob demo: a seven-fold spread difference became under 1.5).
> **A** is wrong: Map sizes are equalized by the adaptive bell curve.
> **B** is wrong: Gaps between islands are not reliable.
> **D** is wrong: Nothing on a map labels noise; that needs a density method in the original space.

Q8

In today's fraud-map lab, 0.630 of a fraud's map neighbors are frauds, and 0.614 of its neighbors in the original space are. What does that tell you about the colleague's "fraud is an island" map?

A) The map reproduces a neighborhood structure that exists in the data; now confirm any group in the original space (a clustering found 239 frauds together, 199 elsewhere)

B) The map created the fraud island

C) The map is wrong because the numbers differ

D) Use a lower perplexity to make the island clearer

> **Why:** A map is trustworthy where its neighbor numbers match the original space. The claim is partly right: about half the frauds form a tight group; the rest mix with honest payments.
> **B** is wrong: The same share exists before any map was drawn.
> **C** is wrong: 0.630 against 0.614 is close: the map preserved it.
> **D** is wrong: Tuning a picture until it looks clearer is how maps mislead.

Q9

What happens when this runs?

```
import numpy as np
from sklearn.manifold import TSNE
X = np.random.default_rng(0).normal(size=(12, 3))
TSNE(random_state=42).fit_transform(X)
```

`ValueError: perplexity (30.0) must be less than n_samples (12)`

> **Why:** The default perplexity is 30, and a point cannot attend to 30 neighbors among 12 rows. Use a much smaller perplexity (well below a third of the rows) or more rows.

Q10

Each day's new customers must be placed on an existing map, and the same embedding should feed a model inside a pipeline. Which method?

A) t-SNE with a fixed random_state

B) t-SNE rerun every day on all customers

C) PCA to 2 components, because only PCA has transform

D) UMAP: fit once, then transform new rows; it can sit in a pipeline

> **Why:** A fitted UMAP has transform (0.690 of 500 new images landed among their own class in the demo); t-SNE learns no function for new rows.
>
> **A** is wrong: A seed does not give t-SNE a transform.
>
> **B** is wrong: The whole layout changes every day.
>
> **C** is wrong: UMAP also has transform, and keeps neighborhoods far better than a 2-component PCA view.

Q11

Why reduce 784 pixels to about 50 PCA components before t-SNE?

A) t-SNE refuses more than 50 columns

B) Speed, and the last few hundred components of image data are mostly noise

C) PCA adds the labels

D) It makes the map's axes interpretable

> **Why:** Distances on 50 columns are much cheaper, and dropping noisy low-variance components makes neighborhoods cleaner.
> **A** is wrong: t-SNE accepts any width, just slowly.
> **C** is wrong: PCA never sees labels.
> **D** is wrong: t-SNE axes never mean anything.

Q12

A t-SNE map of 200 rows at perplexity 5 shows six clumps. What do you do before telling anyone there are six groups?

A) Nothing: clumps are clusters

B) Change the seed until the clumps are clearer

C) Raise min_dist

D) Try several perplexities and confirm with a clustering and its profiles in the original space

> **Why:** At low perplexity t-SNE makes clumps out of pure noise (the demo). Groups must be validated in the original data.
> **A** is wrong: The pure-noise map shows clumps from nothing.
> **B** is wrong: Seed-shopping for a picture is not evidence.
> **C** is wrong: min_dist is a UMAP setting, and it changes packing, not truth.

9.3 · T-SNE AND UMAP: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
from sklearn.manifold import TSNE
import umap
print(hasattr(TSNE(), "transform"), hasattr(umap.UMAP(), "transform"))
```

```
False True
```

Why: TSNE has only fit_transform; a fitted UMAP has transform.

P4

Predict the output, then run it.

```
import numpy as np
from sklearn.manifold import TSNE
X = np.random.default_rng(0).normal(size=(25, 4))
print(TSNE(perplexity=5, random_state=42).fit_transform(X).shape)
```

```
(25, 2)
```

Why: Perplexity 5 is below 25 rows, so it runs: one 2-D point per row.

CHECKPOINT 1: RUBRIC AND MODEL ANSWER

Five rows, 0 to 2 points each (10 in all). Score from the learner's file and Part D. 8 to 10: on track. 5 to 7: review the weakest module before Session 8. Under 5: a one-to-one before Module 10.

CRITERION	STRONG (2)	PARTIAL (1)	MISSING (0)
Leak-free setup	Uses the given split; the scaler for Part C is fitted on X_train only; churn is never a clustering feature; permutation importance on the TEST rows	One slip (scaler fitted on all rows, or importance on training rows), noticed or not	Several leaks, or churn used as a feature

CRITERION	STRONG (2)	PARTIAL (1)	MISSING (0)
Interpretation (Module 7)	Top columns days_since_last_order (0.154) and discount_share (0.099); says this describes the MODEL, not causes	Right columns, no caveat (or caveat with the wrong scoring)	Coefficients or impurity importance quoted instead, or no result
Soft segments (Module 8)	Five components profiled by discount and churn; the deal-driven one named by its profile (churn about 0.30); 305 below 0.8 confidence reported as borderline	Components found but named by number, or no confidence count	Hard labels only, or churn used to fit
Projections (Module 9)	7 components for 90%; LD1 led by days_since_last_order (0.92) while PC1 leads with activity columns; AUC LD1 0.824 vs PC1 0.631 with the sign handled	Counts right but no PC1/LD1 comparison, or PC1's sign mishandled	PCA and LDA confused, or LDA fitted on the test rows
Honest written conclusions (Part D)	Three short sentences with one number each, a caveat about causality, and no over-claiming	Sentences without numbers, or one over-claim	Missing, or claims the model proves causes

MODEL ANSWER: LABS/MODULE-06/CHECKPOINT-1/CHECKPOINT_SOLUTION.PY PRINTS

```
A. drop in test AUC when each column is shuffled (top 3):
days_since_last_order    0.154
discount_share           0.099
support_tickets_12m      0.023

B. components (churn used only to profile):
           customers  discount  churn
component
0               1317     0.495  0.298
1               1316     0.095  0.043
2               1287     0.139  0.201
3                975     0.171  0.113
4               1105     0.112  0.158
customers below 0.8 confidence: 305

C. components for 90% of the variance: 7
largest LD1 weight: days_since_last_order 0.92
largest PC1 loading: returns_12m
test AUC of one number: PC1 0.631 (sign-free)   LD1 0.824
```

Reading it. A: shuffling `days_since_last_order` costs 0.154 of test AUC and `discount_share` 0.099; everything else is small. B: component 0 is the deal-driven group (discount share 0.495, churn 0.298), component 1 the frequent low-discount regulars (0.043); numbers are arbitrary, names come from the profile; 305 customers are borderline. C: seven of ten components are needed for 90% (the scree is flat); LD1 leans on recency (0.92) while PC1's largest loading is an activity column (`returns_12m`); one number each: LD1 AUC 0.824 against PC1 0.631.

Model Part D. (1) "Our churn model leans most on how long since a customer's last order (shuffling it costs 0.15 of AUC), then on discount reliance; this describes the model, not what causes churn." (2) "Act first on the discount-driven segment: about 1,300 customers who churn at 30%; leave the 305 borderline customers out of narrowly targeted offers." (3) "For a churn map, use a direction chosen with the churn labels: LDA's single axis ranks customers with AUC 0.82, while the direction of most spread manages 0.63."

LAB REFERENCE SOLUTIONS

- LAB 9.2-A: LABS/MODULE-06/L9_2-LDA/PAYMENT_CATEGORY_LDA_STARTER.PY → PAYMENT_CATEGORY_LDA_SOLUTION.PY

The solution prints:

```
(37500, 9) classes: 10
axes: 9
separation carried: [0.995 0.003 0.001 0.001 0.001 0.    0.    0.    0.   ]
items                       -0.59
card_attempts               -0.04
billing_shipping_mismatch   -0.01
velocity_24h                -0.01
device_new                   0.00
account_age_days             0.01
country_mismatch             0.01
hour                         0.01
amount                       1.42
                LD1  amount
category
Accessories  -0.93   64.98
Camp Kitchen -0.77   87.76
Apparel      -0.38  138.11
Gift Cards   -0.26  150.00
Climbing     -0.24  153.48
Packs        -0.09  166.90
Footwear      0.01  183.16
Sleeping      0.38  227.25
Snow          0.42  239.53
Tents         1.41  350.48

baseline accuracy 0.219   LDA accuracy 0.290
recall by category: {'Climbing': 0.0, 'Camp Kitchen': 0.0, 'Packs': 0.0, 'Gift Cards':
```

Reading it: ten classes allow nine axes, but separation lives on one: high amount with few items (an expensive single product: tents, snow gear, sleeping bags) against cheap multi-item baskets (accessories, camp kitchen). As a classifier it barely beats the baseline and never predicts six categories: payment details say little about what was bought. An honest weak result, explained by one axis.

LAB 9.2-B: LABS/MODULE-06/L9_2-LDA/NOISE_LEAK_STARTER.PY → NOISE_LEAK_SOLUTION.PY

The solution prints:

```
(400, 300) churn rate 0.175; chance accuracy of 'always stays' 0.825
shuffled labels  AUC  LDA outside CV 0.998   LDA inside the pipeline 0.481
true labels      AUC  LDA outside CV 1.000   LDA inside the pipeline 0.541
true labels, 10 real columns only, inside the pipeline: AUC 0.776
```

Reading it: with 300 columns and 400 rows, LDA fitted on all rows separates even shuffled labels (0.998); inside the pipeline the score is chance (0.481). With the true labels the leaky number is also meaningless (1.000), and the honest one is weak (0.541) because 290 noise columns swamp the covariance estimate; the ten real columns alone reach 0.776. Two lessons: keep supervised steps inside the pipeline, and do not add columns hoping a reducer will sort them out.

LAB 9.3-A: LABS/MODULE-06/L9_3-TSNE-UMAP/FASHION_MAP_STARTER.PY → FASHION_MAP_SOLUTION.PY

The solution prints (t-SNE and UMAP digits may vary slightly on another machine):

```
(1500, 784) -> (1500, 50) variance kept 0.869
PCA (first 2)  neighbors kept 0.195   trustworthiness 0.922
t-SNE          neighbors kept 0.573   trustworthiness 0.991
UMAP           neighbors kept 0.495   trustworthiness 0.986
Shirt          0.32
Coat           0.51
Pullover       0.62
Sneaker        0.71
Dress          0.82
Bag            0.87
Sandal         0.87
Ankle boot     0.88
T-shirt/top    0.88
Trouser        0.90

500 new images placed (500, 2); landing among their own class: 0.730
saved fashion_umap.png
```

Reading it: both nonlinear maps keep several times more true neighbors than a two-component PCA view; t-SNE keeps the most local detail. The garments the map mixes up (shirt, coat, pullover) are the ones that look alike at 28 by 28 pixels; trousers and footwear form clean islands. UMAP places 500 images it never saw with 73% landing among their own class, which t-SNE cannot do at all. The figure goes to `OUTPUT`, never next to the data.

LAB 9.3-B: LABS/MODULE-06/L9_3-TSNE-UMAP/FRAUD_MAP_STARTER.PY → FRAUD_MAP_SOLUTION.PY

The solution prints:

```
(1500, 9) fraud share in the sample 0.292 (in all payments 0.009)
original space: 0.614 of a fraud's 10 nearest neighbors are frauds
perplexity  5: neighbors kept 0.633   fraud share among a fraud's map neighbors 0.630
perplexity 30: neighbors kept 0.696   fraud share among a fraud's map neighbors 0.631

K-Means k=2 in the original space: silhouette 0.406
is_fraud     0    1
cluster
0           20  239
1         1042  199
```

Reading it: the maps reproduce the fraud neighborhoods that already exist in the scaled data (0.63 against 0.61), so the red island is not an artifact; but the evidence is the original space, where K-Means puts 239 frauds with only 20 honest payments, and the other 199 frauds sit among honest ones. "Fraud is its own cluster" is half true: about half of it is.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 9.2 · LINEAR DISCRIMINANT ANALYSIS

1. **C** (300, 3): min(3 features, 5 - 1) axes.
2. **A** PCA maximizes total variance; LDA between-class separation relative to within-class spread.
3. **B** LDA saw the test folds' labels while building its axes.
4. **A** More flagged, recall rises, ROC AUC unchanged.
5. **C** solver="eigen", shrinkage="auto".

LESSON 9.3 · T-SNE AND UMAP

1. **C** Very little about size or distance; check those in the original data.
2. **A** True False: TSNE has fit_transform but no transform.
3. **B** UMAP, using transform on the new rows.
4. **A** The map makes the segments look better separated than they are.
5. **C** Try other perplexities and check for clusters in the original data.

EXIT TICKET

1. LDA gives at most classes minus one axes (one for two classes); PCA gives one component per column.
2. Cluster sizes and distances between clusters (and the axes).
3. UMAP: fit once on today's customers, then `transform` tomorrow's through the same fitted scaler or PCA (best as one pipeline).

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 C ((200, 4)).** One axis per class. Recite: classes minus one, and never more than features.
- **Q4 A (nine useful directions).** Reads the axis count, not the separation shares.
- **Q5 A.** Thinks shuffled labels keep signal. Ask what shuffling destroys.
- **Q7 A or B.** Reads map sizes and gaps; the five rules again.
- **Q10 C.** Thinks only PCA can transform; UMAP can.
- **Checkpoint, Part C:** scaler fitted on all rows, or PC1's AUC reported as 0.369 (a flipped sign: PCA signs are arbitrary).

INSTRUCTOR KIT · ANSWER KEY

Session 7: Neural Networks and PyTorch

Answers and reasoning for every Session 7 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs and debug handouts, the lesson pages' own quizzes and the exit ticket.

Session 7 | 12 self-check items | Confidential

WARM-UP

Retrieval on Lessons 9.2 and 9.3. Accept answers in the learner's own words.

W1

What does this print? (Lesson 9.2)

```
from sklearn.datasets import make_classification
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
X, y = make_classification(n_samples=300, n_features=6, n_informative=4, n_classes=
print(LinearDiscriminantAnalysis().fit(X, y).transform(X).shape)
```

`(300, 2)`: LDA gives at most (classes - 1) axes, so 3 classes give 2 columns, whatever the number of features.

W2

PCA and LDA both flatten data onto a few directions. What is the one big difference in how they choose them? (9.2)

PCA looks only at X and keeps the directions of largest total spread; LDA uses the labels y and keeps the directions where the class means are far apart relative to the spread inside each class.

W3

A colleague fits LDA on all 200 rows of 500 random features, then cross-validates a classifier on the LDA output and gets 0.99. What happened? (9.2)

Leakage: LDA saw the labels of the rows later used as test folds. With 500 features it can separate any labels. Inside a pipeline, refitted per fold, the score falls to chance.

W4

On a t-SNE map of customers, one island is twice the size of the others and far away from them. What may you conclude from the size and the gap? (9.3)

Very little: t-SNE keeps neighborhoods, not sizes or distances between groups. Check spread and separation in the original data (silhouette, profiles).

W5

Next month 200 new customers arrive and you want to place them on this month's 2-D map. t-SNE or UMAP, and why? (9.3)

UMAP: a fitted UMAP has transform for new rows. t-SNE only has fit_transform; it positions the rows it was given and learns no function.

10.1 · NEURAL NETWORKS FROM SCRATCH: SELF-CHECK

Q1

A hidden layer takes 10 inputs and has 16 units. How many parameters (weights and biases) does it have?

A) 160

B) 176

C) 26

D) 170

Why: One weight per input per unit (10 × 16 = 160) plus one bias per unit (16): 176. The lesson's churn network adds the output layer's 16 weights and 1 bias for 193 in all.

A is wrong: That forgets the 16 biases.

C is wrong: Adding inputs and units counts nothing real.

D is wrong: Ten biases would be one per input; biases belong to units.

Q2

What does this print?

```
import numpy as np
x = np.array([1.0, 2.0])
w = np.array([0.5, -1.0])
z = x @ w + 1.0
print(z, max(0.0, z), round(1 / (1 + np.exp(-z)), 3))
```

`-0.5 0.0 0.378`

> **Why:** z = 1.0 × 0.5 + 2.0 × (-1.0) + 1.0 = -0.5. ReLU of a negative number is 0.0; sigmoid(-0.5) = 0.378. A ReLU unit with a negative z for a row passes no gradient back for that row.

Q3

After 2,000 epochs your network's training loss is still `0.6931` and its test accuracy is 0.500. Which bug is the most likely?

A) The learning rate is far too large

B) The test set is too small

C) Every weight was initialized to zero

D) The network has too many hidden units

> **Why:** 0.6931 is log(2), the coin-flip loss, and it did not move at all. With all-zero weights every ReLU unit outputs 0 and passes no gradient, so only the output bias can learn (the lesson's Mistake 3).
> **A** is wrong: A wild learning rate makes the loss jump around or explode, not sit perfectly still at 0.6931.
> **B** is wrong: The training loss itself is stuck; the test set has nothing to do with it.
> **D** is wrong: Extra units add capacity; they do not freeze learning.

Q4

What does this print?

```
w = 0.0
for step in range(2):
    w = w - 0.25 * 2 * (w - 1)        # gradient descent on (w - 1) ** 2
print(w)
```

`0.75`

Why: The slope of $(w - 1)^2$ is $2(w - 1)$. Step 1: $w = 0 - 0.25 \times (-2) = 0.5$. Step 2: $w = 0.5 - 0.25 \times (-1) = 0.75$. Each step halves the distance to the valley at 1.

Q5

Why is ReLU the default activation for hidden layers rather than the sigmoid?

A) Its slope is 1 wherever it is active, so gradients pass back through many layers undiminished

B) It outputs probabilities between 0 and 1

C) It keeps the network linear, which trains faster

D) It has no weights of its own, unlike the sigmoid

Why: Learning is driven by slopes. The sigmoid's flat ends have slopes near 0, so stacked sigmoid layers pass back ever smaller gradients (the vanishing gradient problem). ReLU's slope is exactly 1 where it is on.
B is wrong: That describes the sigmoid, which is why it sits at the OUTPUT of a yes-or-no classifier.
C is wrong: ReLU is nonlinear (its bend is the point); without a nonlinearity the layers collapse into one.
D is wrong: Neither activation has weights; that is not the difference.

Q6

What happens when this runs?

```
import torch
w = torch.tensor([1.0, 2.0], requires_grad=True)
out = (w * 3).sum()
out.numpy()
```

A) It prints `9.0`

B) It prints `[3. 6.]`

C) `TypeError`: a sum cannot be converted to NumPy

D) `RuntimeError`: Can't call numpy() on Tensor that requires grad

> **Why:** `out` was computed from a tensor with `requires_grad=True`, so it carries its gradient history and NumPy arrays cannot. Use `out.detach().numpy()`, or compute it inside `with torch.no_grad():` (the lesson's Mistake 6).
> **A** is wrong: That would be the value (3 + 6) if the conversion were allowed.
> **B** is wrong: The sum is a single number; and the conversion is refused anyway.
> **C** is wrong: The message is a RuntimeError about gradients, not a TypeError.

10.1 · NEURAL NETWORKS FROM SCRATCH: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
import numpy as np
X = np.ones((5, 3))
W = np.full((3, 2), 2.0)
print((X @ W).shape, (X @ W)[0])
```

`(5, 2) [6. 6.]`

> **Why:** Inner dimensions (3 and 3) match and disappear: (5, 2). Each entry is three products of 1 × 2, so 6.0.

P2

Predict the output, then run it.

```
import torch
w = torch.tensor(1.5, requires_grad=True)
loss = (2 * w) ** 2
loss.backward()
print(w.grad)
```

`tensor(12.)`

> **Why:** loss = $4w^2$, whose slope is 8w; at w = 1.5 that is 12. One call of backward, so no accumulation.

10.2 · PYTORCH FUNDAMENTALS: SELF-CHECK

Q7

What does this print?

```
from torch import nn
layer = nn.Linear(4, 3)
print(layer.weight.shape, sum(p.numel() for p in layer.parameters()))
```

`torch.Size([3, 4]) 15`

> **Why:** `nn.Linear(4, 3)` stores its weight as (outputs, inputs) = (3, 4), the transpose of Lesson 10.1's convention, and computes `x @ weight.T + bias`. Parameters: 12 weights + 3 biases = 15.

Q8

What happens when this runs?

```
import torch
from torch import nn
logits = torch.zeros(8, 1)
targets = torch.ones(8)
nn.BCEWithLogitsLoss()(logits, targets)
```

A) `ValueError`: Target size (torch.Size([8])) must be the same as input size (torch.Size([8, 1]))

B) It returns a loss of about 0.6931

C) `RuntimeError`: expected integer (Long) targets

D) It returns a loss of 0.0

Why: `BCEWithLogitsLoss` compares outputs and targets element by element and insists on identical shapes. The fix is `targets.unsqueeze(1)`, as the lesson's `to_tensors` does. (With matching shapes the loss of logit 0 would be 0.6931.)

B is wrong: That is what it WOULD return with targets of shape (8, 1).

C is wrong: Integer class targets belong to `CrossEntropyLoss`, not the binary loss.

D is wrong: A logit of 0 is a probability of 0.5, never a perfect answer.

Q9

Which statement about `model.eval()` and `torch.no_grad()` is true?

A) They are two names for the same switch

B) `torch.no_grad()` switches dropout off; `model.eval()` stops gradient tracking

C) `model.eval()` changes how layers such as dropout behave; `torch.no_grad()` stops gradient tracking; prediction code uses both

D) `model.eval()` is only needed when the model runs on a GPU

Why: They are independent. eval() switches layers that behave differently in training (dropout, batch normalization); no_grad() stops autograd's bookkeeping. The lesson's Mistake 4 shows a dropout model giving three different answers under no_grad() until eval() is called.

A is wrong: One changes layer behavior, the other gradient tracking.

B is wrong: That is exactly backwards.

D is wrong: Devices have nothing to do with it.

Q10

What does this print?

```
import torch
from torch.utils.data import TensorDataset, DataLoader
data = TensorDataset(torch.zeros(1000, 5), torch.zeros(1000, 1))
loader = DataLoader(data, batch_size=64, shuffle=True)
sizes = [len(xb) for xb, yb in loader]
print(len(loader), sizes[-1])
```

`16 40`

Why: 1,000 rows in batches of 64: 15 full batches (960 rows) and one last batch of 40, so 16 batches per epoch. The lesson's 3,600 fitting rows give 57 batches, the last with 16 rows.

Q11

In the lesson's early-stopping function, why does the best state go through `copy.deepcopy(model.state_dict())`?

A) `torch.save` cannot save a state_dict that was not copied

B) `state_dict()` returns references to the live tensors, which later epochs would keep changing

C) `deepcopy` switches the model to evaluation mode

D) It converts the weights to float64 so nothing is lost

Why: Without the copy, the "best" snapshot would silently follow the model as training continues, and loading it at the end would restore the LAST epoch, not the best one.
A is wrong: `torch.save(model.state_dict(), ...)` works directly, as the lesson's saving section shows.
C is wrong: Copying changes no mode.
D is wrong: The copy keeps the dtype (float32).

Q12

You predict `loyalty_tier` (Basic, Silver, Gold) with `nn.CrossEntropyLoss`. What must the targets be?

A) float32 values 0.0 to 2.0, shape (n, 1)

B) the strings Basic, Silver and Gold

C) float32 values 0.0 or 1.0, shape (n, 1)

D) integer class indices 0, 1, 2 (`torch.long`), shape (n,)

> **Why:** CrossEntropyLoss takes k logits per row and one integer class index per row; it applies log-softmax itself. The lesson's Exercise 3 maps the tiers to 0, 1, 2 with dtype torch.long.
> **A** is wrong: Float targets of shape (n, 1) are the binary loss's format.
> **B** is wrong: Tensors hold numbers; map the labels to integers first.
> **C** is wrong: That is the binary (churn) target format.

10.2 · PYTORCH FUNDAMENTALS: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
import torch
print(torch.tensor([1, 2, 3]).dtype, torch.tensor([1.0, 2.0]).dtype)
```

`torch.int64 torch.float32`

> **Why:** PyTorch's default for Python integers is int64 and for Python floats float32 (unlike NumPy's float64).

P4

Predict the output, then run it.

```
from torch import nn
net = nn.Sequential(nn.Linear(3, 5), nn.ReLU(), nn.Linear(5, 2))
print(sum(p.numel() for p in net.parameters()))
```

`32`

> **Why:** First layer 3 × 5 + 5 = 20, second 5 × 2 + 2 = 12; ReLU has no parameters: 32.

LAB REFERENCE SOLUTIONS

- LAB 10.1-A: LABS/MODULE-07/L10_1-NEURAL-NETWORKS/CIRCLES_NETWORK_STARTER.PY → CIRCLES_NETWORK_SOLUTION.PY

The solution prints:

```
(750, 2) (250, 2) inner-ring share 0.50
logistic regression test accuracy: 0.488

probabilities shape: (750,)  first three: [0.5   0.479 0.411]
untrained loss 0.6927   coin flip 0.6931
W2[5, 0]  backprop 0.03266046   numeric 0.03266046   agree: True

8 hidden units: loss 0.6927 -> 0.0156, test accuracy 0.984

width sweep (2,000 epochs, learning rate 0.5, seed 42)
1 hidden units: final loss 0.5965  test accuracy 0.648
2 hidden units: final loss 0.4494  test accuracy 0.752
3 hidden units: final loss 0.0497  test accuracy 0.992
4 hidden units: final loss 0.0266  test accuracy 0.984
8 hidden units: final loss 0.0156  test accuracy 0.984
Each ReLU unit adds one straight edge. Two edges make a wedge; three make a triangle,
the first shape that can close around the inner ring.

learning-rate check (8 hidden units, 500 epochs): loss after 100 / 300 / 500 epochs
learning rate  0.05: 0.644  0.567  0.487
learning rate   0.5: 0.306  0.078  0.045
learning rate    50: 3.914  4.464  4.718

saved ring_three_units.png
```

What each TODO teaches: (1) shapes written beside each line; returning `p` as shape (n,) matters, because an (n, 1) `p` against an (n,) `y` broadcasts into an n × n matrix and the loss is silently wrong; (2) the untrained loss sits just under log(2); (3) a numeric check on an output weight agrees to eight decimals; (4) the four-line loop; (5) the width sweep: one unit is a half-plane (0.648), two a wedge (0.752), three a triangle that can close around the ring (0.992); more units only smooth the corners; (6) the three loss shapes: crawl, fall, bounce. The saved figure shows the triangle.

- LAB 10.1-B: LABS/MODULE-07/L10_1-NEURAL-NETWORKS/CHURN_NEURON_STARTER.PY → CHURN_NEURON_SOLUTION.PY

The solution prints:

```
(4500, 10) (1500, 10)
start: every probability is 0.50, loss 0.6931
epoch    1  loss 0.6931
epoch   10  loss 0.3689
epoch  100  loss 0.3416
epoch 1000  loss 0.3416

test ROC AUC  neuron 0.8235   LogisticRegression 0.8235
largest weight difference: 0.0022
                       neuron  LogisticRegression
tenure_months           -0.12               -0.12
orders_12m              -0.32               -0.32
spend_12m               -0.12               -0.12
avg_order_value         -0.00               -0.00
days_since_last_order    0.88                0.88
returns_12m             -0.02               -0.02
return_rate              0.07                0.07
support_tickets_12m      0.40                0.40
discount_share           0.67                0.67
app_user                -0.34               -0.34

four zero-initialized hidden units, first customer: [0. 0. 0. 0.]
Every hidden unit outputs 0 and has slope 0 there, so no unit can ever start learning.
A single neuron has no hidden units to tie: its gradient (p - y) is not zero, so it lea
```

Reading it: a single sigmoid neuron trained by gradient descent on cross-entropy IS logistic regression, so it converges to the same weights (largest difference 0.0022, the remainder of `C=1e6`'s tiny penalty and the stopping point) and the same AUC. Zero initialization is harmless here because the gradient of one neuron is (p - y) times the inputs, which is not zero; in a hidden layer of zero weights every ReLU outputs 0 with slope 0, so nothing can move (the lesson's Mistake 3).

HANDOUT: LABS/MODULE-07/L10_1-NEURAL-NETWORKS/DEBUG_BACKPROP.PY (DELIBERATELY BROKEN)

1. `ValueError: matmul: Input operand 1 has a mismatch in its core dimension 0 ... (size 1 is different from 16)` → `W2` was created as (1, n_hidden); it must be (n_hidden, 1).
2. No error: the backward pass forgot the ReLU gate (`dz1 = dz2 @ params["W2"].T`). The gradient check prints backprop 0.03319750 against numeric 0.00780703, `agree: False` → multiply by `(z1 > 0)`.
3. No error: the update climbs uphill (`params[name] += ...`). Loss 0.6714 → 13.8155 (with an overflow warning from `np.exp`), test accuracy 0.500 → `-=`.
4. Fixed: `agree: True`, loss 0.6714 → 0.1432, test accuracy 0.972: exactly the lesson's numbers, which is the best possible confirmation.

LAB 10.2-A: LABS/MODULE-07/L10_2-PYTORCH/FRAUD_MLP_STARTER.PY → FRAUD_MLP_SOLUTION.PY

The solution prints (yours may vary slightly on another machine):

```
fit (30000, 19), val (7500, 19), test (12500, 19)
frauds: fit 263, val 66, test 109
torch.float32 (30000, 1)
parameters: 673
stopped after epoch 24; best epoch 19, validation loss 0.0245

model                          test ROC AUC  average precision
neural network (PyTorch)              0.855              0.483
logistic regression                   0.844              0.456
gradient boosting (defaults)          0.825              0.399

frauds among the network's 100 riskiest payments: 50 (a random 100 would hold 0.9)
```

Reading it: 673 parameters = 19 × 32 + 32 + 32 + 1. Early stopping picked epoch 19 by validation loss; with only 66 validation frauds that choice is noisy, so do not over-read it. The network beats logistic regression and default boosting on this split. Default boosting is weak here because 263 training frauds is a small class it overfits; Lessons 2.5 and 5.3 show tuned boosting doing much better on fraud, so the fair conclusion is "the network is a strong candidate; tune the challenger before deciding". Average precision (0.483) is the more informative score for a rare class: half of the network's top 100 are frauds, against 0.9 by chance.

- LAB 10.2-B: LABS/MODULE-07/L10_2-PYTORCH/BATCH_SIZE_STARTER.PY → BATCH_SIZE_SOLUTION.PY

The solution prints:

```
torch.Size([3600, 10]) torch.Size([900, 10])

batch   epoch  steps so far  val loss  val AUC
   16       1           225     0.337    0.835
   16      10          2250     0.303    0.871
   64       1            57     0.398    0.767
   64      10           570     0.310    0.866
  512       1             8     0.604    0.495
  512      10            80     0.356    0.819
 3600       1             1     0.673    0.459
 3600      10            10     0.586    0.518
Ten epochs is the same data, but not the same number of steps: 2,250 steps at batch 16,

full batch, 570 steps (570 epochs): val loss 0.301  val AUC 0.873
Given the same number of steps, the full batch catches up: steps drive progress; small
simply give you many more of them per pass over the data.

checkpoint keys: ['scaler_mean', 'scaler_scale', 'settings', 'state_dict']
rebuilt model, val AUC 0.866 (original 0.866)
```

Reading it: an epoch is the same data for every batch size, but not the same number of updates. Batch 16 makes 2,250 steps in ten epochs and reaches val AUC 0.871; the full batch makes ten steps and barely moves (0.518). Given

570 steps, the full batch catches up (0.873), so the number of steps drives progress; small batches simply give many more steps per pass, plus some helpful noise. The checkpoint carries `settings` and the scaler's numbers as plain lists, which `weights_only=True` accepts, so the model can be rebuilt from the file alone.

HANDOUT: LABS/MODULE-07/L10_2-PYTORCH/DEBUG_TRAINING_LOOP.PY (DELIBERATELY BROKEN)

1. `RuntimeError: mat1 and mat2 must have the same dtype, but got Double and Float` → `torch.from_numpy(...)` kept float64; use `torch.tensor(..., dtype=torch.float32)`.
2. `ValueError: Target size (torch.Size([64])) must be the same as input size (torch.Size([64, 1]))` → `y_fit` needs `.unsqueeze(1)`.
3. No error: `nn.Sigmoid()` at the end of the model with `BCEWithLogitsLoss` (a sigmoid applied twice). With this and the next two bugs the script prints 0.503 / 0.502 / 0.499.
4. No error: no `optimizer.zero_grad()`, so every step uses the sum of all previous gradients. With the sigmoid removed: 0.766 / 0.764 / 0.780.
5. No error: `val_auc` never calls `model.eval()`, so dropout stays on and the three lines differ (0.824 / 0.825 / 0.829 once `zero_grad()` is back). Fixed: 0.839 three times.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 10.1 · NEURAL NETWORKS FROM SCRATCH

1. **B** (32, 16): the inner dimensions (10 and 10) match and disappear; one row per sample, one column per unit.
2. **A** Only a straight line, like logistic regression: matrix products in a row collapse into one.
3. **C** `tensor(6.)`: `.grad` accumulates; three backward calls of slope 2 give 6.
4. **B** The learning rate is too large: a loss that jumps and ends in nan overshoots the valley.
5. **A** Backpropagation gets all gradients in one backward sweep; the numeric check needs two forward passes per parameter.

LESSON 10.2 · PYTORCH FUNDAMENTALS

1. **B** `torch.float64`: `from_numpy` keeps NumPy's dtype (and shares its memory).
2. **C** `optimizer.zero_grad()`, forward and loss, `loss.backward()`, `optimizer.step()`.
3. **A** Apply `torch.sigmoid` to the model's outputs (in eval mode, under `no_grad()`).
4. **B** The model is still in training mode; call `model.eval()`: `no_grad()` does not switch dropout off.
5. **C** The test AUC, computed once after training: the validation set chose the epoch.

EXIT TICKET

1. **The loop:** `optimizer.zero_grad()` (reset), `loss = loss_fn(model(xb), yb)` (forward and loss), `loss.backward()` (gradients), `optimizer.step()` (update). Accept any wording that keeps the order and names the four calls.
2. **Probabilities:** `torch.sigmoid(model(X))` with `model.eval()` and `torch.no_grad()`. `BCEWithLogitsLoss` applies the sigmoid internally, in a numerically safe way; a second sigmoid in the model squashes outputs into 0.5 to 0.73 as seen by the loss, which can then never fall below about 0.31 for a sure churner (demo Step 12).
3. **0.846 vs 0.850:** gradient boosting. The gap is within the wobble of one test set (a tie), and boosting needed no scaling, architecture, learning rate, dropout or early stopping. A full answer adds that networks earn their keep on images, sequences and text.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 A (160).** Forgets biases. Count them out loud: one per unit.
- **Q3 A (learning rate too large).** Has the loss shapes mixed up. A wild learning rate moves the loss a lot; a frozen 0.6931 means nothing can move.
- **Q5 B ("it outputs probabilities").** Confuses the hidden and output roles. Draw the network: ReLU inside, sigmoid at the end.
- **Q8 B ("0.6931").** Read the code as it would run with matching shapes. Ask them to print both shapes first, always.
- **Q9 A or B.** The most common confusion of the session. Replay demo Step 12: three different numbers in train mode under `no_grad()`.
- **Q12 A or C.** Carried the binary format over to three classes. The lesson's Exercise 3 is the fix: integers, `torch.long`, shape (n,).
- **Exit ticket 3: "the network, it is more modern" or "the network, 0.846 is good".** The habit this course builds is comparing against a strong baseline and weighing the cost of every choice. Point back to the project's Step 5 text.

INSTRUCTOR KIT · ANSWER KEY

Session 8: CNNs and Sequence Models

Answers and reasoning for every Session 8 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs and debug handouts, the lesson pages' own quizzes and the exit ticket.

Session 8 | 12 self-check items | Confidential

WARM-UP

Retrieval on Lessons 10.1 and 10.2. Accept answers in the learner's own words.

W1

What does this print? (Lesson 10.2)

```
import numpy as np
import torch
print(torch.tensor(np.zeros(3)).dtype, torch.tensor(np.zeros(3), dtype=torch.float3
```

`torch.float64 torch.float32`: torch.tensor keeps NumPy's float64; layers want float32, so convert as you create.

W2

Put the four lines of the batch loop in order: loss.backward(), optimizer.step(), optimizer.zero_grad(), loss = loss_fn(model(xb), yb). (10.2)

optimizer.zero_grad(), loss = loss_fn(model(xb), yb), loss.backward(), optimizer.step().

W3

Your binary model ends with nn.Linear(32, 1) and trains with BCEWithLogitsLoss. Where does the sigmoid go? (10.2)

Inside the loss during training; apply torch.sigmoid yourself only to report probabilities. Today's ten-class models follow the same rule with softmax and CrossEntropyLoss.

W4

A model with dropout gives a different score for the same customer on every call, inside with torch.no_grad(). Why? (10.2)

It is still in training mode: no_grad() stops gradient tracking but does not switch dropout off. Call model.eval().

W5

On the churn table the PyTorch network scored test AUC 0.846 and gradient boosting 0.850. What was the verdict, and what kind of data should change it? (10.1, 10.2)

Recommend boosting: a tie within the wobble, with far fewer choices. Images, sequences and text are where networks earn their keep: today's session.

10.3 · CONVOLUTIONAL NEURAL NETWORKS: SELF-CHECK

Q1

What does this print?

```
import torch
from torch import nn
conv = nn.Conv2d(3, 16, kernel_size=3, stride=2, padding=1)
print(tuple(conv(torch.zeros(8, 3, 32, 32)).shape))
```

`(8, 16, 16, 16)`

> **Why:** Per side: (32 + 2 × 1 - 3) / 2 + 1 = 16.5, rounded down to 16. The batch stays 8 and the 16 filters give 16 channels.

Q2

How many parameters does `nn.Conv2d(3, 8, kernel_size=3)` have?

A) 224

B) 216

C) 72

D) 80

> **Why:** Each of the 8 filters spans all 3 input channels: 8 × (3 × 3 × 3) = 216 weights, plus 8 biases = 224.
> **B** is wrong: That forgets the 8 biases.
> **C** is wrong: That counts one input channel only (8 × 9).
> **D** is wrong: One input channel plus biases: every filter spans ALL incoming channels.

Q3

Why does the lesson use `padding=1` with every 3 × 3 kernel?

A) It halves the height and width

B) It adds a border of learnable weights

C) It keeps the height and width unchanged: (n + 2 - 3) / 1 + 1 = n

D) Grayscale images require it

> **Why:** A one-pixel border of zeros lets the 3 × 3 window sit centered on every pixel, so 28 stays 28. Without it, 28 becomes 26. Halving is the pooling layer's job.
> **A** is wrong: That is `MaxPool2d(2)` (or stride 2).
> **B** is wrong: The border is zeros, not weights.
> **D** is wrong: Padding works the same for any number of channels.

Q4

What does this print?

```
import torch
import torch.nn.functional as F
x = torch.arange(16.0).reshape(1, 1, 4, 4)
print(F.max_pool2d(x, 2).squeeze())
```

`tensor([[ 5., 7.], [13., 15.]])`

> **Why:** The 4 × 4 grid holds 0 to 15 row by row. Max pooling keeps the largest value of each 2 × 2 block: 5 (0, 1, 4, 5), 7, 13 and 15. It has no weights.

Q5

In the shift lab, the lesson's CNN loses about 12 points of test accuracy when every image moves 2 pixels to the right. Convolutions share weights across positions, so why does it lose anything?

A) The model has no max pooling

B) The shifted images were scaled differently

C) Each convolution uses different weights at different positions

D) The `Flatten` + `Linear` head reads WHERE each feature map lit up, so positions still matter

> **Why:** The convolutions find a pattern anywhere, and pooling forgives a pixel or so. But the flattened maps go into a dense layer with one weight per position, which learned where features usually appear in centered images. Training on shifted copies (augmentation) is the usual remedy.
> **A** is wrong: It has two max-pooling layers; they buy only a pixel or two of tolerance.
> **B** is wrong: Shifting moves pixels; their values are unchanged.
> **C** is wrong: That is exactly what weight sharing rules out.

Q6

This model fails. What should the first argument of its nn.Linear be?

```
import torch
from torch import nn
model = nn.Sequential(
    nn.Conv2d(1, 64, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
    nn.MaxPool2d(2), nn.Flatten(), nn.Linear(64 * 14 * 14, 10),
)
model(torch.zeros(64, 1, 28, 28))
```

A) 12544 (64 × 14 × 14)

B) 3136 (64 × 7 × 7)

C) 1568 (32 × 7 × 7)

D) 64 (one number per filter)

> **Why:** The message reads `RuntimeError: mat1 and mat2 shapes cannot be multiplied (64x3136 and 125...`: the rows really have 3,136 numbers. Two poolings take 28 to 14 to 7, with 64 channels: 64 × 7 × 7. Trace a dummy batch instead of computing on paper.
> **A** is wrong: That counts only one pooling.
> **C** is wrong: That is the lesson's SmallCNN with 32 channels.
> **D** is wrong: 64 would need global average pooling first.

10.3 · CONVOLUTIONAL NEURAL NETWORKS: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
import torch
from torch import nn
print(nn.MaxPool2d(2)(torch.zeros(5, 8, 28, 28)).shape)
```

`torch.Size([5, 8, 14, 14])`

> **Why:** Max pooling with a 2 × 2 window halves height and width and keeps batch and channels: (5, 8, 14, 14).

P2

Predict the output, then run it.

```
from torch import nn
print(sum(p.numel() for p in nn.Conv2d(1, 4, kernel_size=5).parameters()))
```

`104`

Why: Four filters of 1 × 5 × 5 = 25 weights, plus 4 biases: 104.

10.4 · SEQUENCE MODELS: SELF-CHECK

Q7

What does this print?

```
import torch
from torch import nn
out, h = nn.GRU(5, 8, batch_first=True)(torch.zeros(4, 10, 5))
print(tuple(out.shape), tuple(h.shape))
```

`(4, 10, 8) (1, 4, 8)`

Why: With batch_first=True, out holds the hidden state of size 8 after every one of the 10 steps of each of the 4 sequences; h holds only the last one, with a leading dimension for the number of layers.

Q8

A store has 100 days of sales. How many training examples do windows of 7 days (each predicting the next day) give?

A) 100

B) 94

C) 93

D) 7

Why: The first target that has 7 days before it is day 8 (index 7), so targets run from index 7 to 99: 100 - 7 = 93 windows, exactly as the lesson's make_windows gives 10 - 4 = 6.
A is wrong: The first seven days have no full window before them.
B is wrong: Off by one: count the targets, from index 7 to 99.
D is wrong: That is the window length.

Q9

For Pinecrest's daily store sales, which forecast is the seasonal naive one?

A) Tomorrow = the same weekday last week

B) Tomorrow = today

C) Tomorrow = the average of the training years

D) Tomorrow = the same date last year

Why: The season that dominates daily sales is the week (Saturday sells almost twice what Tuesday does), so the seasonal naive forecast copies the value 7 days earlier: MAE $1,072 in 2025, against $1,350 for yesterday's value.
B is wrong: That is the plain naive forecast.
C is wrong: That ignores the weekly cycle and the trends.
D is wrong: A yearly season exists too, but last year's date falls on a different weekday.

Q10

What does this print?

```
import numpy as np
keys = np.array([[1, 0], [0, 1], [1, 0]])
values = np.array([10.0, 50.0, 30.0])
query = np.array([1, 0])
w = np.exp(5 * (keys @ query))
w = w / w.sum()
print(w.round(3), round(w @ values, 1))
```

`[0.498 0.003 0.498] 20.1`

> **Why:** The query matches keys 1 and 3 (score 5) and not key 2 (score 0), so the softmax gives them almost all the weight, equally: the output is about the average of 10 and 30. That is the lesson's "four Saturdays" lookup in miniature.

Q11

Your LSTM predicts standardized log sales. What must happen before you compute the MAE you report?

A) Round the predictions to whole numbers

B) Nothing: targets and predictions are on the same scale, so the MAE is fair

C) Multiply the predictions by sd

D) Undo the standardization, then the log (expm1), and compare in dollars

> **Why:** The error must be in units a manager understands and comparable with the naive forecasts. The lesson's Mistake 5 shows the same model at 0.222 in scaled log units and $702 in dollars.
>
> **A** is wrong: Rounding changes nothing about the units.
>
> **B** is wrong: A scaled error flatters the model and cannot be compared with anything.
>
> **C** is wrong: Half the undo: add mu, then expm1.

Q12

You create `nn.LSTM(8, 32)` and forget `batch_first=True`, while your windows are (batch, 28, 8). What happens?

A) A RuntimeError about the input size

B) It runs and trains on nonsense: PyTorch reads (batch, time) as (time, batch)

C) PyTorch notices and transposes the input

D) The output becomes (28, 64, 32), so the linear head fails

> **Why:** The default layout is (time, batch, features). Your 64 windows become 64 time steps of 28 "sequences"; out[:, -1] still has a usable shape, so nothing complains (the lesson's Mistake 3).
> **A** is wrong: The feature size (8) still matches, so nothing raises.
> **C** is wrong: PyTorch never guesses the layout.
> **D** is wrong: The output keeps the input's (64, 28, ...) layout, which is why the bug is silent.

10.4 · SEQUENCE MODELS: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
import numpy as np
series = np.arange(6)
X = np.stack([series[i:i + 3] for i in range(len(series) - 3)])
print(X.shape, series[3:])
```

`(3, 3) [3 4 5]`

> **Why:** 6 values and windows of 3 give 6 - 3 = 3 windows of 3; the targets are the values after each window: 3, 4, 5.

P4

Predict the output, then run it.

```
from torch import nn
print(sum(p.numel() for p in nn.LSTM(input_size=2, hidden_size=4).parameters()))
```

`128`

> **Why:** Four gates, each with input weights (4 × 2) and hidden weights (4 × 4) and two bias vectors of 4: 4 × (8 + 16 + 8) = 128.

LAB REFERENCE SOLUTIONS

- LAB 10.3-A: LABS/MODULE-08/L10_3-CNN/EDGE_FEATURES_STARTER.PY → EDGE_FEATURES_SOLUTION.PY

The solution prints (CNN column yours may vary slightly):

```
(60000, 1, 28, 28) (10000, 1, 28, 28)
edge maps: (4, 2, 28, 28)  filter shape: (2, 1, 3, 3)
hand-made features per image: 48

training images   hand-made filters + logistic regression   SmallCNN, 3 epochs
          1,000                                      0.795                0.676
          5,000                                      0.819                0.782
         20,000                                      0.827                0.863

With little data, two filters chosen by a person win: they encode knowledge the CNN mus
learn from examples. With 20,000 images the CNN's sixteen learned filters overtake ther
and more data and epochs widen the gap (the lesson's Scaling Up: about 90 to 92% on al
```

Reading it: 48 = 2 filters × 16 regions + 16 brightness numbers. The hand-made pipeline is strong with little data because the person supplied the knowledge (edges matter, roughly where they are matters); the CNN must learn sixteen filters and a classifier from examples, and at 1,000 images in 3 epochs it cannot. By 20,000 it passes the fixed features. Common sticking point: forgetting `.abs()`, so positive and negative edge responses cancel when averaged (accuracy drops noticeably).

- LAB 10.3-B: LABS/MODULE-08/L10_3-CNN/SHIFT_TEST_STARTER.PY → SHIFT_TEST_SOLUTION.PY

The solution prints:

```
loaded: 101770 and 105866 parameters
unshifted test accuracy  dense 0.840   CNN 0.864
a boot moved 3 pixels right: first 3 columns are now empty: True

shift (pixels)   right: dense   CNN    down: dense   CNN
             0         0.840   0.864         0.840   0.864
             1         0.756   0.838         0.778   0.821
             2         0.552   0.743         0.708   0.770
             3         0.355   0.471         0.661   0.726
             4         0.252   0.301         0.553   0.592

CNN accuracy lost with a 2-pixel shift right, worst three classes:
  Dress        -0.334
  T-shirt/top  -0.229
  Coat         -0.217

The CNN holds up better at every shift, but it is NOT shift-proof: its convolutions fi
pattern anywhere, yet the Flatten + Linear head still reads WHERE each feature map lit
Pooling buys tolerance of a pixel or two; training on shifted copies (augmentation wit
crops, the lesson's Exercise 4) is the standard way to teach a network more.
```

Reading it: shifting down hurts less than shifting right for both models (the clothes are centered and wider than tall, so a horizontal shift moves more of the outline). The CNN degrades more gently at every shift, but by 3 pixels to the right it has lost almost 40 points. The convolutions are shift-equivariant; the dense head after `Flatten` is not, and two poolings only forgive a pixel or two. `torch.roll` would wrap pixels around from the opposite edge, which is why the lab fills with zeros.

HANDOUT: LABS/MODULE-08/L10_3-CNN/DEBUG_CNN.PY (DELIBERATELY BROKEN)

1. `RuntimeError: Given groups=1, weight of size [16, 1, 3, 3], expected input[1, 64, 28, 28] to have 1 channels, but got 64 channels instead` → `X_train` needs `.unsqueeze(1)`.
2. `RuntimeError: mat1 and mat2 shapes cannot be multiplied (64x1568 and 25088x64)` → `nn.Linear(32 * 7 * 7, 64)`.
3. No error: `nn.Softmax(dim=1)` as the last layer before `CrossEntropyLoss`. With this and the next bug: 0.655 / 0.653.
4. No error: `test_accuracy` never calls `model.eval()`, so dropout stays on: 0.688 with only this bug left. Fixed: 0.747 on both lines.

LAB 10.4-A: LABS/MODULE-08/L10_4-SEQUENCES/TRANSACTIONS_GRU_STARTER.PY → TRANSACTIONS_GRU_SOLUTION.PY

The solution prints (GRU lines yours may vary slightly):

```
mean transactions per day in 2025: {'Asheville': 61.0, 'Bend': 77.5, 'Boise': 115.5, 'P
mu 4.287  sd 0.482  z (1096, 4)  calendar (1096, 9)
windows (4272, 28, 8)  training 2812  test 1460  parameters 4,065
windows (4272, 28, 10)  training 2812  test 1460  parameters 4,257

next-day transactions, test year 2025 (MAE in transactions per store-day)
naive (yesterday)                 19.09
seasonal naive (last week)        15.50
GRU, weekday only                  9.15
GRU, weekday + promo + closed      7.56
Weekday flags alone cut the seasonal naive error by about 40%; adding the promotion and
flags, known in advance, brings it to half.
```

Reading it: same windows as the lesson (2,812 training windows ending in 2023 to 2024, 1,460 test windows in 2025), a different target and a GRU. Seasonal naive beats naive for the same reason as with sales (the weekly cycle). The weekday-only GRU cuts its error by about 40%; the promotion and closed flags, known in advance, bring it to half. For staffing: about 7.6 customers off per store-day against 15.5 for "same day last week".

- LAB 10.4-B: LABS/MODULE-08/L10_4-SEQUENCES/ATTENTION_LOOKUP_STARTER.PY → ATTENTION_LOOKUP_SOLUTION.PY

The solution prints:

```
2025: 365 days, 20 of them special (promotion or closed)

seasonal naive (same day last week)      all $ 1,072   ordinary $   949   special $ 3,1
weekday keys, 28 days (the lesson)       all $   979   ordinary $   856   special $ 3,0
weekday keys, 364 days                   all $ 1,151   ordinary $ 1,030   special $ 3,2
calendar keys, 28 days                   all $   950   ordinary $   856   special $ 2,5
calendar keys, 364 days                  all $ 1,039   ordinary $ 1,030   special $ 1,1
routed (28 ordinary / 364 special)       all $   875   ordinary $   856   special $ 1,1

Boise, 2025-11-28 (Black Friday): forecast $12,014, actual $12,702
  weight 0.226 on 2024-11-29 Friday    (Black Friday) $12,644
  weight 0.226 on 2025-05-23 Friday    (Memorial Day sale) $13,178
  weight 0.226 on 2025-08-29 Friday    (Labor Day sale) $13,235
  weight 0.226 on 2024-12-20 Friday    (Holiday rush) $10,077
Its query (Friday + promotion) matched the past year's sale Fridays, not ordinary Frida
```

Reading it: with weekday keys, a longer memory is worse everywhere (it averages Saturdays from a year of trend and season). Calendar keys over 28 days help special days only a little, because the last 28 days rarely contain a matching sale day. Calendar keys over a year find past sale days and cut the special-day error from $3,097 to $1,198, at the cost of ordinary days. Routing each query to the right memory takes the best of both ($875 overall). A trained attention layer learns its own keys, queries and, in effect, this routing.

- HANDOUT: LABS/MODULE-08/L10_4-SEQUENCES/DEBUG_SEQUENCE.PY (DELIBERATELY BROKEN)

1. `ValueError: RNN input dtype (torch.float64) does not match weight dtype (torch.float32)...` → `torch.tensor(np.array(X), dtype=torch.float32)`.
2. No error: `nn.LSTM(8, 32)` without `batch_first=True` reads (batch, time) as (time, batch).
3. No error: a random choice of 365 windows as the test set; overlapping windows leak. With the next bug fixed first, the dollar MAE is a flattering 863.2 (825.8 once `batch_first` is fixed).
4. No error: the MAE is computed on scaled log values (prints 0.2). Undo the scaling and the log before the metric.
5. Fixed: 365 test windows, LSTM MAE 1,039.2 against seasonal naive 1,355.6 for Boise in 2025.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 10.3 · CONVOLUTIONAL NEURAL NETWORKS

1. **B** `torch.Size([4, 8, 24, 24])`: 28 - 5 + 1 = 24 without padding; 8 filters give 8 channels.
2. **C** 4,640: 32 × (16 × 3 × 3) = 4,608 weights plus 32 biases.
3. **A** Small filters reused at every position: patterns learned once and found anywhere.
4. **A** So dropout stops zeroing activations and batch norm uses its stored training statistics.
5. **C** A plain `nn.Linear` with 10 outputs (logits): `CrossEntropyLoss` applies log-softmax itself.

LESSON 10.4 · SEQUENCE MODELS

1. **B** (10, 7, 16): one hidden state of size 16 for each of the 7 steps of the 10 sequences.
2. **A** It is a free forecast that already captures the weekly cycle; a model must beat it.
3. **C** Gradients vanish over many steps, so plain RNNs struggle to learn from the distant past.
4. **B** Their keys matched the query (tomorrow's weekday) and the softmax turned the higher scores into most of the weight.
5. **C** Overlapping windows leaked the test days into training: too good to be true.

EXIT TICKET

1. **Shapes:** (64, 16, 28, 28) after the convolution (16 filters, padding keeps 28), then (64, 16, 14, 14) after pooling.
2. **Time split:** a forecast is made on the day before, knowing only the past. A random split of overlapping windows puts the days around each test window, including its target, into training, so the model is graded on data it has effectively seen. Accept "leakage" with any correct reason.

3. **$515 vs $616 / $569:** both far better than seasonal naive ($1,072) and in the same league; the LSTM leads here, but boosting was simpler, fast, untuned, and easier to explain; the calendar information mattered more than the architecture. Good answers recommend starting with the simpler model or keeping both under test; weak answers just say "the LSTM, it is lower".

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q2 B (216).** Forgets biases again: a Session 7 echo. **Q2 C or D:** forgets that every filter spans all input channels.
- **Q5 C.** Has not understood weight sharing; replay the stencil sliding over the image.
- **Q6 A (12544).** Counted one pooling. The habit to build is tracing, not arithmetic.
- **Q8 B (94).** Off by one: count targets, not windows' start positions.
- **Q11 B.** Believes a scaled error is fair because both sides are scaled; ask them to compare it with $1,072.
- **Q12 A or D.** Expects PyTorch to complain. The lesson's Mistake 3 is the counterexample: always print shapes when building a model.

INSTRUCTOR KIT · ANSWER KEY

Session 9: Transfer Learning and Text Classification

Answers and reasoning for every Session 9 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs and handouts, the lesson pages' own quizzes and the exit ticket.

Session 9 | 12 self-check items | Confidential

WARM-UP

Retrieval from Lessons 10.3 and 10.4. The code answers are real output.

W1

What shape does this print?

```
import torch
from torch import nn
print(nn.Conv2d(1, 8, kernel_size=3)(torch.zeros(4, 1, 28, 28)).shape)
```

```
torch.Size([4, 8, 26, 26])
```

> **Why:** A 3 × 3 kernel with no padding loses one pixel on each side: (28 + 0 - 3) / 1 + 1 = 26. Batch 4, 8 output channels.

W2

How many parameters? (This is ResNet-18's first layer; Lesson 10.3's Before the Next Lesson asked the same.)

```
from torch import nn
print(sum(p.numel() for p in nn.Conv2d(3, 64, kernel_size=7, bias=False).parameters
```

```
9408
```

> **Why:** 64 filters × 3 channels × 7 × 7 = 9,408, and no bias. Today we borrow those 9,408 numbers, and 11 million more, ready-trained.

W3

What does this print?

```
import torch
from torch import nn
lstm = nn.LSTM(input_size=2, hidden_size=8, batch_first=True)
out, (h, c) = lstm(torch.zeros(5, 14, 2))
print(tuple(out.shape), tuple(h.shape))
```

`(5, 14, 8) (1, 5, 8)`

> **Why:** batch_first=True: out is (batch, time steps, hidden) = (5, 14, 8); h is (layers, batch, hidden) = (1, 5, 8), the last step only.

W4

Why must `model.eval()` come before measuring test accuracy of a model with dropout and batch norm?

So dropout stops zeroing activations and batch norm uses its stored training statistics (not the test batch's).

> **Why:** This comes straight back in 10.5: features extracted in training mode are silently wrong.

W5

A colleague splits overlapping sales windows with `train_test_split(..., random_state=42)` and beats the seasonal naive forecast by a mile. Your first suspicion?

Leakage: neighboring windows share days, so test days were in training. Split by time.

> **Why:** Same habit as today's text pipeline: anything learned from data (windows, vocabulary, IDF) must come from the training part only.

10.5 · TRANSFER LEARNING: SELF-CHECK

Q1

With a frozen backbone, the lesson computes the 512 features of every tile ONCE and then trains the head on that table. Why is that allowed?

A) Frozen weights never change, so each tile's features are the same in every epoch

B) Computing features once is required by PyTorch for pretrained models

C) The head needs the features as NumPy arrays, not tensors

D) Features computed once are more accurate than features computed every epoch

> **Why:** A frozen backbone in eval() mode is a fixed function: the same tile always gives the same 512 numbers. Recomputing them every epoch would cost the full forward pass for nothing. Once you fine-tune (unfreeze layer4), the features change every step and caching no longer works.
> **B** is wrong: PyTorch has no such rule; the project trains through the frozen backbone every epoch and that works too, just slowly.
> **C** is wrong: The demo's head is an nn.Linear trained on tensors; NumPy is only needed for scikit-learn.
> **D** is wrong: Same numbers either way; the gain is speed, not accuracy.

Q2

What does this print?

```
import torch
from torch import nn
from torchvision import models
m = models.mobilenet_v3_small(weights=None)
for p in m.parameters():
    p.requires_grad = False
m.classifier = nn.Linear(576, 3)
print(sum(p.numel() for p in m.parameters() if p.requires_grad))
```

`1731`

> **Why:** Everything existing is frozen first; the new head is created afterwards and new layers are trainable by default: 576 × 3 weights + 3 biases = 1,731.

Q3

The label curve: frozen features reach 0.755 with 100 labeled tiles and 0.902 with 2,000; the from-scratch CNN reached 0.719 with 2,000. Pinecrest offers to pay for 5,000 more labels for the frozen-feature model. Your advice?

A) Yes: accuracy always keeps growing with more labels

B) Yes: the from-scratch CNN would then beat transfer learning

C) Probably not: the curve has flattened (1,000 to 2,000 added under a point); fine-tuning is the better next step

D) No: transfer learning cannot use more than 2,000 labels

> **Why:** Read the curve like a budget: above about 500 tiles each extra label buys very little for a frozen backbone. Letting the features adapt (fine-tuning layer4) moved validation accuracy from 0.876 to 0.924 in the project.
> **A** is wrong: The curve shows diminishing returns: +0.007 from 1,000 to 2,000.
> **B** is wrong: Possible in principle with far more data, but nothing on the page suggests it at 7,000 tiles.
> **D** is wrong: Nothing limits the number of labels; the gain per label is the issue.

Q4

A learner extracts features with a freshly created ResNet-18 (never called `.eval()`) under `torch.no_grad()`. What goes wrong?

A) Nothing: no_grad already switches the model to evaluation mode

B) Batch normalization uses each batch's statistics, so a tile's features depend on its batch, and the stored ImageNet statistics are overwritten

C) PyTorch raises an error because the model is in training mode

D) The features are computed with dropout, so they are all zero

> **Why:** no_grad stops gradient tracking; eval() changes how layers behave. In training mode batch norm normalizes with the current batch and updates its running averages even without gradients. The demo showed it: same tile, different features in a batch of 8 and a batch of 2.
> **A** is wrong: no_grad and eval() are independent switches.
> **C** is wrong: No error at all: that is why this mistake is dangerous.
> **D** is wrong: ResNet-18 has no dropout; and dropout zeroes some activations, not all.

Q5

What happens when this runs?

```
import torch
from torchvision import models
net = models.resnet18(weights=None).eval()
net(torch.zeros(1, 1, 64, 64))
```

A) It returns 1,000 ImageNet scores for the image

B) It returns 512 features because the model is in eval() mode

C) `ValueError`: the image must be resized to 224 pixels first

D) `RuntimeError`: the first convolution expects 3 channels and the image has 1

> **Why:** The first layer has 64 filters spanning 3 color channels. Fix: repeat the gray channel (`x.repeat(1, 3, 1, 1)` or `transforms.Grayscale(num_output_channels=3)`). The message: `RuntimeError: Given groups=1, weight of size [64, 3, 7, 7], expected i...`
>
> **A** is wrong: It never gets past the first convolution.
>
> **B** is wrong: fc was not replaced, and the channel error comes first anyway.
>
> **C** is wrong: ResNet accepts any reasonable size thanks to its average pooling; 64 pixels runs once the channels are right.

Q6

What does this print?

```
from torchvision import models
print(models.resnet18(weights=None).fc)
```

`Linear(in_features=512, out_features=1000, bias=True)`

> **Why:** fc is ResNet-18's head: 512 features in, 1,000 ImageNet class scores out. Transfer learning replaces it, with nn.Identity() for features or nn.Linear(512, 10) for EuroSAT.

10.5 · TRANSFER LEARNING: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
from torch import nn
head = nn.Linear(576, 10)
print(sum(p.numel() for p in head.parameters()))
```

`5770`

> **Why:** 576 × 10 weights plus 10 biases = 5,770 trainable numbers: that is all a frozen-feature head learns.

P2

Predict the output, then run it.

```
import torch
from torchvision import transforms
norm = transforms.Normalize(mean=[0.5, 0.5, 0.5], std=[0.25, 0.25, 0.25])
x = torch.full((3, 2, 2), 0.75)
print(norm(x)[:, 0, 0])
```

`tensor([1., 1., 1.])`

> **Why:** (0.75 - 0.5) / 0.25 = 1 in each channel. ImageNet normalization does exactly this with its own three means and standard deviations.

11.1 · TEXT CLASSIFICATION: SELF-CHECK

Q7

What does this print?

```
from sklearn.feature_extraction.text import CountVectorizer
print(CountVectorizer().build_analyzer()("I am a 2-person tent fan!"))
```

`['am', 'person', 'tent', 'fan']`

> **Why:** The default token pattern keeps runs of two or more word characters, lowercased: one-letter words (I, a) and the digit 2 disappear, and 2-person becomes person. Punctuation is dropped.

Q8

Why does the lesson put `TfidfVectorizer` inside the pipeline instead of vectorizing all 3,095 reviews first?

A) Pipelines make TF-IDF faster on sparse matrices

B) A vectorizer outside a pipeline cannot produce bigrams

C) The vocabulary and IDF values are learned from data, so they must come from the training text only (also inside every CV fold)

D) LogisticRegression refuses a sparse matrix that was not produced inside a pipeline

> **Why:** A vectorizer is a fitted transformer like a scaler. Fitted on all reviews, test-only words join the vocabulary and every IDF shifts (334 words instead of 333; idf of leaked 4.17 instead of 4.145): the test score then describes a model that saw the future.
> **A** is wrong: Speed is the same.
> **B** is wrong: ngram_range works anywhere.
> **D** is wrong: Logistic regression accepts any sparse matrix.

Q9

What does this print?

```
from sklearn.feature_extraction.text import CountVectorizer
print(CountVectorizer(stop_words="english").build_analyzer()("Would not buy it agai
```

A) `['buy']`

B) `['would', 'not', 'buy', 'again']`

C) `['not', 'buy']`

D) `['would', 'buy', 'again']`

> **Why:** scikit-learn's English stop-word list contains would, not, it and again; it is gone because it is a stop word too. A review that said Would not buy it again reaches the model as the single word buy: the stop-word trap in one line.
> **B** is wrong: That is the default analyzer without a stop-word list (it would also keep it).
> **C** is wrong: not IS on the list: that is the whole problem.
> **D** is wrong: would and again are on the list as well.

Q10

The training matrix is 2,321 × 333 with 5.3% of cells filled. Why does scikit-learn return it as a sparse matrix?

A) Sparse matrices make logistic regression more accurate

B) So that it can be turned into a pandas DataFrame

C) Because TF-IDF values are always between 0 and 1

D) Almost every cell is zero, so storing only the non-zero values saves memory, which matters with real vocabularies

> **Why:** Each review uses a handful of the vocabulary's words. With 50,000 documents and 100,000 words, a dense table would need about 40 GB; the sparse one fits in a few megabytes. Avoid .toarray() on anything bigger than a toy.
> **A** is wrong: Accuracy is the same; only memory and speed differ.
> **B** is wrong: Sparse output cannot become pandas output (SHELL convention: leave text vectorizers sparse).
> **C** is wrong: True but irrelevant: dense matrices can hold values between 0 and 1 too.

Q11

What does this print? (scikit-learn's default IDF is ln((1 + n) / (1 + df)) + 1.)

```
from sklearn.feature_extraction.text import TfidfVectorizer
v = TfidfVectorizer().fit(["warm tent", "warm bag", "light tent", "light bag"])
print(round(v.idf_[v.vocabulary_["warm"]], 3))
```

`1.511`

> **Why:** warm appears in 2 of the 4 documents: ln(5 / 3) + 1 = 1.511. A word in every document would get exactly 1, the smallest possible value.

Q12

In the review-triage project, lowering the flagging threshold for P(negative) from 0.5 to 0.3 does what?

A) Raises both recall and precision for negative reviews

B) Catches more unhappy customers (recall up) at the cost of more false alarms (precision down)

C) Changes the model's coefficients

D) Leaks the test set into training

> **Why:** On the training folds recall rose from 0.570 to 0.851 while precision fell from 0.828 to 0.709. The threshold is a business choice made on cross-validated probabilities, then graded once on the test set (0.858 / 0.698).
> **A** is wrong: They trade against each other.
> **C** is wrong: The threshold is applied after the model; nothing is refitted.
> **D** is wrong: Only if you choose it by looking at the test set; the project uses cross_val_predict on training reviews.

11.1 · TEXT CLASSIFICATION: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
from sklearn.feature_extraction.text import CountVectorizer
print(CountVectorizer(ngram_range=(1, 2)).fit(["not warm at all"]).get_feature_name
```

```
['all' 'at' 'at all' 'not' 'not warm' 'warm' 'warm at']
```

> **Why:** Unigrams and bigrams, sorted alphabetically: four words and three pairs. Bigrams such as `not warm` are what keep negation.

P4

Predict the output, then run it.

```
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd
v = TfidfVectorizer().fit(["tent leaked", "tent warm", "warm bag", "warm boots"])
print(pd.Series(v.idf_, index=v.get_feature_names_out()).round(3).to_dict())
```

```
{'bag': 1.916, 'boots': 1.916, 'leaked': 1.916, 'tent': 1.511, 'warm': 1.223}
```

> **Why:** n = 4. leaked, bag and boots are in 1 document: ln(5/2) + 1 = 1.916; tent in 2: ln(5/3) + 1 = 1.511; warm in 3: ln(5/4) + 1 = 1.223. Rarer means heavier.

LAB REFERENCE SOLUTIONS

FIXTURE: LABS/MODULE-09/L10_5-TRANSFER-LEARNING/PREPARE_EUROSAT_FEATURES.PY

Not an exercise and not run by the checker. It computes frozen ResNet-18 features (128 px, ImageNet-normalized, `eval()`, `inference_mode`) for positions 3,500 to 6,499 of the seed-42 permutation (2,000 train, 1,000 test: NOT the lesson's tiles) and saves them as float16 to `labs/module-09/_fixtures/eurosat_resnet18_feats.npz` (2.4 MB). About 30 seconds on a laptop CPU. Rerun it if the fixture is lost or the image cache changes.

LAB 10.5-A: LABS/MODULE-09/L10_5-TRANSFER-LEARNING/FEATURE_TABLE_STARTER.PY → FEATURE_TABLE_SOLUTION.PY

The solution prints:

```
(2000, 512) (1000, 512)
majority-class baseline (SeaLake): test accuracy 0.126
logistic regression         CV accuracy 0.899 ± 0.011
scaled logistic regression  CV accuracy 0.896 ± 0.009
test accuracy (logistic regression, used once): 0.903
 1-nearest neighbors (cosine): test accuracy 0.850
 5-nearest neighbors (cosine): test accuracy 0.866
15-nearest neighbors (cosine): test accuracy 0.861

weakest classes: {'River': 0.759, 'Highway': 0.835, 'AnnualCrop': 0.86}
most frequent mix-ups: {'River -> Highway': 11, 'AnnualCrop -> Pasture': 6, 'Forest ->
confidence < 0.6: 3.8% of tiles to a human; accuracy in the queue 0.474, on the rest 0
confidence < 0.8: 10.5% of tiles to a human; accuracy in the queue 0.571, on the rest (
PCA to  10 components: test accuracy 0.808
PCA to  50 components: test accuracy 0.870
PCA to 200 components: test accuracy 0.896
```

Reading it: (1) a cross-validated head on deep-learning features is ordinary Module 1 work, and scaling adds nothing here (the features are already on similar scales); test 0.903 matches the lesson's 0.902 on other tiles; (2)

kNN with cosine distance and no training reaches 0.866: the features alone organize the tiles; (3) River and Highway are the weakest classes and River → Highway the top mix-up, the same long-thin-line confusion as the lesson's project; (4) the human queue is a business tool: a tenth of the tiles to a person, 94% accuracy on the rest. Stretch: 50 of the 512 numbers keep most of the accuracy (0.870).

- LAB 10.5-B: LABS/MODULE-09/L10_5-TRANSFER-LEARNING/FREEZE_AUDIT_STARTER.PY → FREEZE_AUDIT_SOLUTION.PY

The solution prints:

```
freeze, then new head : 5,130 trainable
new head, then freeze : 0 trainable  <- nothing would ever learn
head only                     5,130 trainable (0.05% of 11,181,642)
head + layer4             8,398,858 trainable (75.11% of 11,181,642)
head + layer3 + layer4   10,498,570 trainable (93.89% of 11,181,642)
both groups  groups (lr, parameters): [(0.0001, 8393728), (0.001, 5130)]  trainable but
layer4 only  groups (lr, parameters): [(0.0001, 8393728)]  trainable but NOT in the opt

batch: (16, 3, 128, 128) labels: [4, 2, 0, 1, 4, 6, 7, 1, 9, 6, 0, 9, 1, 3, 6, 7]
eval() mode, loss 2.341: conv1.weight (frozen) same, layer4 conv (trainable) CHANGED, t
train() mode, loss 1.820: conv1.weight (frozen) same, layer4 conv (trainable) CHANGED,
(gradients flow in eval() mode too: eval() changes layer behavior, not gradient trackin

tensors that get a gradient: 17 of 62
```

Reading it: attaching the head before freezing leaves 0 trainable parameters (loud at `backward()`); an optimizer built without the head's group trains layer4 and leaves the random head untouched (silent: the loss barely moves). One step in eval() mode updates exactly the trainable tensors; in train() mode the frozen stem's batch-norm running mean moves too, because running averages are not gradients. That is the lesson's warning box: with batches under about 16, keep frozen parts in eval().

- HANDOUT: LABS/MODULE-09/L10_5-TRANSFER-LEARNING/DEBUG_TRANSFER.PY (DELIBERATELY BROKEN)

1. `RuntimeError: mat1 and mat2 shapes cannot be multiplied (64x1000 and 512x10)` → ResNet's head is `fc`, not `classifier`: `backbone.fc = nn.Identity()`. (Setting a `classifier` attribute on a ResNet raises nothing and does nothing: the features printed `(400, 1000)`.)
2. `RuntimeError: element 0 of tensors does not require grad and does not have a grad_fn` → the loop freezes the head too: freeze only `backbone.parameters()`.
3. Silent: no `backbone.eval()`, so features depend on the batch. Silent: no `Normalize(mean=..., std=...)` from the weights.
4. Measured on 400 / 200 tiles: loud bugs fixed only 0.800; plus eval() only 0.780; plus Normalize only 0.845; all four fixed 0.895. Two silent bugs together cost almost 10 points, with no message.

- LAB 11.1-A: LABS/MODULE-09/L11_1-TEXT-CLASSIFICATION/RECOMMEND_FLAG_STARTER.PY → RECOMMEND_FLAG_SOLUTION.PY

The solution prints:

```
(4000, 8) recommended share 0.655
recommended share by star rating: {1: 0.009, 2: 0.116, 3: 0.448, 4: 0.884, 5: 0.974}
3000 train, 1000 test; baseline accuracy 0.655
unigrams                       CV accuracy 0.726 ± 0.008
unigrams, stop words removed   CV accuracy 0.719 ± 0.015
unigrams + bigrams, min_df=2   CV accuracy 0.732 ± 0.013
                  precision    recall  f1-score   support

not recommended      0.716     0.481     0.575       345
    recommended      0.767     0.899     0.828       655

       accuracy                          0.755      1000
      macro avg      0.741     0.690     0.702      1000
   weighted avg      0.749     0.755     0.741      1000

toward NOT recommended: {'not': -2.81, 'not happy': -1.58, 'never again': -1.54, 'would
toward recommended    : {'really happy': 1.08, 'great': 1.16, 'really': 1.4, 'would buy
test accuracy by star rating: {1: 0.794, 2: 0.543, 3: 0.464, 4: 0.845, 5: 0.963}
Would buy again.                        P(recommended) 0.82
Would not buy again.                    P(recommended) 0.22
Not bad at all, but it runs small.      P(recommended) 0.58
It leaked on the first night.           P(recommended) 0.66
Great value. Though the zipper broke.   P(recommended) 0.68
```

Reading it: the checkbox is a noisier target than the stars (45% of 3-star reviewers recommend, 12% of 2-star), so the same pipeline reaches 0.755 against a 0.655 baseline instead of 0.877 against 0.745. Bigrams help a little in CV; stop words hurt again. The model is strong at the extremes and below a coin flip on 3-star reviews: an honest negative result worth reporting. The "leaked" probe at 0.66 shows that loose words appear in mixed reviews that still recommend.

- LAB 11.1-B: LABS/MODULE-09/L11_1-TEXT-CLASSIFICATION/CATEGORY_ROUTER_STARTER.PY → CATEGORY_ROUTER_SOLUTION.PY

The solution prints:

```
{'Apparel': 948, 'Accessories': 527, 'Paddling': 518, 'Climbing': 410, 'Footwear': 391,
baseline (always 'Apparel'): test accuracy 0.237
test accuracy 0.615, macro F1 0.585
lowest recall: {'Tents': 0.333, 'Camp Kitchen': 0.377, 'Footwear': 0.408}
highest recall: {'Accessories': 0.803, 'Apparel': 0.911}
most frequent wrong routes: {'Paddling -> Apparel': 63, 'Footwear -> Apparel': 43, 'Car
strongest words for Tents: ['camp canopy', 'canopy', 'camp', 'person', 'tent', 'this ca
Apparel      (0.31)  It kept us dry through a storm and the poles went up fast.
Apparel      (0.30)  Runs half a size small, my toes hit the front on descents.
Accessories  (0.21)  Arrived quickly. Great purchase.
confidence >= 0.5: 42.7% of messages routed automatically, accuracy 0.932; the rest go
class_weight='balanced': accuracy 0.580, macro F1 0.553, Tents recall 0.452, Apparel re
```

Reading it: many reviews never name the product, so routing is hard (0.615); large classes (Apparel) absorb small ones (Tents, Camp Kitchen). The probes show it: no product word, Apparel by default. Routing only confident messages (42.7% at 0.932) is the practical design. `class_weight="balanced"` trades overall accuracy for small-team recall: a business choice.

HANDOUT: LABS/MODULE-09/L11_1-TEXT-CLASSIFICATION/DEBUG_TEXT_PIPELINE.PY (DELIBERATELY BROKEN)

1. `ValueError: Found input variables with inconsistent numbers of samples: [1, 2321]` → `labeled["text"]` (a Series), not `labeled[["text"]]`.
2. Silent: `TfidfVectorizer(stop_words="english")` removes not; accuracy 0.864 instead of 0.877.
3. Silent: the threshold is chosen on the TEST set. Fix: `cross_val_predict(model, X_train, y_train, cv=StratifiedKFold(5, shuffle=True, random_state=42), method="predict_proba")` and the training labels. On this split both ways give 0.2: the number happens to agree, the method is still a leak (the recall you would report is not an honest estimate).
4. `ValueError: np.nan is an invalid document, expected byte or unicode string.` → `emails.fillna("")`. Fixed output: 0.877, threshold 0.2, P(negative) `[0.05 0.3 0.13]`. The refund email scores only 0.13: refund is not in the training vocabulary. Right for the wrong reasons is still worth a sentence in the debrief.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 10.5 · TRANSFER LEARNING

1. **C** So the network outputs its 512 features instead of 1,000 ImageNet class scores (freezing is requires_grad; Identity has no weights).
2. **A** `5130`: everything was frozen first, then a new Linear(512, 10) was attached: 5,120 weights + 10 biases.
3. **B** A very large labeled dataset of images that look nothing like everyday photos.

4. **B** The pretrained weights are already good and only need nudging; big steps could destroy them.
5. **A** The pretrained weights were learned on inputs scaled with exactly those numbers.

LESSON 11.1 · TEXT CLASSIFICATION

1. **B** `[[0 0 2]]`: the vocabulary is light, tent, warm; jacket is unknown and ignored; warm counted twice.
2. **A** The list removes negation words such as not, which carry a lot of sentiment.
3. **C** 1, the smallest possible value: ln((1 + n)/(1 + n)) + 1.
4. **A** The word appears mostly inside "not waterproof", and single words cannot see the not.
5. **C** cross_val_predict probabilities on the training reviews, then grade the chosen threshold once on the test set.

EXIT TICKET

1. **Backbone and head:** the backbone is every convolutional block (conv1 to layer4 plus pooling), frozen and in eval() mode; the head is `fc`, replaced by `nn.Identity()` (features) or a new `nn.Linear(512, 10)`.
2. **Bag-of-words failure:** Would not buy again. (unigrams 0.21, stop words removed 0.51) or Not bad at all. Fix: bigrams (`ngram_range=(1, 2)`): not bad gets +1.46, Would not buy again. drops to 0.11. Typos → character n-grams; synonyms → embeddings.
3. **Learned from training only:** 10.5: the head and anything chosen on validation (stage 1 vs 2); the ImageNet statistics are borrowed, not learned. 11.1: vocabulary and IDF (inside the pipeline), and the threshold (cross-validated probabilities).

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 D ("more accurate").** Confuses caching with improving. Same numbers, less time.
- **Q2 written as 0.** Thinks the freeze loop also covers the new head. Order matters: the head came after the loop.
- **Q4 A ("no_grad switches to eval").** The most common confusion in Module 10. no_grad = no gradient tracking; eval() = layer behavior.
- **Q9 C ("['not', 'buy']").** Assumes not is too important to be a stop word. That is exactly the trap.
- **Q11 written as 1.0.** Remembered the lesson quiz (word in every document) without counting: warm is in 2 of 4.
- **Exit ticket 3 citing the ImageNet mean and std as learned from training data.** They are the pretrained network's, not ours, and not learned at all here.

INSTRUCTOR KIT · ANSWER KEY

Session 10: Recommendations and Reproducibility

Answers and reasoning for every Session 10 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs and handouts, the lesson pages' own quizzes and the exit ticket.

Session 10 | 12 self-check items | Confidential

WARM-UP

Retrieval from Lessons 10.5 and 11.1. The code answers are real output.

W1

What does this print?

```
from torch import nn
from torchvision import models
m = models.resnet18(weights=None)
for p in m.parameters():
    p.requires_grad = False
m.fc = nn.Linear(512, 5)
print(sum(p.numel() for p in m.parameters() if p.requires_grad))
```

```
2565
```

Why: Frozen first, then a new 5-class head: 512 × 5 + 5 = 2,565. New layers are trainable by default.

W2

What does this print?

```
from sklearn.feature_extraction.text import CountVectorizer
print(CountVectorizer().build_analyzer()("Not bad, NOT bad at all!"))
```

```
['not', 'bad', 'not', 'bad', 'at', 'all']
```

Why: Lowercased, punctuation dropped, every occurrence kept in order by the analyzer. Counted as a bag: not 2, bad 2, at 1, all 1. A unigram model cannot tell Not bad from bad.

W3

Why must the backbone be in `eval()` mode while you extract features?

Batch normalization must use its stored ImageNet statistics; in training mode a tile's features depend on its batch and the stored statistics drift.

> **Why:** no_grad does not do this: it only switches off gradient tracking.

W4

Someone fits a `TfidfVectorizer` on all reviews, then splits. What is wrong, and how do you fix it?

Test reviews shape the vocabulary and the IDF values: a leak. Split first, fit inside a pipeline.

> **Why:** Today's recommender has the same rule in a new costume: the matrix is built from training purchases only.

W5

Frozen features: 0.885 with 500 labels, 0.902 with 2,000. What is the better next step than labeling more?

Fine-tune the top block (layer4) with a small learning rate: validation 0.876 → 0.924 in the project.

> **Why:** Measuring what more data buys, before paying for it, is the habit.

11.2 · RECOMMENDATION SYSTEMS: SELF-CHECK

Q1

A new recommender scores HR@10 = 0.004 on the leave-last-out test, where a random list scores 0.047. What is the most likely explanation?

A) A strong model on a hard dataset

B) A bug, such as sorting the scores ascending or forgetting to mask owned products

C) Too few factors in the SVD

D) The test item should have been chosen at random

> **Why:** A model that loses to random is almost always a bug, not a bad model. The lesson's ascending argsort scored 0.001: it picked the ten LEAST recommended products. Always compare with the random baseline.
> **A** is wrong: Even a weak model should beat random; far below it means something is inverted.
> **C** is wrong: Two factors still scored 0.220.
> **D** is wrong: The latest purchase is the right test item; it is not the cause.

Q2

What does this print?

```
import numpy as np
from scipy.sparse import csr_matrix
from sklearn.metrics.pairwise import cosine_similarity
R = csr_matrix(np.array([[1, 0, 1], [1, 1, 0], [0, 1, 1], [1, 0, 0]]))
print(cosine_similarity(R.T)[0].round(2))
```

`[1. 0.41 0.41]`

> **Why:** Columns are products. Product 0 was bought by customers 0, 1 and 3; product 1 by 1 and 2; product 2 by 0 and 2. Each pair shares one buyer: $1 / \sqrt{(3 \times 2)} = 0.41$. A product is identical to itself: 1.

Q3

Why does `top_k` set the scores of owned products to minus infinity before sorting?

A) Because cosine similarity is undefined for owned products

B) To make the hit rate look better than it is

C) Because SciPy cannot sort sparse matrices

D) Owned products are similar to what the customer bought, so they score high and crowd the list; customers would see items they already have

> **Why:** Without the mask 14% of list slots went to owned products and HR@10 fell from 0.308 to 0.270. Repeat-purchase items (socks, fuel) deserve a separate "buy it again" list.
>
> **A** is wrong: Cosine similarity is defined for every pair.
>
> **B** is wrong: The mask is honest: the test item is never owned in training. It raises the hit rate because the list is no longer wasted.
>
> **C** is wrong: The scores are a dense NumPy array by then.

Q4

The best-seller list shows 31 different products; item-kNN shows 161 and hits more often. What does that number measure?

A) Catalog coverage: how many different products appear on anyone's list

B) How many customers got a hit

C) How many products have at least one buyer

D) How many factors the model learned

> **Why:** Coverage is a second view next to hit rate: a list that only ever shows the head of the long tail serves the shop and its customers less well. Item-kNN reaches about 80% of the catalog.
> **B** is wrong: That is the hit rate (times 3,000).
> **C** is wrong: All 200 products have buyers.
> **D** is wrong: Item-kNN has no factors.

Q5

What does this print?

```
import numpy as np
from scipy.sparse import csr_matrix
from sklearn.decomposition import TruncatedSVD
svd = TruncatedSVD(n_components=3, random_state=42)
factors = svd.fit_transform(csr_matrix(np.eye(6, 8)))
print(factors.shape, svd.components_.shape)
```

`(6, 3) (3, 8)`

> **Why:** fit_transform returns the row factors (one row per customer, k columns); components_ holds the column factors (k rows, one column per product). Their product rebuilds a 6 × 8 score matrix.

Q6

You plot HR@10 against the number of SVD factors on the test item and pick the peak, k = 5. What is wrong with reporting 0.311?

A) Nothing: the test item was never used for training

B) SVD needs more than 5 factors to be valid

C) k was tuned on the test item, so 0.311 is optimistic; choose k on a validation item and score the test item once

D) Hit rate cannot be used to compare SVD models

> **Why:** Choosing a setting by its test score is tuning on the test set. The project holds out each customer's second-to-last purchase for validation (k = 5 scored 0.285 there, item-kNN 0.286), then measures the chosen model once.
> **A** is wrong: Using the test item to choose k is using it, even without training on it.
> **B** is wrong: k = 5 matches the five tastes in this data.
> **D** is wrong: Hit rate is exactly how they are compared.

11.2 · RECOMMENDATION SYSTEMS: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
import numpy as np
top = np.array([[3, 1], [0, 2], [4, 5]])
test = np.array([3, 2, 5])
print((top[:, :1] == test[:, None]).any(axis=1).mean().round(3))
```

`0.333`

> **Why:** Only the first column counts: customer 0's first pick 3 matches; customer 1's 0 is not 2; customer 2's 4 is not 5. One of three: 0.333. A shorter list hits less often (Exercise 1: HR@5 0.189 vs HR@10 0.308 for item-kNN).

P2

Predict the output, then run it.

```
import numpy as np
scores = np.array([0.9, 0.2, 0.5, 0.7])
owned = np.array([True, False, False, True])
scores[owned] = -np.inf
print(np.argsort(-scores)[:2])
```

`[2 1]`

> **Why:** Owned products 0 and 3 become minus infinity; the highest remaining scores are 0.5 (index 2) and 0.2 (index 1). The 0.9 product is never shown: the customer has it.

12.1 · REPRODUCIBILITY AND EXPERIMENT TRACKING: SELF-CHECK

Q7

A colleague reruns your churn notebook and gets test AUC 0.8560 instead of 0.8612. The data fingerprint and every library version match. Which ingredient most likely differs?

A) The data file

B) The scikit-learn version

C) The random seed of the forest

D) AUC is computed differently on each run

> **Why:** With the same data, code and environment, the remaining ingredients are randomness and settings. The lesson printed exactly these two numbers for seed 42 and seed 7. A third-decimal difference between seeds is noise.
> **A** is wrong: The fingerprint matches.
> **B** is wrong: The versions match.
> **D** is wrong: roc_auc_score is deterministic.

Q8

What does this print?

```
import hashlib
h = hashlib.sha256(b"customers_churn.csv contents").hexdigest()
print(len(h), h == hashlib.sha256(b"customers_churn.csv contents").hexdigest())
```

`64 True`

> **Why:** SHA-256 always gives 64 hexadecimal characters, and the same bytes always give the same fingerprint. One changed byte gives a completely different one.

Q9

What happens on the last line?

```
import skops.io as sio
from sklearn.ensemble import HistGradientBoostingClassifier
sio.dump(HistGradientBoostingClassifier().fit([[0], [1], [2], [3]], [0, 1, 0, 1]),
model = sio.load("q9.skops")
```

A) `UntrustedTypesFoundException` : it names the HistGradientBoosting tree type, which you must review and pass in `trusted=`

B) The model loads, because skops files are always safe

C) `pickle.UnpicklingError` : skops cannot read its own files

D) The model loads, but prints a warning about the scikit-learn version

> **Why:** skops only rebuilds types on its safe list. Here the file needs TreePredictor; the message even explains its risk. Review it, then load with a trusted= list that names exactly that type.
> **B** is wrong: Safer is not unconditional: unknown types need your decision.
> **C** is wrong: skops does not use pickle.
> **D** is wrong: Same environment; the refusal is about types, not versions.

Q10

Why should a model you share with another team be saved with skops rather than joblib?

A) skops files are smaller

B) joblib cannot save pipelines

C) skops makes the model more accurate

D) Loading a joblib file (a pickle) can run arbitrary code; skops only rebuilds known types or types you trust explicitly

> **Why:** The lesson's harmless pickle called print while LOADING. A malicious one could call anything with your permissions. Treat a model file like a program; skops removes the run-any-function attack.
> **A** is wrong: Often larger: in Lab 12.1 B the skops file was 462 kB and the joblib file 142 kB.
> **B** is wrong: joblib saved the whole pipeline in the lesson.
> **C** is wrong: Same model, same predictions.

Q11

What does this print?

```
from sklearn.ensemble import RandomForestClassifier
X, y = [[0], [1], [2], [3], [4], [5]], [0, 0, 1, 1, 0, 1]
a = RandomForestClassifier(n_estimators=5, random_state=3).fit(X, y).predict_proba(
b = RandomForestClassifier(n_estimators=5, random_state=3).fit(X, y).predict_proba(
print((a == b).all(), RandomForestClassifier().get_params()["random_state"])
```

`True None`

> **Why:** The same random_state gives the same forest, bit for bit. The default random_state is None: every fit draws new random numbers, so an unseeded forest changes run to run.

Q12

Which call lists the runs of an experiment with the best cross-validated AUC first?

A) `mlflow.search_runs(experiment_names=["churn"], order_by=["params.cv_auc DESC"])`

B) `mlflow.search_runs(experiment_names=["churn"], order_by=["metrics.cv_auc DESC"])`

C) `mlflow.search_runs(experiment_names=["churn"], order_by=["metrics.cv_auc ASC"])`

D) `mlflow.search_runs(experiment_names=["churn"], filter_string="max(cv_auc)")`

> **Why:** cv_auc was logged as a METRIC (a number), and higher is better, so DESC. Parameters are stored as text; for an error such as MAE you would sort ASC (Lab 12.1 A).
> **A** is wrong: cv_auc is a metric, not a parameter.
> **C** is wrong: ASC puts the worst first.
> **D** is wrong: Not valid filter syntax; filters look like metrics.cv_auc > 0.85.

12.1 · REPRODUCIBILITY AND EXPERIMENT TRACKING: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
import hashlib
same_hash = hashlib.sha256(b"6").hexdigest() == hashlib.sha256(b"6.0").hexdigest()
print(same_hash, float("6") == float("6.0"))
```

`False True`

> **Why:** The bytes 6 and 6.0 differ, so the fingerprints differ, although the numbers are equal. That is the lesson's Exercise 1: a re-saved CSV with the same data can have a new fingerprint.

P4

Predict the output, then run it.

```
import numpy as np
a = np.random.default_rng(42).permutation(5)
b = np.random.default_rng(42).permutation(5)
c = np.random.default_rng(7).permutation(5)
print((a == b).all(), (a == c).all())
```

`True False`

> **Why:** The same seed gives the same permutation; another seed almost surely a different one. Seeds make randomness repeatable, not absent.

LAB REFERENCE SOLUTIONS

- LAB 11.2-A: LABS/MODULE-10/L11_2-RECOMMENDATIONS/SEGMENT_BASELINES_STARTER.PY → SEGMENT_BASELINES_SOLUTION.PY

The solution prints:

```
(3000, 200) global best sellers HR@10 0.209
stores: {'Online': 1319, 'Boise': 531, 'Bend': 409, 'Asheville': 376, 'Flagstaff': 365}
favorite activity: {'camping': 811, 'general': 611, 'hiking': 492, 'climbing': 441, 'ru
global best sellers             HR@10 0.209   products shown  31   top-20 slots 100%
home-store best sellers         HR@10 0.209   products shown  36   top-20 slots 98%
favorite-activity best sellers  HR@10 0.265   products shown 183   top-20 slots 31%
item-kNN                        HR@10 0.308   products shown 161   top-20 slots 51%

favorite 'general' (611 customers): activity list 0.229, item-kNN 0.280; everyone else
```

Reading it: stores barely differ in what they sell, so the home-store list is the global list again (0.209, 98% of slots from the top 20). The favorite activity carries real taste: 0.265 and 183 products shown. Item-kNN still wins (0.308) because it uses the whole basket, not one label. Ties in `argmax` go to the first activity alphabetically (camping), one reason camping is the most common favorite. Stretch: customers whose favorite is "general" (sunscreen, first aid) get the least from either.

- LAB 11.2-B: LABS/MODULE-10/L11_2-RECOMMENDATIONS/USER_BASED_STARTER.PY → USER_BASED_SOLUTION.PY

The solution prints:

```
fit 26,178  validation 3,000  test 3,000
customer similarity matrix: (3000, 3000)
user-kNN,   10 neighbors: validation HR@10 0.202
user-kNN,   50 neighbors: validation HR@10 0.282
user-kNN,  200 neighbors: validation HR@10 0.292
user-kNN, 1000 neighbors: validation HR@10 0.266
item-kNN               : validation HR@10 0.286
chosen: 200 neighbors
test HR@10: user-kNN (200) 0.321, item-kNN 0.308
last purchase only: test HR@10 0.212
history + 1 x last purchase: test HR@10 0.305
history + 3 x last purchase: test HR@10 0.298
```

Reading it: too few neighbors (10) is noisy, too many (1,000) drifts toward popularity; 200 is chosen on validation, and only then does the test item speak: user-kNN 0.321 beats item-kNN 0.308. The last-purchase-only list (0.212) is barely above best sellers, and extra weight on the last purchase does not help: in this synthetic data the dates within the two years carry little sequence, so the whole basket is the signal. On real shop data with strong sequences (a tent, then a footprint) the result could differ: test it.

- HANDOUT: LABS/MODULE-10/L11_2-RECOMMENDATIONS/DEBUG_RECOMMENDER.PY (DELIBERATELY BROKEN)

1. `ValueError: operands could not be broadcast together with shapes (3000,10) (3000,)` → compare with `test_col[:, None]`.
2. `ValueError: matmul: dimension mismatch with signature (n,k=200),(k=3000,m)->(n,m)` → `cosine_similarity(R.T)`.
3. Silent: `np.argsort(scores, axis=1)` sorts ascending: random 0.013, best sellers 0.002, item-kNN 0.001 (below random: the giveaway) → `np.argsort(-scores, axis=1)`.
4. Silent: `.head(1)` holds out each customer's FIRST purchase, so the training rows contain the future (random 0.055, best sellers 0.228, item-kNN 0.310: plausible-looking numbers from the wrong question) → `.tail(1)`. Fixed: 0.047, 0.209, 0.308.

- LAB 12.1-A: LABS/MODULE-10/L12_1-REPRODUCIBILITY/TRACK_DEMAND_STARTER.PY → TRACK_DEMAND_SOLUTION.PY

The solution prints (MLflow's INFO lines go to the error stream and are not shown):

```
data fingerprint: c48cd2582060519c
fit 3,840  validation 1,248  test 4,896
ridge   validation MAE 12.63
forest  validation MAE 11.31
hgb     validation MAE 10.74
tags.mlflow.runName                tags.estimator  metrics.val_mae
               hgb HistGradientBoostingRegressor            10.74
            forest         RandomForestRegressor            11.31
             ridge                         Ridge            12.63
chosen: hgb; test MAE (2025, used once) 8.21
runs with validation MAE below 12: ['hgb', 'forest']
runs in the store: 4
hgb    seed 42 twice: 10.7445 and 10.7445; seed 7: 10.7445
forest seed 42 twice: 11.3146 and 11.3146; seed 7: 11.2821
```

Reading it: the store chooses hgb (validation MAE 10.74, sorted `ASC` because lower is better); one test score on 2025, 8.21, in the same league as the lessons' 8.15 to 8.21 with other features. Validation (October to December) is harder than 2025 as a whole. The store holds 4 runs: three candidates and the final run. The HGB regressor ignores its seed here (no subsampling; early stopping is off below 10,000 rows), the forest does not: record seeds anyway, because you cannot know in advance which models use them.

- LAB 12.1-B: LABS/MODULE-10/L12_1-REPRODUCIBILITY/SAFE_HANDOVER_STARTER.PY → SAFE_HANDOVER_SOLUTION.PY

The solution prints:

```
train 41,738, test (June) 8,262; test average precision 0.4743
untrusted types: ['sklearn.ensemble._hist_gradient_boosting.predictor.TreePredictor']
load without trusted=: UntrustedTypesFoundException
same predictions after reload: True
logistic regression  untrusted types: []
random forest        untrusted types: ['sklearn.tree._tree.Tree']
card keys: ['model_file', 'estimator', 'config', 'data_sha256', 'test_average_precisio
original data  -> data unchanged, score reproduced, AP: (True, True, 0.4743)
byte-for-byte copy ->  (True, True, 0.4743)
re-saved by pandas ->  (True, True, 0.4743)
one cent changed   ->  (False, True, 0.4743)
skops 462 kB, joblib 142 kB
```

Reading it: the trust list depends on the model (and on the data: this fraud pipeline needs no `numpy.dtype`, the lesson's churn pipeline did): logistic regression needs nothing reviewed, a forest needs `Tree`, HGB `TreePredictor`. A pandas re-save of THIS file is byte-identical (no missing values to turn into floats, unlike the lesson's Exercise 1), while one changed cent breaks the fingerprint although the score still reproduces: a fingerprint says the bytes differ, not what changed or whether it matters. skops is safer, not smaller (462 kB vs 142 kB).

- HANDOUT: LABS/MODULE-10/L12_1-REPRODUCIBILITY/DEBUG_REPRODUCIBILITY.PY (DELIBERATELY BROKEN)

1. `MlflowException: The filesystem tracking backend (e.g., './mlruns') is in maintenance mode...` → `mlflow.set_tracking_uri(f"sqlite:///{OUTPUT / 'mlflow.db'}")`.
2. `MlflowException: Changing param values is not allowed...` → one run per depth: put `with mlflow.start_run(run_name=f"forest-depth-{depth}"):` INSIDE the loop.
3. Silent: the forests in the loop have no `random_state`, so CV AUC changes on every run (two runs printed 0.8584 / 0.8674 and 0.8552 / 0.8662) → `random_state=42`.
4. Silent: `trusted=sio.get_untrusted_types(file=path)` trusts whatever the file asks for → `trusted=["sklearn.tree._tree.Tree"]`. Fixed, every run prints max_depth 4: 0.8584, max_depth 8: 0.8690, reloaded forest, test AUC 0.8637.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 11.2 · RECOMMENDATION SYSTEMS

1. **B** `0.667`: customers 0 and 2 have their test product in their top 2, customer 1 does not.
2. **C** It mimics predicting the next purchase from the past, so no future information leaks into training.
3. **A** With 40 factors the model memorizes past baskets instead of broad tastes, so it generalizes worse.
4. **C** Content-based filtering on product attributes (collaborative methods need purchases).
5. **A** `[1. 1. 0.]`: products 0 and 1 have exactly the same buyers; product 2 shares none.

LESSON 12.1 · REPRODUCIBILITY AND EXPERIMENT TRACKING

1. **C** Loading it can run arbitrary code on your computer.
2. **A** `True False`: same seed, same numbers; no seed, new numbers.
3. **B** `log_param("learning_rate", 0.05)` and `log_metric("cv_auc", 0.861)`: settings are params, results are metrics.
4. **B** Check each type is expected for this kind of model, then pass the reviewed types explicitly.
5. **A** The file is not byte-for-byte the one the model saw; the hash cannot say what changed.

EXIT TICKET

1. **0.308 vs 0.209:** the personal row contains the product a customer really bought next for about 31% of customers against 21% for the best-seller row: about 50% more hits, measured honestly on a held-out purchase.

2. **Five ingredients:** code (Git, a script like `train_churn.py`), data (SHA-256 fingerprint), environment (versions, `pip freeze` lock file), randomness (`random_state`, `default_rng(42)`), settings (one CONFIG, logged with `log_params` / `log_dict`).
3. **model.joblib:** loading it runs whatever the file says. Load only if you fully trust the sender; better, ask for skops, review `get_untrusted_types`, load with a hand-written `trusted=` list.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 A ("a strong model on a hard dataset").** Has not internalized the random baseline. Ask: "What would ten random picks score?"
- **Q2 written as `[1. 0.33 0.33]`.** Divided shared buyers by the total buyers instead of the geometric mean of the two counts.
- **Q6 A ("nothing wrong").** Thinks only training on the test set is a leak; choosing on it is too.
- **Q7 B ("the version").** Missed that the question says the versions match; seeds are the remaining ingredient.
- **Q10 A ("smaller files").** The opposite in Lab 12.1 B. Safety is the reason.
- **Q12 C ("ASC").** Correct instinct for an error metric (Lab 12.1 A), wrong for AUC.

INSTRUCTOR KIT · ANSWER KEY

Session 11: Serving, Monitoring and the Churn Case Study

Answers and reasoning for every Session 11 item: the warm-up, both self-checks, the predict-then-run items, the lab reference outputs, the lesson pages' own quizzes, the capstone plan check and the exit ticket.

Session 11 | 12 self-check items | Confidential

WARM-UP

W1

Lesson 11.2 hid each customer's LATEST purchase for testing, not a random one. Why?

It mimics the real job (predict the next purchase from earlier ones) and keeps future purchases out of the training matrix: the same rule as splitting by time in Lesson 1.2.

W2

Best sellers scored a hit rate at 10 of 0.209 and item-kNN 0.308. Say what 0.308 means in one sentence.

For about 31% of customers, the product they really bought next was in their personal top-10 list (already-owned products excluded): about 50% more often than the best-seller list.

W3

Why did SVD with 40 factors recommend worse than with 5?

With many factors it memorizes each customer's past basket instead of broad tastes, so it generalizes worse (0.19 against 0.311 at k = 5).

W4

A colleague emails you `churn_model.joblib`. What is the risk of loading it, and what did Lesson 12.1 use instead?

A joblib file is a pickle: loading it can run arbitrary code. Lesson 12.1 saved with skops, which loads only reviewed types passed to `trusted=`.

W5

Your model card stores a SHA-256 of `customers_churn.csv`. Today the hash differs. What do you know, and what don't you?

The file is not byte-for-byte the one the model was trained on; the hash cannot say what changed (one row, the order, a column).

12.2 · SERVING AND MONITORING MODELS: SELF-CHECK

Q1

A caller posts a customer with `"home_store": "Denver"` to the lesson's `/predict`. What happens?

A) The one-hot encoder ignores the unknown store and the caller gets status 200 with a probability

B) pydantic rejects it before the model runs: status 422, and the body names `home_store` and the allowed stores

C) uvicorn crashes and the caller gets status 500

D) `to_frame` raises a `KeyError` and the caller gets status 500

> **Why:** `home_store` is a `Literal` of five stores in `schemas.py`. FastAPI validates the body with the schema before calling the endpoint, so the model never runs and the 422 lists every problem at once.
> **A** is wrong: That is what would happen if a caller called the PIPELINE directly: the order window exists so it never does.
> **C** is wrong: A 500 means the server failed; an invalid request is the caller's problem, answered with 422.
> **D** is wrong: to_frame is never reached: validation comes first.

Q2

What does this print?

```
from pydantic import BaseModel, Field
class Visit(BaseModel):
    days: int = Field(ge=0, le=365)
    share: float = Field(ge=0, le=1)
v = Visit(days="30", share=1)
print(v.days * 2, v.share)
```

`60 1.0`

> **Why:** pydantic's default (lax) mode converts the text `"30"` to the integer 30 and the integer 1 to the float 1.0 the schema asks for; both pass their limits. Doubling an int gives 60.

Q3

Why does `to_frame` in `serve.py` end with `.astype(DTYPES)`?

A) It makes predictions faster

B) pydantic cannot produce a DataFrame without it

C) skops refuses to load the model unless the dtypes are stored

D) A one-row table with a `None` gets the `object` dtype, the imputer does not see it as missing, and the model scores a slightly different customer with no error

> **Why:** This is training-serving skew (Common Mistake 3): `object 0.0719` without the dtypes, `str 0.0708` with them. The cure is to store the training dtypes with the model and test that the API matches the notebook on real rows, gaps included.
> **A** is wrong: Speed is not the point; the numbers are.
> **B** is wrong: pydantic hands over plain Python values; pandas builds the frame.
> **C** is wrong: skops checks types inside the model file, not the request's columns.

Q4

This week `email_opens_90d` has a PSI of 8.4, its missing rate is 100% (9% in the reference), and the mean score barely moved. What is the best reading?

A) A broken data feed: fix it and re-score the week; do not retrain on it

B) Customers changed: retrain on this week's batch

C) Nothing to do: the scores did not move, so the model is fine

D) PSI cannot be trusted above 1, so ignore it

Why: Every value fell into the missing bin: the customers did not change, the pipe did. The imputer quietly filled the median, so the scores looked normal while 500 customers were scored as average email readers (the lesson's week 3).
B is wrong: Retraining on a broken feed teaches the model the bug, and the labels are months away anyway.
C is wrong: That is exactly the trap: only the missing-rate and PSI checks warned.
D is wrong: A huge PSI is honest here: it is the signature of a broken feed.

Q5

What does this print? (A two-bin PSI.)

```
import numpy as np
ref = np.array([0.5, 0.5])
cur = np.array([0.8, 0.2])
print(round(float(np.sum((cur - ref) * np.log(cur / ref))), 3))
```

A) `0.0`

B) `-0.416`

C) `0.416`

D) `0.3`

Why: (0.8 - 0.5) × ln(1.6) + (0.2 - 0.5) × ln(0.4) = 0.141 + 0.275 = 0.416. Both terms are positive: PSI never goes below 0, and it is well past the 0.25 "real shift" line.
A is wrong: That would need identical shares.
B is wrong: Each term is (difference) × ln(ratio), which always has the same sign twice: never negative.
D is wrong: 0.3 is the change in one share, not the index.

Q6

With 30,000 rows, a Kolmogorov-Smirnov test on discount share gives p < 0.0001, while the PSI is 0.021. Should the monitor raise an alert?

A) Yes: p below 0.05 means the inputs drifted

B) No: with enough rows any difference is detectable; PSI says the shift is tiny, and an alert should be about size

C) Yes: PSI is unreliable on large batches

D) No: KS tests only work on categories

> **Why:** Lesson 12.2 Exercise 4: the same 3% nudge gives p = 0.42 on 500 rows and p < 0.0001 on 30,000, while PSI stays near 0.02. A test answers "can I detect any difference?"; PSI answers "how big is it?".
> **A** is wrong: Significance grows with the sample size, not with the size of the change.
> **C** is wrong: PSI is about shares, so it hardly moves with the count.
> **D** is wrong: KS compares numeric distributions.

12.2 · SERVING AND MONITORING MODELS: PREDICT, THEN RUN

P1

Predict the output, then run it.

```
from pydantic import BaseModel, Field
class Order(BaseModel):
    units: int = Field(ge=1, le=10)
    note: str | None = None
print(Order(units="7").units + 1, Order(units=7).note)
```

`8 None`

> **Why:** Lax mode turns the text "7" into the integer 7 (7 + 1 = 8); `note` is optional and defaults to None.

P2

Predict the output, then run it.

```
import numpy as np
ref = np.array([0.25, 0.25, 0.25, 0.25])
for cur in [np.array([0.25, 0.25, 0.25, 0.25]), np.array([0.10, 0.20, 0.30, 0.40])]
    print(round(float(np.sum((cur - ref) * np.log(cur / ref))), 3))
```

`0.0 0.228`

> **Why:** Identical shares give 0.0. Moving 15 points of share from the first bin to the last gives 0.228: in the "watch" band, just under 0.25.

13.1 · CASE STUDY: PREDICTING CHURN: SELF-CHECK

Q7

Next quarter the retention offer costs $25 and is still worth $60 when it reaches a would-be churner. If the probabilities are calibrated, above which churn probability does an offer pay?

A) `0.25`

B) `0.5`

C) `0.417`

D) `2.4`

> **Why:** Offer when p × 60 > 25, so p > 25 / 60 = 0.417. The cost-scenario lab confirms it on out-of-fold predictions: the best threshold on the training data is 0.43.
>
> **A** is wrong: That was the $15 offer.
>
> **B** is wrong: 0.5 is predict's default, not a business rule.
>
> **D** is wrong: That is value / cost, upside down.

Q8

What does this print? ($15 an offer, $60 a churner reached.)

```
import numpy as np
y = np.array([1, 0, 1, 0, 0, 0])
flagged = np.array([1, 1, 1, 1, 0, 0])
print(60 * ((flagged == 1) & (y == 1)).sum() - 15 * flagged.sum())
```

`60`

Why: Four offers, two of them reach churners: 2 × 60 - 4 × 15 = 120 - 60 = 60.

Q9

Cross-validated AUC: random forest 0.868 ± 0.020, gradient boosting (defaults) 0.861 ± 0.013. What may you conclude?

A) The data does not yet say which tree family is better: the gap is inside one fold-to-fold spread

B) The forest is better and should be deployed

C) Boosting is better because its spread is smaller

D) Open the test set to break the tie

Why: A gap of 0.007 with spreads of 0.013 to 0.020 is noise. The case study tunes boosting because its defaults leave the most room; the tuned model's 0.879 then clears both by more than a spread.

B is wrong: 0.007 is well inside the noise.

C is wrong: A smaller spread is not a higher score.

D is wrong: The test set is opened once, after every choice is made: never to choose.

Q10

Which order matches the case study?

A) Tune, split, leak check, threshold, test, explain

B) Split, threshold on the test set, compare, tune, explain

C) Compare on the test set, tune with CV, choose the threshold on the test set, explain

D) Split, leak check, compare with CV, tune with CV, threshold with CV, test once, explain and write up

Why: Everything before "Open the Test Set, Once" uses the 4,500 training customers through cross-validation. The test set gives one honest estimate of the whole plan.
A is wrong: Tuning before the split lets the test rows shape the model.
B is wrong: Choosing the threshold on the test set inflates the reported profit (Mistake 4: $5,280 looks better than the honest $5,220).
C is wrong: Every choice made on the test set makes its score optimistic.

Q11

The manager asks whether the coupon actually keeps customers, or only goes to people who would have stayed anyway. What does the memo suggest?

A) Look at permutation importance: the top feature is the cause

B) Hold back the offer from a random 10% of the list and compare who stays

C) Compare churn of customers who used discounts with those who did not

D) Raise the threshold until precision is 100%

Why: Only an experiment answers a causal question. A random hold-out from the same high-risk list gives two comparable groups; the difference in churn is what the offer really does.
A is wrong: Importance describes what the model relies on, not why customers leave.
C is wrong: Discount users differ from others in many ways: a comparison, not an experiment.
D is wrong: Precision says the list is accurate, not that the coupon works.

Q12

The budget allows 200 offers; 360 test customers are above the break-even probability. Which rule does the case-study project use?

A) The 200 riskiest customers, and never anyone below the break-even probability

B) 200 customers drawn at random from the 360

C) All 360: the budget is a guideline

D) The 200 customers closest to the break-even probability

Why: Rank by probability and spend the budget from the top; an offer below break-even loses money even if the budget allows it. On the test customers that list reaches 131 churners and earns $4,860, against -$540 for 200 random offers.

B is wrong: Random offers from the list waste the ranking.

C is wrong: It breaks the budget.

D is wrong: Those are the least valuable offers above the line.

13.1 · CASE STUDY: PREDICTING CHURN: PREDICT, THEN RUN

P3

Predict the output, then run it.

```
import numpy as np
y = np.array([1, 1, 0, 0, 0])
offer = np.array([1, 0, 1, 1, 0])
print(60 * ((offer == 1) & (y == 1)).sum() - 15 * offer.sum())
```

`15`

Why: Three offers, one reaches a churner: 60 - 45 = 15.

P4

Predict the output, then run it.

```
for cost, value in [(15, 60), (20, 50), (30, 60)]:
    print(f"{cost / value:.2f}", end="  ")
```

`0.25 0.40 0.50`

Why: 15/60 = 0.25, 20/50 = 0.40, 30/60 = 0.50. A pricier offer needs a riskier customer to pay.

LAB REFERENCE SOLUTIONS

LAB 12.2-A: LABS/MODULE-11/L12_2-SERVING/FRAUD_API_STARTER.PY → FRAUD_API_SOLUTION.PY

The solution prints:

```
train 33,182 payments (309 fraud), May 8,556, June 8,262
types to review: ['functools.partial', 'sklearn.ensemble._hist_gradient_boosting.predic
matches the reviewed list: True
threshold 0.0775  reference hold rate 0.0101  reference mean score 0.0084

4 problems
  amount -> Input should be greater than 0
  hour -> Input should be less than or equal to 23
  items -> Field required
  category -> Input should be 'Accessories', 'Apparel', 'Camp Kitchen', 'Climbing', 'Fo

 {'status': 'ok', 'model_version': 'fraud-hgb-2025-04-30'}
200 {'fraud_score': 0.0021, 'hold': False, 'model_version': 'fraud-hgb-2025-04-30'}
422 ['amount', 'hour', 'items', 'category']

API matches the notebook: True
June: 83 of 8,262 held (1.00%; reference 1.01%), 35 of 64 frauds among them
```

Reading it: the threshold is the 99th percentile of May's scores, so about 1% are held (the team's capacity); June holds 83 (1.00%) with 35 of 64 frauds. The trusted list has three entries; two (`functools.partial`, `check_array`) are there because `categorical_features="from_dtype"` stores a preprocessing function in the model. They are expected for this model, which is exactly what a reviewer must confirm before writing them out. The 422 lists all four problems; the match test is the habit from the lesson.

LAB 12.2-B: LABS/MODULE-11/L12_2-SERVING/FRAUD_MONITOR_STARTER.PY → FRAUD_MONITOR_SOLUTION.PY

The solution prints:

```
reference: 24,881 payments, fraud rate 0.95%, hold threshold 0.0984
       max_psi  category_psi  mean_score  hold_rate
month
April    0.003        0.0007      0.0088     0.0089
May      0.003        0.0013      0.0084     0.0089
June     0.003        0.0009      0.0082     0.0080
reference mean_score 0.0095, hold_rate 0.0100
       fraud_rate  held  frauds_held  frauds  avg_precision
month
April      0.0087    74           37      72          0.519
May        0.0076    76           33      65          0.521
June       0.0077    66           33      64          0.531
June 1-15 : velocity PSI 0.000, hold rate 0.0074, frauds held 14 of 29
June 16-30: velocity PSI 2.523, hold rate 0.0025, frauds held 10 of 35
June 16-30 with the feed intact: hold rate 0.0086, frauds held 19 of 35

April to June: no input drift (every PSI under 0.01) and the hold rate stays near 1%. W
arrived, average precision was steady, so the falling fraud rate did not hurt the rank:
June 16 to 30: velocity_24h shifted (PSI far above 0.25) because the feed broke. Fix th
re-score those payments; do not retrain on them.
```

Reading it: real months do not drift in their inputs (every PSI ≤ 0.003); the fraud rate falls from 0.95% to 0.77%, so the hold rate drifts slightly under 1%, yet average precision is steady once labels arrive. "No drift, and the model kept its ranking" is a real, reportable result. The broken feed is the contrast: unlike the lesson's email tracker (where the imputer hid the damage), a zeroed `velocity_24h` drops the scores: 10 instead of 19 frauds held in the second half of June.

HANDOUT: LABS/MODULE-11/L12_2-SERVING/DEBUG_SERVING.PY (DELIBERATELY BROKEN)

1. `KeyError: 0` on the first run: the schema says `amount: int`, so every payment with cents is rejected (422) and the response is `{"detail": [...]}`, not a list. Fix: `amount: float`. Lesson: print `response.status_code` before reading the body.
2. A crash comparing a float with a string (`ufunc 'greater_equal' did not contain a loop...`): the threshold was stored as text in the JSON. Fix: store it as a number (or `float(...)`).
3. Silent: `round(float(p), 2)`. Fraud scores are around 0.01, so two decimals turn most of them into 0.0 or 0.01; the match test prints `False`. Fix: four decimals, as the lesson.
4. Silent: `hold` comes from `model.predict` (cut-off 0.5): 28 held instead of 83. Fix: `scores >= threshold`. Fixed, the file prints `API matches the notebook: True` and `June: 83 held, 35 of them fraud`.
5. Not a bug, worth saying: scikit-learn 1.9's gradient boosting with `from_dtype` maps categories by value, so a frame whose categories are listed in another order still scores correctly; one-hot encoding done outside a pipeline would not be so forgiving.

LAB 13.1-A: LABS/MODULE-11/L13_1-CASE-STUDY/LEAK_AUDIT_STARTER.PY → LEAK_AUDIT_SOLUTION.PY

The solution prints:

```
                       auc_alone  filled_if_1  filled_if_0  distinct                  
column                                                                               
cancellation_reason        0.815        0.630        0.000         6  FILLED FOR ONE CL
days_since_last_order      0.715        1.000        1.000       346                    
spend_12m                  0.652        1.000        1.000      5843                    
discount_share             0.633        1.000        1.000       722                    
orders_12m                 0.627        1.000        1.000        21                    
loyalty_tier               0.596        1.000        1.000         3                    
email_opens_90d            0.589        0.859        0.921        36                    
app_user                   0.575        1.000        1.000         2                    
avg_order_value            0.567        1.000        1.000      5034                    
support_tickets_12m        0.566        1.000        1.000         7                    
preferred_category         0.561        1.000        1.000         9                    
tenure_months              0.560        1.000        1.000        96                    
returns_12m                0.524        1.000        1.000        10                    
region                     0.516        1.000        1.000         6                    
signup_date                0.515        1.000        1.000      2384                    
home_store                 0.503        1.000        1.000         5                    
customer_id                0.500        1.000        1.000      6000               ID-L
age_group                  0.495        0.917        0.919         6                    
return_rate                0.477        1.000        1.000       436                    

flagged: {'cancellation_reason': 'FILLED FOR ONE CLASS', 'customer_id': 'ID-LIKE'}
strongest single honest column: days_since_last_order (0.715)
cancellation_reason alone: AUC 0.815, filled for 63% of churners and 0% of stayers

                          auc_alone  filled_if_1  filled_if_0  distinct     flag
column                                                                          
velocity_24h                  0.782          1.0          1.0         8         
amount                        0.708          1.0          1.0     27674         
account_age_days              0.689          1.0          1.0      3351         
device_new                    0.686          1.0          1.0         2         
card_attempts                 0.651          1.0          1.0         5         
hour                          0.616          1.0          1.0        24         
items                         0.614          1.0          1.0         7         
billing_shipping_mismatch     0.599          1.0          1.0         2         
country_mismatch              0.565          1.0          1.0         2         
category                      0.559          1.0          1.0        10         
transaction_id                0.500          1.0          1.0     50000  ID-LIKE

flagged: {'transaction_id': 'ID-LIKE'}
```

Reading it: only one rule catches the planted leak, "filled for one class": its single-column AUC (0.815) is high but not absurd, so a "too good" rule alone would miss it. `customer_id` is a label (AUC 0.500 because test IDs never

appear in training). `return_rate` below 0.5 means a single straight line on it has no stable direction. The fraud table is clean: say so in a capstone's data section.

LAB 13.1-B: LABS/MODULE-11/L13_1-CASE-STUDY/COST_SCENARIOS_STARTER.PY → COST_SCENARIOS_SOLUTION.PY

The solution prints:

```
4500 training customers, out-of-fold probabilities from 5 folds, mean 0.164
            break_even  best_threshold  offers  profit_best  profit_at_0.25
offer_cost
10               0.167            0.17    1362        22800           21890
15               0.250            0.24    1067        16935           16755
20               0.333            0.31     825        12000           11620
25               0.417            0.43     522         8490            6485
        lowest_p  churners  block_profit
offers
100        0.754        83          3480
200        0.679        82          3420
300        0.590        72          2820
400        0.503        58          1980
500        0.445        53          1680
600        0.392        34           540
700        0.355        34           540
800        0.317        48          1380
900        0.288        29           240
1000       0.258        35           600
1100       0.231        29           240
1200       0.213        17          -480
1300       0.183        20          -300
1400       0.162        15          -600
1500       0.145        17          -480
1600       0.130        11          -840
the block ending at offer 1200 is the first that loses money (its lowest probability is
$25 offer, old threshold 0.25  :  360 offers, profit $1,620
$25 offer, new threshold 0.43  :  183 offers, profit $2,745
$25 offer, break-even 0.417    :  194 offers, profit $2,770
```

Reading it: the best threshold on out-of-fold predictions tracks cost / 60 within 0.02, because the tuned boosting model is close to calibrated (13.1 Exercise 3). Keeping 0.25 after the coupon grows to $25 costs about $2,000 per 4,500 customers on the training data and $1,125 on the 1,500 test customers. Blocks of 100 offers stop paying around p = 0.21 to 0.23, which is the break-even rule seen from the budget side.

THE LESSON PAGES' OWN QUICK QUIZZES

LESSON 12.2 · SERVING AND MONITORING MODELS

1 **B** FastAPI answers 422 and the body explains which field is wrong: validation runs before the endpoint.

2. **C** 6: lax mode converts "3" to the integer 3; doubled, 6.
3. **A** Find out why recency changed (data bug or real change): drift says where to look.
4. **C** The AUC on last quarter's customers: the only one that needs labels.
5. **B** Training-row predictions are over-confident; out-of-fold predictions look like new data.

LESSON 13.1 · CASE STUDY: PREDICTING CHURN

1. **A** 0.25: $20 / $80.
2. **C** Remove it: it is recorded after the outcome, so it is a leak.
3. **B** Once, after the model, hyperparameters and threshold are all chosen.
4. **B** Customers who rely heavily on discounts are more likely to leave (a description, not a cause).
5. **A** 30: one churner reached (+60) and two offers (-30).

CAPSTONE CHOICE: WHAT TO LOOK FOR

A plan paragraph is ready when it names a decision and its user, the prediction moment, the split the template uses, a metric with its business meaning, the baseline number, and a "do not deploy" bar. The most common gaps: no prediction moment (so no leakage test is possible), a metric without a unit ("accuracy" for fraud), and no bar. Coaching notes per problem are in `final-assessment.html`.

EXIT TICKET

1. Two silent failures: a frame without the training dtypes (caught by the match test on rows with gaps); a broken feed hidden by the imputer (caught by missing rates and PSI); an impossible value (caught by schema limits).
2. 0.333 (= 20 / 60). Check it on out-of-fold predictions or with TunedThresholdClassifierCV; the rule assumes calibrated probabilities. The lab's best for $20 was 0.31.
3. A letter, a decision and a number, for example "D: the ten products on each customer's home page; best-seller hit rate 0.209".

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q1 A ("the encoder ignores it").** Thinks of the pipeline as the front door. Draw the window again: the schema runs first.
- **Q3 A or C.** Has not seen the silent skew. Rerun demo Step 10 for them.

- **Q4 B ("retrain").** Treats drift as a verdict. Ask which labels they would retrain on.
- **Q5 B (a negative PSI).** Expects the terms to cancel. Each term is a difference times the log of the same ratio: always ≥ 0.
- **Q9 B.** Reads a gap without its spread. Ask for the $\pm$ numbers every time.
- **Exit ticket 2: "0.333, done".** Right number, missing check: the case study's whole point is theory confirmed by data.

INSTRUCTOR KIT · ANSWER KEY

Session 12: Capstone Kickoff and Studio I

Answers and reasoning for Session 12: the warm-up, the twelve self-check items, the predict-then-run items, the two capstone labs' reference outputs, the lesson's own quiz and the exit ticket.

Session 12 | 12 self-check items | Confidential

WARM-UP

W1

Why does `serve.py` rebuild the request frame with the stored training dtypes?

Otherwise a None in a one-row table makes an object column the imputer does not treat as missing: a silently different score (training-serving skew).

W2

Of input drift, data quality, prediction drift and performance, which needs labels?

Only performance (AUC, offer precision), and the labels arrive months later.

W3

Where does the churn threshold 0.25 come from, and what does it assume?

$15 / $60: an offer pays when p × 60 > 15. It assumes calibrated probabilities, which the case study confirmed with TunedThresholdClassifierCV.

W4

In the case study, when is the test set used?

Once, after features, model, hyperparameters and threshold are all chosen.

W5

Your capstone in one sentence: the decision, and the baseline number you must beat.

No single answer. Listen for a decision (not a topic) and a number; anyone without one gets the first visit in the plan clinic.

13.2 · CAPSTONE PROJECT: SELF-CHECK

Q1

Which first line of a capstone plan names a DECISION, not just a target?

A) Each morning the two-person review team decides which of the day's payments to hold; the model ranks them

B) Predict is_fraud from the transaction columns

C) Build the most accurate fraud model possible

D) Use gradient boosting on transactions.csv

Why: A decision has a user and an action (who holds which payments, how many). From it follow the prediction moment, the capacity (about 1% a day) and the right metric.
B is wrong: A target with no user or action: rubric Framing row 1.
C is wrong: "Most accurate" names no decision and invites accuracy on a rare event.
D is wrong: A method is a choice made late, not a plan.

Q2

The demand order is placed three weeks ahead. Which column must NOT be a feature as it stands?

A) `promo` (planned weeks ahead)

B) `avg_high_temp_f`, the actual temperature of the forecast week

C) `week_of_year`

D) `list_price`

Why: The week's real temperature is not known when the order is placed. Use something known then (last year's temperature for that week) or leave it out; on this data it adds nothing measurable once the month is in.
A is wrong: Planned promotions are known ahead.
C is wrong: The calendar is always known.
D is wrong: List prices are set in advance.

Q3

What does this print?

```
import pandas as pd
df = pd.DataFrame({"c": ["a", "a", "b", "b", "b"], "d": [3, 1, 2, 5, 4]}).sort_valu
print(df.groupby("c").tail(1)["d"].tolist())
```

`[3, 5]`

> **Why:** After sorting by c and d, tail(1) keeps the last row of each group: the largest d for a (3) and for b (5). The recommendations template uses the same move to hide each customer's latest purchase.

Q4

Why does the fraud harness validate on May instead of a shuffled 5-fold split of January to May?

A) Shuffled folds are slower

B) May has more fraud

C) The model will score future months, and the fraud rate drifts; validating on a later month mimics that

D) scikit-learn cannot shuffle time stamps

> **Why:** Train on the past, check on the next month: the same rule as the June test. On this data logistic regression scores 0.539 average precision on May.
>
> **A** is wrong: Speed is not the issue.
>
> **B** is wrong: May has the LOWEST rate of the five training months (0.76%).
>
> **D** is wrong: It can; it should not here.

Q5

On recommendations, how should you choose the number of SVD factors k?

A) The k with the best hit rate on the test purchases

B) Always k = 5, as in Lesson 11.2

C) The largest k the laptop can fit

D) The best k on a validation purchase carved from the training rows (each customer's second-to-last)

Why: Choosing on the test item makes the test score optimistic. The validation item gives SVD k = 5 0.285 (k = 3: 0.245, k = 10: 0.248); then the test is used once.
A is wrong: That uses the test set to choose.
B is wrong: A good guess, but a capstone shows its own evidence.
C is wrong: More factors memorize: worse, not better.

Q6

K-Means on the four UNscaled behavior columns gives silhouette 0.545, on scaled columns 0.470. Which segments should you keep?

A) The scaled ones: unscaled distances are dominated by average order value, so the higher number comes from a worse clustering

B) The unscaled ones: a higher silhouette is always better

C) Neither: silhouette below 0.6 means no segments

D) Whichever has more equal cluster sizes

Why: Without scaling, a column in dollars outweighs shares between 0 and 1. The unscaled clusters split mostly on order value (sizes 286, 620, 686, 1616, 2792) and agree poorly with the scaled ones (ARI 0.267). A metric compares like with like only on the same features.
B is wrong: Silhouette is only comparable on the same feature space.
C is wrong: There is no such cut-off; 0.47 is clear structure here.
D is wrong: Size balance is not the goal.

Q7

What does this print? (A per-product-mean baseline.)

```
import numpy as np
from sklearn.metrics import mean_absolute_error
train_units = {"TNT-1": [10, 20], "SNW-1": [0, 4]}
test_units = [("TNT-1", 18), ("SNW-1", 1)]
means = {k: np.mean(v) for k, v in train_units.items()}
print(mean_absolute_error([u for _, u in test_units], [means[p] for p, _ in test_un
```

`2.0`

Why: The means are 15 (tent) and 2 (snow). Errors |18 - 15| = 3 and |1 - 2| = 1; the mean absolute error is 2.0.

Q8

Your first pipeline does not beat the baseline in your harness. What does the lesson say to do first?

A) Try a neural network

B) Tune harder

C) Look for a bug (a split, a feature, a leak in reverse) before looking for a better model

D) Report "do not deploy" at once

Why: A leak-free simple pipeline that loses to a per-product average usually has a feature or split problem. Only once that is ruled out is a close result a finding.

A is wrong: A bigger model hides the bug.

B is wrong: Tuning a broken setup tunes the bug.

D is wrong: Maybe later, with evidence; not before checking.

Q9

Review triage drops 3-star reviews from training. What belongs in the plan?

A) Nothing: 3-star reviews are neutral

B) Add them to the positive class

C) Delete 3-star reviews in production too

D) Say what the live system does with them and measure it (how many get flagged as negative)

Why: Production will receive them. On this data the model flags about a fifth of 3-star reviews as negative at 0.5, and two thirds at a high-recall threshold: a behavior to report, not to hide.
A is wrong: "Neutral" is an assumption the model does not share.
B is wrong: That changes the question without saying so.
C is wrong: You cannot choose which reviews customers write.

Q10

When should the "do not deploy" bar be written?

A) After the test score, to explain it

B) Before any modeling, in the plan

C) Only if the model does badly

D) In the model card's last section

Why: A bar chosen after seeing the result can always be met. Written in the plan, it makes "do not deploy" a legitimate, rubric-worthy answer (Framing row, Exemplary).
A is wrong: That is rationalizing.
C is wrong: Then it is never written when it matters.
D is wrong: The card reports it; the plan sets it.

Q11

In June, new accounts are 8% of legitimate payments but about 29% of the payments the fraud model wrongly holds. What does the responsible-ML checklist ask?

A) Name who bears the errors, keep a human review with a fast release path, and report results by account age

B) Remove account_age_days so the model cannot see it

C) Nothing: new accounts really are riskier

D) Lower the threshold until the share matches

Why: Holds are reviewed, not declined, and the burden falls on new customers: say so, design for it, and show the subgroup numbers in the card.
B is wrong: Removing a useful, non-sensitive feature can make every group worse; measure first.
C is wrong: True, and still a cost someone pays: it must be written down.
D is wrong: That changes capacity, not who bears the errors.

Q12

Demand harness inside 2024 (rolling folds): ridge MAE 13.13 against the baseline's 13.92, yet on 2025 ridge scores 11.29 against 13.42. Why is the in-2024 harness so pessimistic?

A) Ridge overfits 2024

B) The test set is easier by chance

C) Each validation block contains months the folds have never seen, so a month effect learned from one year cannot be checked forward inside that year

D) MAE is not a valid metric for rolling folds

Why: With one training year, rolling folds always validate on unseen seasons. The harness still ranks candidates fairly against the same baseline (13.924 vs 13.128); say in the card why its numbers are higher than the test's.
A is wrong: The 2025 score shows it generalizes.
B is wrong: The gap appears for every model the same way.
D is wrong: MAE works for any fold.

PREDICT, THEN RUN

P1

Predict the output, then run it.

```
import pandas as pd
s = pd.Series([10, 0, 30, 0])
print(s.shift(1).tolist())
```

```
[nan, 10.0, 0.0, 30.0]
```

> **Why:** shift(1) moves every value one row later: row 2 sees row 1's value. The first row has no past: NaN. This is how a lag feature only sees earlier days.

P2

Predict the output, then run it.

```
import pandas as pd
df = pd.DataFrame({"customer": ["a", "a", "a", "b", "b"], "item": ["x", "y", "z", "
print(df.groupby("customer").tail(1)["item"].tolist(), len(df.groupby("customer").h
```

```
['z', 'w'] 3
```

> **Why:** tail(1) keeps each customer's last row (z for a, w for b): the test items. head(-1) keeps all but the last row of each group: 2 + 1 = 3 training rows.

WORKED EXAMPLE, EXPLAINED

Shuffled 5-fold CV ($551) is optimistic because neighboring days leak into training; the time-ordered folds ($659) are pessimistic because early folds train on little history; the one look at 2025 gives $600. The lesson's worked example uses more lag features (lag_14, a 7-day mean, events) and reaches $561.

LAB REFERENCE SOLUTIONS

- CAPSTONE LAB 1: LABS/MODULE-12/L13_2-CAPSTONE/BASELINES_STARTER.PY → BASELINES_SOLUTION.PY

The solution (all five problems) prints:

```
                                          metric   value
problem  baseline
demand   per-product mean                MAE (units)  13.419
         per product x store mean        MAE (units)  12.953
         same week last year             MAE (units)  13.810
fraud    no model                  average precision   0.008
         rule: velocity_24h        average precision   0.250
reviews  always positive                    accuracy   0.745
         keyword rule                       accuracy   0.788
recs     most popular unowned         hit rate at 10   0.209
         random unowned               hit rate at 10   0.053
segments existing loyalty tiers           silhouette   0.079
         five random groups               silhouette  -0.023

fraud rule, top 1% of June (83 payments): 19 of 64 frauds
reviews keyword rule: negative recall 0.477 (always positive: 0.000)
```

Reading it: the per product × store mean is a stronger demand baseline than the template's per-product mean, while "same week last year" is WORSE (one noisy week against a yearly average). The velocity rule shows that a sensible one-line rule already catches 19 of 64 June frauds in its top 1%: the bar a fraud model must clear by a margin. The keyword rule beats "always positive" on reviews and finds about half the complaints. Random groups give a silhouette near 0: the loyalty tiers (0.079) are barely better than random.

CAPSTONE LAB 2: LABS/MODULE-12/L13_2-CAPSTONE/HARNESS_STARTER.PY → HARNESS_SOLUTION.PY

The solution prints:

```
                                    score  second
problem  candidate
demand   per-product mean           13.924   1.986
         ridge, one-hot             13.128   2.330
fraud    no model                    0.008     NaN
         logistic regression         0.539     NaN
reviews  always positive             0.745   0.000
         tf-idf + logistic regression  0.857   0.577
recs     popularity                  0.209     NaN
         item-kNN                    0.286     NaN
         SVD k=3                     0.245     NaN
         SVD k=5                     0.285     NaN
         SVD k=10                    0.248     NaN
segments existing loyalty tiers      0.079     NaN
         k-means k=4                 0.413   0.999
         k-means k=5                 0.470   1.000
         k-means k=6                 0.433   0.953

harness per problem: {'demand': 'MAE, 4 rolling folds (lower is better)', 'fraud': 'ave
segments k=5: model fitted on half the customers agrees with the full model, ARI 0.995
```

Reading it: every harness uses training data only. Demand rolling folds are pessimistic (validation months unseen in one year), but they rank the ridge above the baseline as the test does. On fraud, a plain logistic regression already reaches 0.539 on May. On reviews, the first pipeline's negative recall (0.577) is the number to improve. On recs, validation picks k = 5 and item-kNN ties with SVD. On segments, k = 5 has the best silhouette and is perfectly stable across seeds; the half-sample check agrees (ARI 0.995).

THE LESSON PAGE'S OWN QUICK QUIZ (13.2)

1. **B** Replace the actual temperature with something known three weeks ahead, such as last year's temperature for that week.
2. **C** What the review team would catch at its capacity, with average precision next to the no-model baseline.
3. **A** `[nan, nan, 15.0, 25.0]`: shift first, then a 2-row mean.
4. **B** The model relies heavily on discount share; this describes the model, not why customers leave.
5. **C** Check stability across seeds and subsamples, and that each segment has a distinct, nameable profile.

EXIT TICKET

1. A harness sentence names training rows, validation rows and the metric; on time-ordered data validation always comes after training.
2. Two baselines with numbers on the same split and metric (see lab 1 for the reference values).
3. Any column that fails the timing test, a label such as an ID, or a sensitive attribute kept out of the model and checked afterwards.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q2 A (promo).** Confuses "changes week to week" with "unknown in advance". Promotions are planned.
- **Q5 A (test hit rate).** The most common capstone flaw on recommendations; Lesson 11.2's teaching curve is on the test set, so say why the capstone must differ.
- **Q6 B.** Reads a metric without its feature space.
- **Q12 A ("overfits").** Does not yet see that the harness, not the model, is pessimistic.

INSTRUCTOR KIT · ANSWER KEY

Session 13: Capstone Studio II and Showcase

Answers and reasoning for Session 13: the warm-up, the twelve self-check items, the predict-then-run items, the capstone labs' reference outputs (including the review handout's honest numbers), the lesson's quiz and the exit ticket.

Session 13 | 12 self-check items | Confidential

WARM-UP

W1

Why does the fraud harness validate on May, not on shuffled folds?

The model scores future months and the fraud rate drifts; validating on a later month mimics that.

W2

What is the one loop the lesson does not allow?

From the test set back to tuning: once you have looked at the test score, the model is finished.

W3

Name two parts of a model card that a weak card usually lacks.

The baseline on the same test set (with an interval for the model), and limitations specific to the data; also often the out-of-scope uses.

W4

What does "explanations are not causes" mean for your card?

Importance and SHAP describe what the model relies on; words like "causes" or "drives" need a design that supports them (none in this course).

W5

Status: your final candidate, its CV score and the baseline's, in one line.

No single answer. Listen for a candidate chosen on training data, and for anyone who has already looked at the test set: they write that in their card.

13.2 · CAPSTONE PROJECT: SELF-CHECK

Q1

When may you compute the test score of your capstone?

A) After every choice (features, model, hyperparameters, threshold or list size) is made on the training data, once

B) After each improvement, to see whether it helped

C) Before tuning, to know where you stand

D) Whenever, as long as you report the best one

Why: The test set estimates how the finished model does on new data. Every look you act on turns it into training data. If you peeked, say so in the card.
B is wrong: That is tuning on the test set (Common Mistake 4).
C is wrong: Then the tuning is guided by it.
D is wrong: The best of many looks is luck, not performance.

Q2

What does this print?

```
import numpy as np
print(np.percentile(np.arange(1, 101), [2.5, 97.5]))
```

`[ 3.475 97.525]`

Why: The 2.5th and 97.5th percentiles of 1 to 100, with linear interpolation: 3.475 and 97.525. A bootstrap interval reads the same two percentiles from 1,000 resampled scores.

Q3

June average precision: tuned boosting 0.556, logistic regression 0.539; the paired bootstrap interval of the difference is -0.046 to +0.077. What goes in the card?

A) Boosting is better: 0.556 is higher

B) Logistic regression is better: it is simpler

C) The two cannot be told apart on 64 June frauds; choose on other grounds (simplicity, explainability, cost) and say so

D) Neither beats the baseline

Why: The interval includes 0: the data cannot rank them. The capstone lab's numbers: +0.017 (-0.046 to +0.077). Both are far above the no-model 0.008.
A is wrong: A higher point estimate inside the noise is not evidence.
B is wrong: Possibly the right choice, but not because of these numbers: say the real reason.
D is wrong: Both are around 0.54 against 0.008.

Q4

Demand: resampling store-product ROWS gives a 95% interval of 4.90 to 5.53 units saved; resampling whole WEEKS gives 4.68 to 5.81. Which belongs in the card?

A) The row interval: it uses more data points

B) Neither: intervals are optional

C) The narrower one, whichever it is

D) The week interval: rows in the same week share the same conditions, so they are not independent

Why: Rows within a week move together (season, holidays), so treating 4,896 rows as independent makes the interval too narrow. The capstone lab: 4.68 to 5.81 over 51 weeks.
A is wrong: More rows are not more independent evidence.
B is wrong: The rubric's Evaluation row asks for one.
C is wrong: Choosing the interval by width is choosing by wishful thinking.

Q5

Review triage card, first line of Performance. Which is strongest?

A) The model is 87% accurate

B) On 774 held-out reviews the model finds 87% of the negative reviews (the baseline finds none), at accuracy 0.873 (95% CI 0.85 to 0.895) against 0.745

C) The model uses TF-IDF bigrams and balanced logistic regression

D) More data would improve the model

Why: It leads with the business meaning (complaints found), gives the baseline on the same test set and an interval. Numbers from the model-card lab.

A is wrong: No baseline, no interval, and accuracy is not the costly error.

C is wrong: How it works, not how well.

D is wrong: A non-limitation.

Q6

Your segment with the highest churn (about 32%) uses discounts on half its spending. Which sentence may go in the card?

A) Discounts cause these customers to leave

B) Stop discounting to this segment

C) This segment's model was trained on churn

D) Customers in this segment churned more often (about 32%); the segments describe behavior, not why customers leave

Why: The segments were built without the churn column and checked against it afterwards: a description. A causal claim needs an experiment.

A is wrong: A cause the design cannot support.

B is wrong: A decision built on that cause.

C is wrong: Clustering never saw churn: that is what makes the check fair.

Q7

What does this print? (Errors by subgroup.)

```
import pandas as pd
err = pd.DataFrame({"store": ["Bend", "Bend", "Boise", "Boise", "Boise"],
                    "abs_error": [2.0, 4.0, 10.0, 0.0, 5.0]})
print(err.groupby("store")["abs_error"].mean().to_dict())
```

```
{'Bend': 3.0, 'Boise': 5.0}
```

> **Why:** Bend: (2 + 4) / 2 = 3.0; Boise: (10 + 0 + 5) / 3 = 5.0. A subgroup table is exactly this, with the baseline's errors beside it.

Q8

Your demand model's MAE is lowest for Snow (5.8 units) and highest for Footwear (12.5). What should the card say?

A) Snow is forecast best

B) Compare each group's error with its baseline and its typical volume: Snow sells about 12 units a week, Footwear about 46

C) Drop Footwear from the model

D) Nothing: subgroup errors are noise

> **Why:** Absolute errors follow volume. Against the per-product mean, both groups improve (Snow 13.3 → 5.8, Footwear 18.7 → 12.5); relative to volume, Snow's error is not the smallest.
>
> **A** is wrong: Low sales give low absolute errors.
>
> **C** is wrong: Footwear also improves on the baseline.
>
> **D** is wrong: The checklist asks for them; read them with their baselines.

Q9

Which figure may go into a public portfolio repository?

A) Hit rate at 10 by customer history length, for the model and the baseline

B) A table of ten customers with their recommended products

C) The twenty worst forecast errors with store and product IDs

D) Customer IDs and fraud scores for June

> **Why:** Aggregates only: no customer-level predictions, IDs or purchase histories leave the notebook (the checklist's privacy item), even with fictional data, because the habit is the point.
> **B** is wrong: Purchase histories are personal data.
> **C** is wrong: Row-level output; fine in the notebook, not published.
> **D** is wrong: Customer-level predictions.

Q10

In Problem E, what should you do with `age_group`?

A) Add it to the clustering so segments are easier to target

B) Delete it from the data file

C) Keep it out of the clustering, then check whether the segments differ by age

D) Use it to name the segments

> **Why:** The lesson keeps demographic columns out of the clustering and checks afterwards: segments describe behavior, and any age imbalance is visible and reportable.
> **A** is wrong: That builds the segments on a sensitive attribute.
> **B** is wrong: You need it for the check.
> **D** is wrong: Names come from behavior profiles.

Q11

After a final rerun, your card says accuracy 0.877 but the notebook prints 0.873. What is the best fix for good?

A) Change the notebook's seed until it prints 0.877

B) Round both to 0.88

C) Delete the number from the card

D) Write the card from the results (results.json and an f-string), and check every card number against it

> **Why:** The model-card lab does exactly this, and catches a hand-typed number: '0.877'.
> **A** is wrong: That is choosing the seed by the result.
> **B** is wrong: It hides the mismatch.
> **C** is wrong: A card without numbers fails Communication.

Q12

Your three-minute showcase. Which outline fits?

A) The decision and the recommendation, the main number against the baseline with its interval, one specific limitation, pointing at one figure

B) Every model you tried, in order

C) The code, cell by cell

D) The rubric scores you gave yourself

> **Why:** The showcase is the model card spoken aloud: recommendation first, evidence against the baseline, one honest limit. "Do not deploy" with evidence is a full showcase.
> **B** is wrong: That is the log, not the story.
> **C** is wrong: The notebook is the scouting trip.
> **D** is wrong: Self-scoring is for the hand-in.

PREDICT, THEN RUN

P1

Predict the output, then run it.

```
import numpy as np
scores = np.array([0.9, 0.1, 0.4, 0.8, 0.2, 0.05, 0.3, 0.6, 0.7, 0.15])
held = scores >= np.quantile(scores, 0.8)
print(held.sum(), scores[held].tolist())
```

```
2 [0.9, 0.8]
```

> **Why:** The 80th percentile of the ten scores is 0.72; two scores (0.9 and 0.8) are at or above it. A capacity of 1% is the same idea with 0.99.

P2

Predict the output, then run it.

```
import numpy as np
diffs = np.array([0.02, -0.01, 0.03, 0.00, 0.05])
print(round(float(diffs.mean()), 3), bool(np.percentile(diffs, 2.5) < 0))
```

```
0.018 True
```

> **Why:** The mean difference is 0.018, but the 2.5th percentile is below 0: an interval that includes zero means the data cannot say which model is better.

LAB REFERENCE SOLUTIONS

- CAPSTONE LAB 3: LABS/MODULE-13/L13_2-CAPSTONE/BOOTSTRAP_CI_STARTER.PY → BOOTSTRAP_CI_SOLUTION.PY

The solution prints:

```
June: 8,262 payments, 64 frauds; no-model AP 0.008
logistic regression  AP +0.539  95% CI +0.408 to +0.658
tuned boosting       AP +0.556  95% CI +0.423 to +0.668
boosting minus LR    AP +0.017  95% CI -0.046 to +0.077
boosting, top 1%: 35 of 64 frauds held (recall 0.55, 95% CI 0.43 to 0.68)

demand 2025: model MAE 8.19, per-product mean 13.42, saved 5.23 units
95% CI, rows resampled:  4.90 to 5.53  (4,896 rows treated as independent)
95% CI, weeks resampled: 4.68 to 5.81  (51 weeks: the honest unit)
```

Reading it: with 64 frauds, a single average precision has an interval about 0.25 wide; the paired difference between two good models straddles zero, so the card cannot rank them and should choose on other grounds. The capacity recall (0.55, 0.43 to 0.68) is the number the review team understands. On demand, resampling weeks widens the interval by about half: the honest unit.

- CAPSTONE LAB 4: LABS/MODULE-13/L13_2-CAPSTONE/MODEL_CARD_STARTER.PY → MODEL_CARD_SOLUTION.PY

The solution prints:

```
tfidf-1-2gram-lr {'cv_accuracy': 0.86, 'cv_neg_recall': 0.57}
tfidf-1-2gram-lr-balanced {'cv_accuracy': 0.875, 'cv_neg_recall': 0.849}
chosen: tfidf-1-2gram-lr-balanced
{"accuracy": 0.873, "accuracy_ci": [0.85, 0.895], "neg_recall": 0.873} baseline 0.745
types to review: []
reloaded model predicts the same: True
# Model card: review triage, version 1

## Recommendation
Use it to sort incoming 1-2 and 4-5 star reviews so the product team reads likely compl

## Performance
On 774 held-out reviews: accuracy 0.873 (95% CI 0.85 to 0.895)
against 0.745 for "always positive"; it finds 87% of the negative reviews (the baseline

## Limitations
Trained without 3-star reviews; their scores are not validated. Star ratings that contr

6 numbers in the card; not traceable to results.json: []
a card typed from memory: ['0.877']
```

Reading it: the choice is made on CV negative recall (the costly error), the test is scored once with an interval, `results.json` holds every number, the reloaded skops model predicts identically (a TF-IDF + logistic regression pipeline needs no extra trusted types), and the card is generated from the dictionary, so a hand-typed number is caught.

- HANDOUT: LABS/MODULE-13/L13_2-CAPSTONE/DEBUG_CAPSTONE.PY (A CLASSMATE'S CAPSTONE TO REVIEW)

As shipped it prints:

```
best learning rate 0.1: test MAE 7.60
baseline (predict the average week): MAE 16.98
MODEL CARD: our forecast beats the baseline by 55%.
```

1. **A random split of weekly rows** (`train_test_split` on time-ordered data): Data and leakage, Evaluation. Use the template's 2024 / 2025 split.

2. **The actual weekly temperature as a feature:** unknown when the order is placed (Data and leakage). Here it changes little, which is itself a finding.
3. **The learning rate chosen on the test set:** Evaluation (one look). Choose it with rolling folds on 2024.
4. **A baseline that is too weak** (the overall mean, 16.98, instead of the template's per-product mean) and no interval: Evaluation and Communication.

What each mistake is worth, one at a time (same model):

```
random split, with temperature : 7.60
random split, no temperature   : 7.74
2024 -> 2025, with temperature : 8.31
2024 -> 2025, no temperature   : 8.29
overall-mean baseline on 2025  : 17.13
```

All four fixed (template split, no actual temperature, learning rate chosen by CV on 2024, the template baseline):

```
CV MAE by learning rate: {0.03: 12.42, 0.05: 12.06, 0.1: 11.66, 0.2: 11.72, 0.3: 11.83}
learning rate 0.1 (chosen by CV on 2024): 2025 test MAE 8.29
baseline (template, per-product mean): MAE 13.42
improvement over the baseline: 38%
```

The honest model still beats the honest baseline clearly, by 38% rather than 55%: a better story, because it will hold next year.

THE LESSON PAGE'S OWN QUICK QUIZ (13.2)

1. **B** Replace the actual temperature with something known three weeks ahead.
2. **C** What the review team would catch at its capacity, with average precision next to the no-model baseline.
3. **A** `[nan, nan, 15.0, 25.0]`.
4. **B** The model relies heavily on discount share; this describes the model, not why customers leave.
5. **C** Check stability across seeds and subsamples, and distinct, nameable profiles.

EXIT TICKET

1. A metric with its interval next to the baseline on the same test set, in the business's units where possible.
2. Specific limits: the size of the test set, a group the model serves badly, an input that is unvalidated (3-star reviews, new products, new stores), a quirk (stockout zeros).

3. The rest of the card; then save with skops, results.json, requirements.txt; Restart and Run All; check card numbers; self-score; submit.

COMMON WRONG ANSWERS AND WHAT THEY REVEAL

- **Q3 A.** Reads point estimates without intervals: the most common Evaluation weakness.
- **Q4 A.** Thinks more rows means more evidence; ask what the rows have in common.
- **Q8 A.** Compares absolute errors across groups of different volume.
- **Q11 A.** Seed-shopping: the clearest sign the card is being fitted to the notebook instead of written from it.